

Tema 1: Infraestructura como código (laC)



Introducción



1.1 ¿Qué es la laC y por qué es importante?



1.2 Ciclo de vida de la laC



1.3 Buenas prácticas en laC

Introducción

En la primera parte de este módulo, nos enfocaremos en la **Infraestructura como Código (IaC)**, una práctica fundamental en DevOps que permite describir y gestionar la infraestructura de manera programática. Exploraremos en detalle qué es la IaC, por qué es tan importante y cómo su ciclo de vida se integra en el ecosistema DevOps. Además, revisaremos las mejores prácticas y patrones de diseño para una implementación exitosa de la IaC.

Posteriormente, profundizaremos en **Terraform**, una herramienta líder en la gestión de la infraestructura como código. Aprenderemos los fundamentos de Terraform, desde su instalación y tipos de bloques hasta los comandos básicos y avanzados. Además, estudiaremos la estructura de un proyecto de Terraform y exploraremos prácticas avanzadas como la automatización del flujo de trabajo, la gestión del estado y la modularización del código.

Continuaremos nuestro viaje explorando la automatización y orquestación de redes en el contexto DevOps. Analizaremos la importancia de la automatización de redes, los conceptos básicos y cómo Terraform puede ser utilizado para automatizar la creación y gestión de recursos de red, como redes virtuales, tablas de rutas y grupos de seguridad.

Una vez comprendida esta parte, nos enfocaremos en la **plataforma de nube pública Azure**, estudiando su estructura general, la selección de regiones y zonas de disponibilidad, la administración de suscripciones y costes, y los principales recursos disponibles, como máquinas virtuales, almacenamiento, redes y bases de datos.

Luego, exploraremos las herramientas y estrategias para realizar despliegues en Azure, incluyendo una introducción al Portal de Azure y a la CLI de Azure. Además, estudiaremos las

plantillas ARM y el lenguaje Bicep, que permiten automatizar y gestionar la infraestructura en Azure.

Finalmente, profundizaremos en Azure DevOps Services, una plataforma integral que abarca la gestión de proyectos ágiles, el control de versiones, la automatización de la integración y entrega continuas, la gestión de artefactos y las pruebas. Exploraremos cada uno de los componentes clave de Azure DevOps y cómo pueden ser utilizados para implementar prácticas DevOps en tu organización.

A lo largo de este módulo, adquirirás una comprensión profunda de DevOps, desde sus principios fundamentales hasta su aplicación práctica en el mundo real. A través del estudio de casos de éxito y la exploración de las últimas tendencias, obtendrás las herramientas y conocimientos necesarios para aplicar DevOps en tu organización, mejorando la colaboración entre equipos, la eficiencia en los procesos y la calidad del producto final. Este conocimiento te preparará para enfrentar los desafíos actuales del desarrollo de software y te posicionará a la vanguardia de la innovación tecnológica.

En este segmento del curso, nos adentraremos en el concepto revolucionario de la Infraestructura como Código (IaC), una metodología que está redefiniendo los paradigmas tradicionales de gestión y configuración de infraestructuras en el ámbito de la tecnología de la información. La IaC emerge como una respuesta a la creciente demanda de eficiencia, rapidez y escalabilidad en el desarrollo y operaciones de sistemas, especialmente en el contexto del *cloud computing*, donde la capacidad de gestionar recursos de manera programática y a gran escala se ha convertido en un requisito indispensable.

La Infraestructura como Código permite a los equipos de desarrollo y operaciones tratar **los recursos de TI** -tales como servidores, redes y almacenamiento- de la misma manera que el código fuente de las aplicaciones. Esto significa que se pueden utilizar herramientas como Terraform para escribir, probar y mantener la infraestructura de manera sistemática y repetible, lo que representa un cambio fundamental respecto a los enfoques manuales y propensos a errores del pasado. La adopción de IaC no solo mejora la eficiencia operativa y reduce el riesgo de errores humanos, sino que también asegura la consistencia entre los entornos de desarrollo, prueba y producción, acelerando así el ciclo de vida del desarrollo de software.

Este tema es de vital importancia porque aborda cómo la **laC** facilita la gestión de infraestructuras complejas y distribuidas, permitiendo a las organizaciones escalar y adaptarse rápidamente a las necesidades cambiantes del mercado. A través de la exploración de **Terraform y su lenguaje HashiCorp Configuration Language (HCL)**, profundizaremos en cómo definir, lanzar y gestionar infraestructuras de manera declarativa y predecible, lo que constituye una habilidad esencial en el panorama tecnológico actual.

Al finalizar este tema, los estudiantes estarán equipados con el conocimiento y las herramientas necesarias para implementar prácticas de laC en sus propios entornos de trabajo. Esto no solo mejorará la colaboración entre los equipos de desarrollo y operaciones, sino que también optimizará los procesos y mejorará la calidad del producto final. La comprensión y aplicación de la Infraestructura como Código es, por lo tanto, un paso crucial para cualquier profesional que busque liderar e innovar en el campo de DevOps, preparándolos para enfrentar los desafíos actuales y futuros en el desarrollo y gestión de infraestructuras tecnológicas.

Objetivos

- **Aprender** los conceptos fundamentales de la Infraestructura como Código (laC) y su importancia en el contexto DevOps.
- **Explorar** la herramienta Terraform, sus fundamentos, estructura de proyectos y prácticas avanzadas para la descripción y gestión de la infraestructura en la nube.
- **Estudiar** la automatización y orquestación de redes en entornos DevOps, incluyendo el uso de Terraform para la creación y gestión de recursos de red.
- **Conocer** la plataforma de servicios en la nube Azure, su estructura, suscripciones, grupos de recursos y principales recursos como máquinas virtuales, almacenamiento, redes y bases de datos.
- **Analizar** las diferentes herramientas y estrategias para realizar despliegues en Azure, incluyendo el uso del portal web, la interfaz de línea

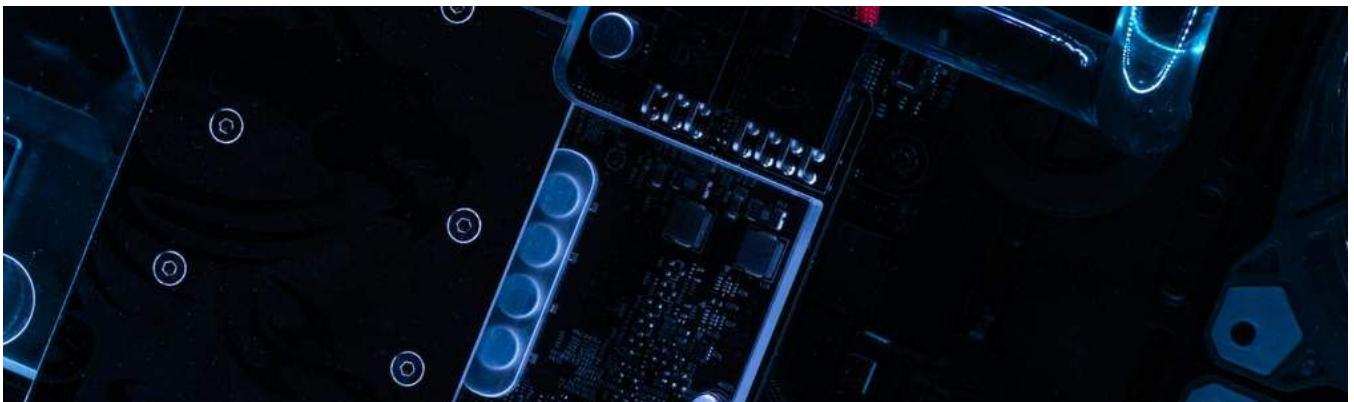
de comandos (CLI) y las plantillas de Azure Resource Manager (ARM).

- **Explorar** la plataforma de Azure DevOps Services, sus componentes y funcionalidades clave como la gestión ágil de proyectos, el control de versiones avanzado, la automatización de la integración y entrega continua, y las pruebas.
- **Estudiar** las capacidades de monitoreo y registro (logging) en Azure, incluyendo métricas, logs y trazas, así como las herramientas disponibles como Azure Application Insights y Azure Log Analytics.

1.1 ¿Qué es la IaC y por qué es importante?

En el campo de la informática, la gestión y configuración de infraestructura han evolucionado significativamente. La necesidad de eficiencia, rapidez y escalabilidad ha llevado al desarrollo de prácticas innovadoras, como **la infraestructura como código (IaC)**. Esta metodología es crucial en la era del *cloud computing*, donde la capacidad de gestionar recursos de forma programática y a gran escala no es una ventaja, sino una necesidad.

La infraestructura como código (IaC) es el **proceso de gestión y provisión de infraestructura a través de archivos de definición legibles por máquina**, en lugar de configurar físicamente el hardware o las herramientas interactivas. Mediante este enfoque, los equipos de desarrollo y operaciones pueden tratar la infraestructura de TI (servidores, máquinas virtuales, conexiones de red, etc.) de forma similar al código de una aplicación. Utilizando herramientas como Terraform, los ingenieros podemos **escribir, probar y mantener los componentes de la infraestructura de una manera sistemática y repetible**.



Antes de la llegada de la IaC, las empresas dependían de procesos manuales y de una documentación extensa para gestionar su infraestructura. Estos métodos daban lugar a numerosos errores, no eran escalables y resultaban ineficientes, sobre todo en entornos de

rápido crecimiento. Con el surgimiento de la nube y las metodologías ágiles, se hizo evidente la necesidad de un enfoque más dinámico y escalable.

1.1.1 Ventajas del uso de IaC

Inicialmente, los scripts eran la principal herramienta para automatizar la configuración de servidores y otros componentes. La IaC introdujo un enfoque **estandarizado y versionable** que puede mantenerse y escalarse con mucha más facilidad.

Este proceso ofrece numerosas ventajas sobre los enfoques tradicionales:

- 1 Automatización:** Reduce la necesidad de intervención manual, lo que disminuye el riesgo de errores humanos y aumenta la eficiencia.
- 2 Consistencia:** Asegura que los entornos de desarrollo, prueba y producción sean idénticos, minimizando así los problemas de incompatibilidad.
- 3 Velocidad:** Permite a los equipos desplegar y modificar infraestructura con rapidez, imprescindible en prácticas ágiles y de integración continua.

Además, la implementación de la IaC también proporciona diversos beneficios tangibles:

- **Escalabilidad:** Facilita la gestión de infraestructura a gran escala, ideal para sistemas distribuidos y aplicaciones en la nube.
- **Replicabilidad:** Permite replicar entornos con precisión, algo indispensable para recuperaciones de desastres y clonación de entornos.
- **Reducción de costes:** Minimiza el coste operativo al reducir el personal técnico necesario para tareas de configuración y mantenimiento manual.

Los archivos de configuración de IaC no solo definen **qué recursos** se necesitan, sino también **cómo deben configurarse y cuándo deben desplegarse**. Gracias a esta capacidad para gestionar la infraestructura con la misma rigurosidad que el software, las organizaciones pueden sacar el

máximo partido a prácticas de desarrollo como el **control de versiones**, las **revisiones de código** y las **integraciones continuas**.

Ejemplo

Por ejemplo, un archivo de IaC puede incluir definiciones para la creación de redes virtuales, configuraciones de seguridad, políticas de acceso y la creación de máquinas virtuales. **Se define con claridad cada uno de los componentes para que cualquier miembro del equipo pueda entender y modificar la infraestructura como si fuera código.**

La automatización mediante IaC puede reducir el tiempo de despliegue de semanas a minutos, **permitiendo a las empresas responder rápidamente a las necesidades del mercado**. La consistencia en los entornos **elimina los errores comunes que se producen al trabajar con configuraciones manuales y las diferencias entre entornos** de prueba y producción.

1.1.2 Impacto en los roles de la organización

Con la IaC, **los operadores de sistemas comienzan a adoptar nuevas prácticas de desarrollo, mientras que los desarrolladores profundizan en la comprensión de la infraestructura**. Esto lleva a un modelo DevOps, en el que la creación, el despliegue y la gestión de infraestructura son responsabilidades compartidas. La cultura DevOps gira en torno a la colaboración, como vimos en los primeros temas del curso.



Es obvio que cada organización es diferente, y no es lo mismo una gran empresa en la que compartamos equipo con 15 personas, que una pequeña en la que nuestro equipo esté formado solo por dos miembros, por lo que nuestras responsabilidades variarán de acuerdo con el entorno.

En este curso nos centraremos en Terraform, herramienta desarrollada por HashiCorp, líder en el ámbito de IaC. Esta herramienta utiliza archivos de configuración que describen la infraestructura deseada, lo que **permite a los usuarios definir recursos de manera clara y declarativa**.

Terraform se encarga crear, cambia y gestiona la infraestructura con un enfoque predictivo, identificando los cambios necesarios para alcanzar el estado deseado sin necesidad de intervención manual. Veremos Terraform y su lenguaje HCL (Hashichorp Configuration Language) con más detalle en los próximos temas.

1.2 Ciclo de vida de la IaC

El ciclo de vida de la infraestructura como código comprende varias etapas críticas, todas ellas diseñadas para facilitar la gestión eficiente y eficaz de la infraestructura. Las etapas son **codificación, planificación, aplicación y destrucción** (en su caso), y cada una contribuye a un flujo de trabajo continuo y optimizado que soporta operaciones ágiles y fiables.



1.2.1. Codificación

La codificación es la base del ciclo de vida de la IaC. En esta etapa, la infraestructura necesaria se define utilizando archivos de configuración escritos en formatos que pueden ser tanto legibles por humanos como por máquinas.

La codificación en el contexto de IaC se refiere al proceso de **escritura de scripts o definiciones de configuración** que describen exactamente cómo se debe configurar y gestionar la infraestructura.



Estos scripts utilizan lenguajes específicos como HashiCorp Configuration Language (HCL) para Terraform, YAML para Ansible, o JSON para AWS CloudFormation.

Un diseño bien pensado en la etapa de codificación puede marcar la diferencia entre una infraestructura robusta, escalable y segura, y otra propensa a fallos y difícil de mantener. Un diseño eficaz debe abordar aspectos como los siguientes:

- La modularidad.
- La reusabilidad de los componentes.
- La seguridad desde el principio.

IDEMPOTENCIA

DOCUMENTACIÓN Y CÓDIGO COMO DOCUMENTACIÓN

- Una característica fundamental de los scripts de laC es su **idempotencia**, la propiedad que garantiza **que ejecutar el script múltiples veces en el mismo entorno dará lugar al mismo resultado**, sin efectos secundarios inesperados ni configuraciones duplicadas.
- Además, es recomendable que nuestros scripts sean dinámicos, ya que trabajaremos con variables cuyos valores se definirán en ficheros separados por entornos. Gracias a ello podremos aplicar un mismo código a diferentes entornos, lo que supone ganar en fiabilidad y agilidad.

IDEMPOTENCIA

DOCUMENTACIÓN Y CÓDIGO COMO DOCUMENTACIÓN

- Los archivos de configuración en laC actúan no solo como medio para provisionar infraestructura, sino **también como documentación**. La sencillez a la hora de leerlos (no exige de que estén bien comentados) permite que cualquier miembro del equipo entienda cómo está configurada la infraestructura sin más que revisar estos archivos de configuración.

1.2.2. Planificación

La planificación es la fase en la que **se revisan y se validan las configuraciones propuestas antes de que se implementen**. Se trata de una etapa indispensable para evitar errores costosos y garantizar que los cambios se alineen con las políticas de seguridad y cumplimiento de la organización. Con Terraform, veremos que es posible hacer este paso de una manera transparente y comprensible.

Antes de implementar cualquier cambio, es importante evaluar su posible impacto en la infraestructura existente. Esto incluye **verificar la compatibilidad, la carga adicional en los recursos y las posibles interrupciones del servicio**.



Los planes generados por herramientas de IaC detallan los cambios que se aplicarán, permitiendo a los equipos revisar estos cambios antes de que se ejecuten. Así se garantiza que solo se realicen modificaciones aprobadas y controladas.

Muchas herramientas de IaC ofrecen funcionalidades para comparar el estado deseado con el estado actual, identificando las diferencias con claridad, lo que permite una evaluación precisa de los cambios necesarios.

Mediante la simulación de cambios antes de su implementación, se identifican posibles problemas en un entorno controlado y se minimizan los riesgos asociados con la implementación real.

1.2.3. Aplicación

La aplicación es **el acto de ejecutar los planes de configuración para materializar los cambios en la infraestructura real**. Esta fase es donde la teoría se convierte en práctica, y los archivos de configuración definen la infraestructura operativa. Todo lo que hemos definido se despliega, generando los cambios pertinentes tal como habíamos previsto. Suele ser una fase segura, que al

igual que la planificación **nos avisará de los cambios que vamos a realizar y de cómo afectan a nuestra infraestructura ya existente.**

- La automatización desempeña un papel protagonista en esta fase, porque gracias a ella los cambios se aplican de manera consistente y sin errores humanos.
- La implementación automatizada también permite una rápida escalabilidad y dar respuesta a las necesidades del negocio.

La gestión de estado es crucial para asegurar que el estado de la infraestructura siempre refleje las especificaciones de los archivos de configuración. Esto incluye rastrear cada cambio aplicado y **garantizar que todos los estados sean coherentes en todas las implementaciones.**

```
aws_emrserverless_application.salesforce: Modifications complete after 1s [id=00fb6kg2h5ldol0p]
aws_lakeformation_permissions.dbt_salesforce_tb[0]: Creating...
aws_iam_role.lambda: Modifications complete after 0s [id=nt-iamrole-lambda-salesforce-materializ
aws_sfn_state_machine.salesforce: Modifying... [id=arn:aws:states:eu-west-1:162076964983:stateMa
aws_iam_policy.nt_iamrole_datalake_emr_policy: Modifications complete after 1s [id=arn:aws:iam:
aws_glue_crawler.bronze_salesforce: Modifications complete after 0s [id=nt-glue-crawler-salesfor
aws_sfn_state_machine.emr_dbt_salesforce: Modifications complete after 1s [id=arn:aws:states:eu-
aws_scheduler_schedule.aws_sfn_state_machine_emr_dbt_salesforce: Modifying... [id=default/nt-ev
aws_glue_crawler.silver_salesforce: Modifications complete after 0s [id=nt-glue-crawler-salesfor
aws_glue_catalog_database.nt_silver__salesforce: Modifications complete after 1s [id=16207696498
aws_lakeformation_permissions.dbt_salesforce_db[0]: Creation complete after 0s [id=2264092723]
aws_sfn_state_machine.salesforce_full_load: Modifications complete after 1s [id=arn:aws:states:
aws_lakeformation_permissions.dbt_salesforce_tb[0]: Creation complete after 0s [id=1928009543]
aws_glue_crawler.gold_salesforce: Modifications complete after 0s [id=nt-glue-crawler-salesforce
aws_scheduler_schedule.aws_sfn_state_machine_emr_dbt_salesforce: Modifications complete after 1s
aws_sfn_state_machine.salesforce: Modifications complete after 1s [id=arn:aws:states:eu-west-1:
Releasing state lock. This may take a few moments...

Apply complete! Resources: 2 added, 24 changed, 2 destroyed.
```

Figura 1. Fragmento del resultado de aplicar un código Terraform utilizando Github Actions.

Después de aplicar los cambios, **debemos verificar que la infraestructura funcione según lo previsto**. Para lo cual deberemos realizar pruebas de salud, verificaciones de seguridad y auditorías de cumplimiento.



Y, en caso de que algo salga mal, con los mecanismos de *rollback* podemos restaurar la infraestructura a su estado anterior, minimizando así el impacto de los fallos.

1.2.4. Destrucción

La fase final del ciclo de vida de la IaC es la destrucción, que es tan importante como las fases de creación y mantenimiento de la infraestructura.

Deshacerse de la infraestructura que ya no necesitamos aumenta la eficiencia operativa y reduce costes. Las acciones clave incluyen las siguientes:

- **Racionalización de recursos:** Identificar y **desmantelar recursos que ya no son necesarios**, como servidores inactivos, almacenamiento no utilizado y redes redundantes. Esto contribuye a optimizar los costes y reduce la complejidad del entorno.

- **Procedimientos de destrucción seguros:** Implementar procedimientos que eliminen los recursos de forma segura, a fin de garantizar que toda la información sensible se desaloja o se destruye correctamente para evitar fugas de datos.
- **Mantener una documentación detallada de los procesos de destrucción y las verificaciones realizadas:** Debemos poder auditar y validar que la destrucción se ha llevado a cabo conforme a los estándares establecidos.

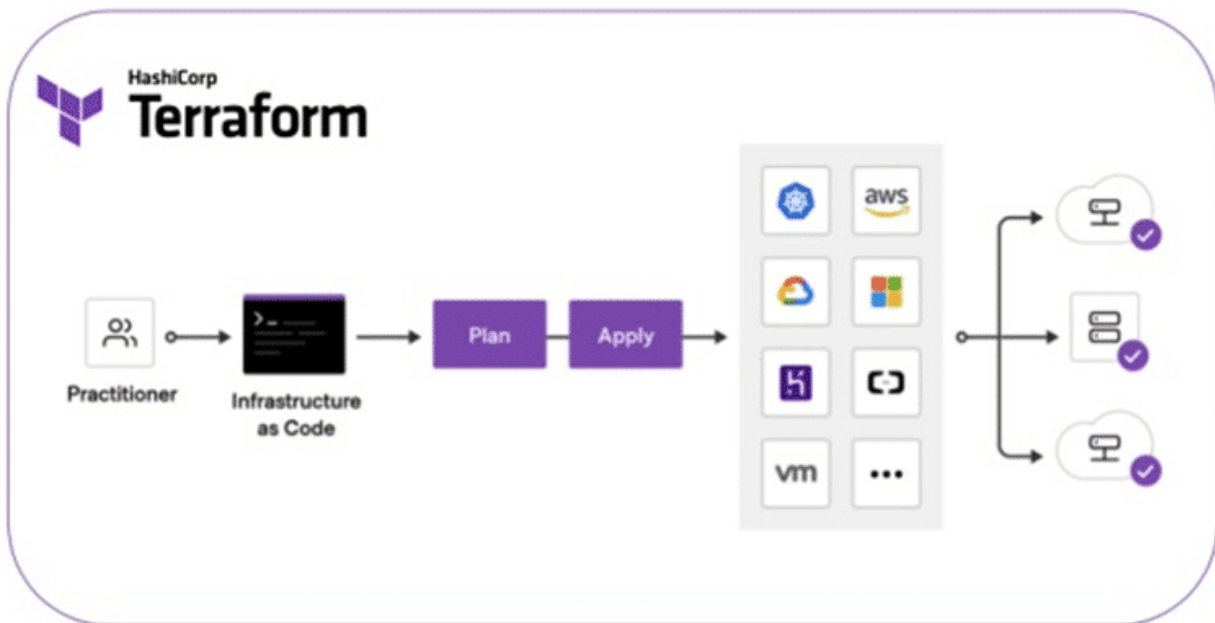


Figura 2. Terraform como herramienta para la automatización de infraestructuras

1.3 Buenas prácticas en IaC

Las buenas prácticas en IaC garantizan la **eficiencia, la seguridad y la escalabilidad** de las operaciones de infraestructura gestionadas a través de código. A continuación, veremos prácticas básicas, divididas en varias categorías esenciales.



1.3.1. Principios de codificación limpia y mantenible

La integración de los principios de código limpio y mantenible en las prácticas de DevOps puede generar resultados muy positivos para el desarrollo de software. Hay que tener en cuenta las siguientes características:

(A) Legibilidad y simplicidad

CLARIDAD EN LA CODIFICACIÓN

Es fundamental disponer de un código claro y bien estructurado. Utiliza nombres de variables y funciones descriptivos y consistentes a lo largo del código. Evita los nombres genéricos y comprueba que cada nombre refleja claramente su función.

COMENTARIOS Y DOCUMENTACIÓN

Es imprescindible añadir comentarios en el código para explicar ciertas decisiones. La documentación debe incluir guías de uso, descripciones de módulos y explicaciones de decisiones complicadas o no intuitivas.

Por ejemplo, en un código Terraform en el que estamos definiendo un grupo de seguridad para una máquina EC2, es importante dejar comentarios sobre el porqué de los rangos de IP que estamos permitiendo, para recordarlo más adelante con un simple vistazo al código, o en caso de que lleguen nuevos compañeros a trabajar con ese código.

```
# Definición de Security Group para una instancia EC2
resource "aws_security_group" "example_sg" {
  name           = "example-security-group"
  description    = "Security group for example EC2 instance"
  vpc_id        = "vpc-12345678" # Asegúrate de reemplazar con el ID de tu VPC

  # Permitir tráfico HTTP desde cualquier lugar
  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Permitir tráfico HTTPS solo desde redes internas
  ingress {
    description = "Allow HTTPS from internal networks"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["192.168.100.0/24", "192.168.101.0/24"]
  }

  # Permitir todo el tráfico saliente
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "Example Security Group"
  }
}
```

Figura 3. Código escrito en HCL (Terraform) en el que se define un grupo de seguridad para una máquina EC2.

(B) Reusabilidad y modularidad

DISEÑO MODULAR

Organiza tu código en módulos con una única responsabilidad. Esto **facilita la reutilización y mejora la capacidad de mantenimiento**. Cada módulo debe ser independiente y poder funcionar sin dependencias innecesarias.

PARÁMETROS Y VARIABLES

Es recomendable utilizar variables para personalizar los módulos sin necesidad de modificar el código y **definir estándares para las variables, a fin de garantizar que sean claras y consistentes**.

Estas prácticas nos ayudan a nosotros mismos, ya que tarde o temprano tendremos que volver a ese código. Aunque nos parezca que en el momento entendemos cómo funciona todo, es probable que pasados unos meses no lo recordemos bien, y además **ayudan a otros compañeros** que necesiten trabajar con él.

1.3.2. Seguridad en la Gestión de secretos

La Infraestructura como Código (IaC) ha revolucionado la forma en que se provisiona y gestiona la infraestructura de TI, aportando agilidad, eficiencia y consistencia a los procesos. Sin embargo, esta nueva forma de trabajar también introduce nuevos desafíos en materia de seguridad.

Se debe prestar especial atención a la **Gestión de secretos**, utilizando los siguientes recursos o herramientas:

(A) Herramientas de gestión de secretos

Las herramientas dedicadas a gestionar secretos y credenciales son imprescindibles. Contribuyen a evitar que los secretos se almacenen en el código y proporcionan un control más seguro y centralizado. Servicios como **AWS Secrets Manager** pueden ayudarnos en esta tarea, alojando secretos que podremos utilizar en nuestros códigos y permitiéndonos incluso establecer políticas de rotación.



Programáticamente accederemos desde nuestro código a esos secretos, por supuesto, sin tener que exponer sus valores.

(B) Accesos controlados

Se considera una buena práctica configurar las herramientas de IaC y scripts para operar con el mínimo nivel de acceso necesario. Esto **minimiza los riesgos en caso de que se expongan las credenciales o si los scripts se ejecutan erróneamente**. Este principio del mínimo privilegio necesario se aplica también a casos en los que, por ejemplo, tengamos que dar accesos a usuarios. Es obvio que deben tener permisos para realizar el trabajo que necesitan, pero no más, siempre con el fin de minimizar los posibles errores humanos.

Ejemplo

Por ejemplo, en AWS trabajaremos con el IAM (Identity Access Manager), que nos permite gestionar los roles de usuarios y perfilarlos hasta su mínima expresión.

Podemos tener un caso de un usuario externo a nuestra organización que solo necesita acceso a un *bucket* de S3 (S3 es el sistema de almacenamiento de ficheros de AWS) y dentro del mismo a una ruta concreta para almacenar periódicamente unos archivos .CSV. Gracias al IAM podremos crear un usuario que solamente le permita hacer esta operación, por lo que si intentara acceder a otro *bucket* distinto no tendría permisos.

Figura 4. Ejemplo de Terraform donde definimos una política para un rol determinado que solo puede acceder a un path concreto.

```

resource "aws_iam_role_policy" "external_user_policy" {
  name = "external_user_policy"
  role = aws_iam_role.external_user_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "s3:GetObject",
          "s3:ListBucket"
        ]
        Resource = [
          "arn:aws:s3:::example-bucket/specific-path/*", # Permite acceso a los objetos dentro del path
          "arn:aws:s3:::example-bucket" # Esto es necesario para poder listar el bucket
        ]
        Condition = {
          Stringlike = {
            "s3:prefix" = [
              "specific-path/*" # Restringe el listado a un 'subdirectorio' específico
            ]
          }
        }
      }
    ]
  })
}

```

1.3.3 Gestión de configuraciones y cambios: Control de versiones

El control de versiones es un componente fundamental de la gestión de configuraciones y cambios (CCM) en Infraestructura como Código (IaC). Permite realizar un seguimiento de los cambios realizados en la infraestructura, facilitando la reversión a versiones anteriores en caso de errores o problemas.

- **Uso de sistemas de control de versiones:** Debemos mantener todos los archivos de configuración bajo control de versiones. Esto nos **facilitará la trazabilidad de todos los cambios y nos permitirá revertir rápidamente a versiones anteriores en caso de problemas.**
- **Flujos de trabajo de revisión de código:** Implementaremos flujos de trabajo de revisión de código para todas las modificaciones en la infraestructura. De esta manera garantiremos **que los cambios sean revisados y aprobados por otros antes de implementarlos.**
En entornos colaborativos, es buena práctica que otro compañero revise y apruebe los cambios antes de pasarlos a la rama principal o desplegarlos.



Pruebas automáticas: Podemos desarrollar y mantener un conjunto de pruebas automatizadas que validen la funcionalidad y seguridad de la infraestructura. Se pueden considerar pruebas de unidad para módulos individuales y **pruebas de integración para asegurar que todos los componentes funcionen correctamente en conjunto.**

Aun así, veremos que herramientas como Terraform son muy seguras, ya que nos avisan en todo momento de los cambios que se van a realizar en los despliegues.

1.3.4. Mejores prácticas operativas

Podemos utilizar herramientas avanzadas de monitorización para observar el comportamiento de la infraestructura en tiempo real. Esto incluye la monitorización de la salud del sistema, la utilización de recursos y los patrones de tráfico. Además, deberemos comprobar que los sistemas de registro estén configurados para capturar información detallada sobre todos los eventos y cambios, puesto que sin ellos no podremos resolver problemas ni llevar a cabo auditorías de seguridad.

Hay que añadir que, como vimos durante los primeros temas del curso, la clave está en la **colaboración entre los equipos de desarrollo y operaciones.** Podemos organizar sesiones de capacitación y talleres periódicos para que todas las personas del equipo estén al día en lo relativo a las mejores prácticas y nuevas herramientas en laC.

Tema 2: Terraform: Descubriendo la nube



Introducción



2.1 Fundamentos de Terraform



2.2 Estructura de un proyecto de Terraform



2.3 Prácticas avanzadas y automatización

Introducción

En este capítulo del curso, nos embarcaremos en un viaje a través de Terraform, una herramienta de Infraestructura como Código (IaC) desarrollada por HashiCorp, que ha revolucionado la forma en que desplegamos y gestionamos infraestructuras en la nube. Terraform utiliza un lenguaje de configuración declarativo, conocido como **HashiCorp Configuration Language (HCL)**, permitiendo a los usuarios definir y provisionar infraestructura en múltiples proveedores de servicios en la nube con un solo lenguaje. Esta capacidad de trabajar a través de **plataformas como AWS, Azure y Google Cloud**, con una única herramienta, simplifica enormemente nuestras tareas de gestión de infraestructura, haciéndolas más eficientes y predecibles.

La relevancia de Terraform en el mundo actual de DevOps y cloud computing es indiscutible. Nos permite automatizar la creación, configuración y gestión de infraestructuras de manera eficiente, reduciendo el potencial de errores humanos y asegurando una mayor consistencia y reproducibilidad en los entornos de TI. Además, su enfoque en la infraestructura como código no solo facilita la automatización, sino que también mejora la colaboración entre los equipos de desarrollo y operaciones, al permitir que la infraestructura sea versionada y revisada como cualquier otro código.

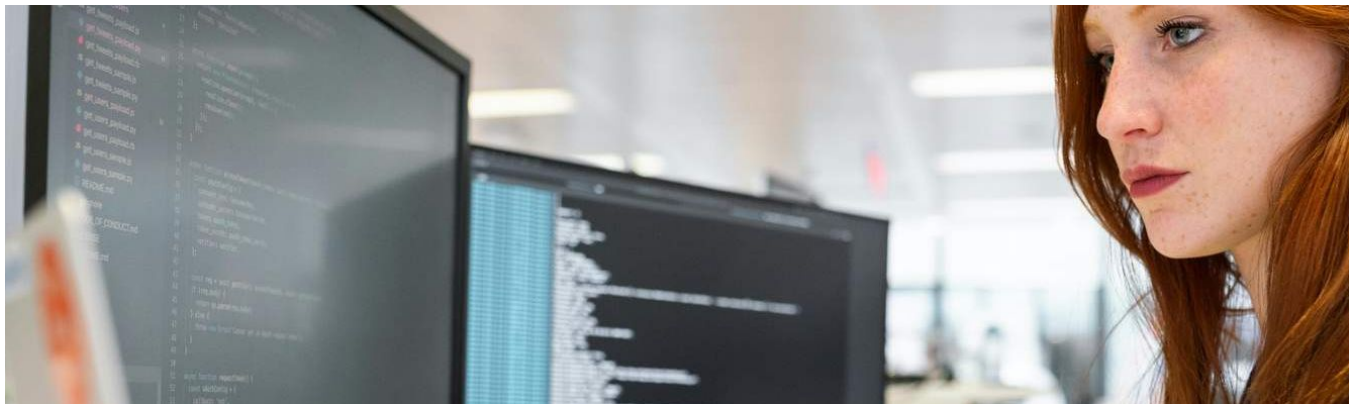
Este capítulo profundizará en los fundamentos de Terraform, explorando sus conceptos clave como *providers*, *resources* y *state management*. A través de ejemplos prácticos, aprenderemos cómo instalar Terraform, escribir y aplicar configuraciones básicas, y cómo estructurar un proyecto de Terraform para maximizar su eficacia y mantenibilidad. Además, discutiremos prácticas avanzadas y estrategias de automatización que pueden llevar nuestra gestión de infraestructura al siguiente nivel.

Al finalizar este capítulo, estarás equipado con el conocimiento y las habilidades necesarias para utilizar Terraform en la creación y gestión de infraestructuras en la nube de manera efectiva. Este conocimiento no solo te preparará para enfrentar los desafíos actuales en la gestión de infraestructuras modernas, sino que también te posicionará a la vanguardia de la innovación tecnológica, mejorando la eficiencia, la escalabilidad y la seguridad de tus proyectos de TI.

2.1 Fundamentos de Terraform

Terraform es una herramienta de infraestructura como código desarrollado por HashiCorp.

Permite a los usuarios **definir y provisionar infraestructura utilizando un lenguaje de configuración declarativo**, conocido como HashiCorp Configuration Language (HCL).



A lo largo de los años, Terraform ha ganado una popularidad notable gracias a su **flexibilidad, portabilidad y capacidad de gestionar múltiples proveedores de servicios en la nube simultáneamente** (puede que en nuestra empresa trabajemos con Azure y AWS, por ejemplo). Es una herramienta que automatiza la creación, los cambios y las mejoras de la infraestructura de manera eficiente y predecible. Además, es compatible con los proveedores cloud más utilizados, lo cual simplifica nuestras tareas, ya que solo tenemos que aprender un único lenguaje para desplegar, por ejemplo, en AWS, Azure o Google Cloud.



Figura 5. Logo de Terraform

Como ya vimos en el tema anterior, Terraform se basa en el principio de infraestructura como código, que permite gestionar la infraestructura a través de archivos de configuración. Lo que permite **una automatización, una reproducibilidad y una gestión de la infraestructura que sería imposible con los enfoques manuales.**

Dicho esto, a continuación, debemos mencionar estos tres conceptos clave de Terraform y tenerlos en cuenta para más adelante.

- **Providers:** Los providers en Terraform son plugins que permiten interactuar con API de proveedores de servicios. Por ejemplo, el provider de AWS permite gestionar recursos como instancias EC2, VPC y políticas de IAM
- **Resources:** Los resources son los componentes más importantes de los archivos de Terraform. Representan elementos individuales de la infraestructura, como una instancia de servidor, una red o un volumen de almacenamiento.
- **State Management:** Terraform mantiene un estado de la infraestructura que gestiona a través de archivos de estado. Esto le permite mapear recursos reales a su configuración y realizar cambios de manera eficiente. Lo explicaremos con más detalle en este tema.

```
# Configura el provider para AWS
provider "aws" {
  region = "eu-west-1" #La región en la que desplegamos nuestra infraestructura
}

# Define un recurso para una instancia EC2
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"
  tags = {
    Name = "CursoDevOpsInstance"
  }
}
```

Figura 6. Configuración de un provider y bloque de un resource

En la imagen superior vemos dos bloques. En el primero configuramos un provider, en este caso AWS. El segundo es un bloque de recurso, en el que definimos una configuración muy básica para una instancia EC2.

2.1.1 Instalación de Terraform

Para un **entorno Windows**, lo más sencillo es que **descarguemos el ejecutable desde la página oficial**. Pero en el caso de **macOS o de Ubuntu** ejecutaremos los siguientes comandos:

```
# macOS
brew install terraform

# Ubuntu
sudo apt-get update && sudo apt-get install terraform
```

Figura 7. Comandos de instalación de Terraform

RECUERDA

Terraform es una herramienta que automatiza la creación, los cambios y las mejoras de la infraestructura de manera eficiente y predecible.

2.1.2 Tipos de bloques en Terraform

Aunque ya avanzamos alguno en los conceptos clave, sin duda el más importante es el recurso (resource). Por supuesto, todos los bloques son importantes, pero el recurso es el que más vamos a utilizar.

- **A. Provider:** Como ya vimos, este bloque define los proveedores de servicios (como AWS, Google Cloud, etc.) Y especifica la configuración necesaria para interactuar con ellos. Solo hace falta declararlo una vez.
- **B. Resource:** Representa un objeto de infraestructura que Terraform creará, modificará o destruirá. Cada recurso se identifica por un tipo y un nombre local. Si volvemos al ejemplo que vimos antes, con la máquina EC2, veremos que el bloque tiene dos nombres: 'aws_instance', que es el tipo de recurso (este nombre es inmutable) y 'example', que es el nombre que le estamos dando al bloque. Este último no tiene relevancia fuera de nuestro código (simplemente lo utilizaremos para referenciar ese bloque en otras partes), así que podemos nombrarlo como queramos.

- **C. Data:** Permite a Terraform usar información generada o gestionada fuera de Terraform, como una AMI existente o datos de otra infraestructura. Con este bloque llamaremos a un recurso ya existente en nuestra infraestructura que no estamos definiendo en el código en el que estamos trabajando. Imaginemos que queremos acceder al ARN (identificador) de una máquina EC2 que tenemos en una cuenta de AWS, pero que no estamos desplegando en el código actual (se creó manualmente o desde otro repositorio); para esto utilizaríamos un bloque data.
- **D. Variable:** Permite la parametrización de configuraciones para facilitar la reutilización y la adaptabilidad sin modificar el código fuente directamente. Las variables se pueden definir con tipos específicos y valores por defecto. En estos bloques declaramos las variables que vamos a utilizar en el código, y por lo general (aunque Terraform no lo requiere), declararemos todas las variables en un fichero separado, para mantener una buena organización.
- **E. Output:** Se usa para extraer información sobre los recursos configurados, es práctico para utilizar en otras partes de la configuración de Terraform o para integraciones externas.
- **F. Module:** Contenedores para grupos de recursos y configuraciones relacionadas que pueden reutilizarse en diferentes proyectos de Terraform.
- **G. Terraform:** Bloque específico que configura comportamientos y versiones específicas de Terraform necesarias para el proyecto. De manera similar al de provider, normalmente solo lo definimos una vez, para establecer la versión de Terraform que vamos a utilizar.

```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"
}

resource "aws_instance" "devops_instance" {
  ami          = "ami-0c55b159cbfafa1f0"
  instance_type = "t2.micro"
}

variable "region" {
  description = "The AWS region in which to deploy the instance."
  type        = string
  default     = "us-west-1"
}

output "instance_ip_addr" {
  # Vemos como se referencia aquí el bloque de la instancia
  value = aws_instance.devops_instance.public_ip
  description = "The public IP address of the instance."
}

```

Figura 8. Uso de los distintos tipos de bloques

En este código vemos el uso de varios de los bloques que hemos mencionado. Definimos un proveedor para AWS, configuramos una instancia EC2, declaramos una variable para la región y exportamos la dirección IP pública de la instancia como un valor de salida.

2.1.3 Comandos básicos de Terraform

Vamos a ver ahora cuáles son los comandos básicos que nos permite Terraform y para qué sirve cada uno, pues serán nuestro día a día, ya los utilizemos a través del terminal o los tengamos integrados en pipelines de despliegue. Los básicos y clave son: **init, plan, apply y destroy**.

- **A. Terraform init:** Este comando se usa para inicializar un directorio de trabajo que contiene archivos de configuración de Terraform. Prepara el directorio para otros comandos, instala los plugins necesarios para la ejecución y configura el backend si se especifica (el backend es donde guardaremos el archivo de estado, que veremos en la próxima sección). Es decir, 'terraform init' es el comando que prepara todo el entorno para después poder ejecutar los comandos que siguen.
- **B. Terraform plan:** Este otro comando crea un plan de ejecución, lo que significa que Terraform determina qué acciones son necesarias para alcanzar el estado deseado especificado en los archivos de configuración. Necesitamos este comando para verificar qué cambios va a realizar Terraform en nuestra infraestructura antes de aplicarlos realmente. En otras palabras: si ejecutamos este comando, nuestro terminal nos mostrará qué sucedería si desplegamos el código Terraform que hemos definido. Nos dirá, por ejemplo, que ese código va a generar una máquina EC2, una red privada virtual y un grupo de seguridad. Este 'terraform plan' no despliega nada, simplemente nos da información, por lo que podremos valorar si el resultado que nos muestra es lo que queremos o si nos falta alguna cosa por definir. Si queremos podemos guardarlo en un archivo .json externo (es útil a veces, ya que hay configuraciones muy grandes).

- **C. Terraform apply:** A través de 'terraform apply' se pueden aplicar los cambios requeridos para alcanzar el estado deseado de la infraestructura, 'terraform apply' realizará las adiciones, eliminaciones o modificaciones configuradas según lo necesario para coincidir con la configuración. Este comando es el que despliega en la práctica lo que hayamos definido en nuestro código. Antes de hacerlo nos mostrará los cambios que se van a hacer (de manera similar al plan) y nos preguntará si queremos continuar, por lo que, si revisamos bien, nos da mucha seguridad.
- **D. Terraform destroy:** Este comando destruye todos los recursos que Terraform ha creado, y es útil cuando queremos deshacernos de toda la infraestructura, por ejemplo, al finalizar una prueba o un proyecto. De igual manera que sucede con el apply, nos mostrará una lista de los elementos que debemos destruir y nos preguntará si queremos continuar antes de hacerlo. En caso de querer destruir solo un elemento concreto podríamos utilizar el flag -target. Por ejemplo, para destruir la instancia EC2 del último ejemplo, lo haríamos así:

```
terraform destroy -target aws_instance.devops_instance
```

2.1.4 Otros comandos útiles

Además de los comandos principales vistos en el punto anterior, existen muchos otros que formarán parte de nuestro día a día en determinadas circunstancias.



A. Terraform validate: 'terraform validate' verifica si los archivos de configuración son sintácticamente válidos y que no contienen errores internos. No verifica la configuración con arreglo a los servicios reales, pero al menos asegura que los archivos están listos para ser planificados y aplicados.

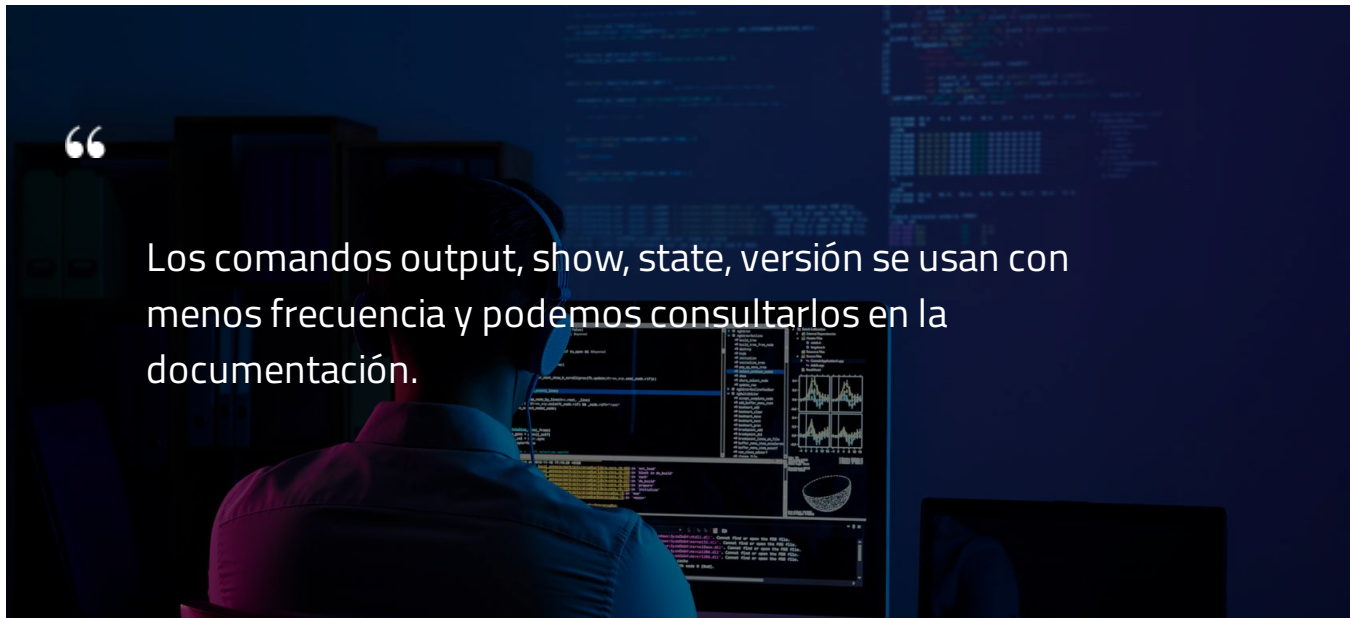


B. Terraform refresh: Este comando actualiza el estado actual de Terraform para reflejar el estado real de los recursos en la nube. Esto puede ser necesario si se han realizado cambios manuales o externos en los recursos gestionados por Terraform.



C. Terraform import: 'terraform import' se usa para importar recursos existentes a la gestión de Terraform. Es útil si has creado recursos fuera de Terraform (desde la consola del proveedor cloud, por ejemplo) y necesitas incluirlos en su gestión sin recrearlos. Los recursos se importan de manera diferente en función de su tipo, y en la documentación de Terraform podemos verificar la manera de hacerlo para cada uno. Estos son los comandos que más utilizaremos, además de los que hemos definido como comandos clave, pero hay bastantes más, como pueden ser output, show, state, versión, etc. Se usan con menos frecuencia y podemos consultarlos en la documentación.

Estos son los comandos que más utilizaremos, además de los que hemos definido como comandos clave, pero hay bastantes más, como pueden ser output, show, state, versión, etc.



Los comandos output, show, state, versión se usan con menos frecuencia y podemos consultarlos en la documentación.

2.1.5 El archivo de estado o tfstate

Como vimos al empezar el tema, dentro de los elementos clave de Terraform existe uno llamado state management. Para administrar el estado, Terraform utiliza un fichero que llamamos .tfstate, que **guarda el estado actual de la infraestructura que administra. Este archivo es clave porque la herramienta lo utiliza para mapear los recursos reales en los servicios de nube con la configuración definida en los archivos de código.** De esta manera, Terraform puede saber qué cambios aplicar a la infraestructura sin necesidad de reconstruir todo desde cero ni de causar interrupciones.

Esta la manera que tiene Terraform de saber qué infraestructura está manejando. Si tenemos un código que genera una **máquina EC2, un grupo de seguridad y una VPC y lo desplegamos en la nube, Terraform generará un fichero .tfstate con el que sabrá que tenemos esos tres elementos desplegados.** Si a nuestro código le añadimos una nueva máquina EC2 y desplegamos de nuevo ('terraform apply'), Terraform no toca los recursos que ya teníamos, porque gracias al .tfstate ya sabe que existen, y simplemente añadiría la nueva máquina.

Funciona igual si, por ejemplo, cambiamos la capacidad de la máquina EC2 en nuestro código. Terraform ya sabe que existe y no vuelve a crear una máquina nueva; modifica la ya existente.

Cuando antes mencionamos 'terraform import', hablamos de incluir en el .tfstate un recurso que se ha desplegado manualmente (por ejemplo, en la consola de AWS), para que Terraform sea consciente de que existe.

Entonces, y aun a riesgo de repetirnos, ¿por qué es importante el tfstate?

Mapeo de recursos	Gestión de dependencias	Detección de cambios	Prevención de conflictos
El archivo de estado permite a Terraform saber a qué recurso en el código corresponde cada recurso en la infraestructura real. Sin este archivo, Terraform no podría diferenciar entre recursos ya creados y los que necesita crear o modificar	Ayuda a Terraform a entender las dependencias entre los recursos, asegurando que se creen, se actualicen o se eliminen en el orden correcto	Cada vez que ejecutas Terraform, compara el estado deseado definido en tu configuración con el estado actual almacenado en el archivo de estado. Esto le permite realizar solo los cambios necesarios para alcanzar el estado deseado.	El archivo de estado puede utilizarse en un entorno de trabajo compartido para asegurar que los cambios hechos por una persona no interfieran sin saberlo con los cambios hechos por otra. Es aún más eficaz si se utiliza el almacenamiento remoto de estado.

Entendido lo anterior, vamos a ver ahora y de manera sencilla cómo funciona el archivo .tfstate:

- **Creación:** Cuando terraform crea un recurso, registra los detalles de ese recurso en el archivo de estado.
- **Actualización:** Si modificamos nuestra configuración y volvemos a ejecutar terraform, comparará la nueva configuración con el estado almacenado y realizará los cambios necesarios.

- **Eliminación:** Si eliminamos un recurso de la configuración y ejecutamos terraform, detectará que el recurso ya no está en la configuración, pero sí en el estado, y procederá a eliminarlo.

Como hemos visto, el archivo .tfstate tiene una importancia considerable dentro de Terraform, lo que hace que sea recomendable aplicar una serie de prácticas cuando hagamos uso de él.

- **Uso de almacenamiento remoto:** Para equipos grandes o críticos, se recomienda almacenar el estado en un backend remoto como AWS S3, azure blob storage o google cloud storage. Esto ayuda a manejar el estado de forma segura y facilita el trabajo en equipo.
- **Bloqueo del estado:** Al utilizar un backend que soporte bloqueo de estado (por ejemplo, una base de datos dynamo DB), puedes evitar que múltiples personas realicen cambios en la misma infraestructura simultáneamente, lo que contribuye a prevenir conflictos y errores.
- **Backups periódico:** Aunque el almacenamiento remoto es más seguro, siempre es una buena práctica tener backups periódicos del estado para recuperarse de posibles errores o corrupciones del archivo de estado.

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket" # Nombre del bucket en el que guardaremos el .tfstate  
    key         = "path/to/my/terraform.tfstate" # Path y nombre del fichero .tfstate  
    region      = "es-west-1" # La región de AWS (en este caso, Irlanda)  
    dynamodb_table = "my-terraform-lock-table" # Nombre de la tabla  
    encrypt     = true  
  }  
}
```

Figura 10. Uso de un bucket S3 para almacenar el archivo de estado físicamente y una tabla DynamoDB para gestionar el bloqueo

2.2 Estructura de un proyecto de Terraform

Cuando estructuramos un proyecto de Terraform, es importante organizar los archivos y carpetas de manera que el código sea fácil de entender, mantener y escalar. A continuación, vemos una estructura de proyecto típica de Terraform con descripciones y ejemplos para cada componente.

```
.
├─ main.tf
├─ variables.tf
├─ outputs.tf
├─ resources
│   ├─ ec2.tf
│   ├─ iam.tf
│   ├─ s3.tf
│   └─ vpc.tf
└─ config
    ├─ production.tfvars
    ├─ sandbox.tfvars
    ├─ production.backend.tfvars
    └─ sandbox.backend.tfvars
```

Figura 11. Estructura típica de un proyecto en Terraform

2.2.1 Descripción de archivos y directorios

- **A. Main.TF:** Contendrá la configuración esencial, incluidos los providers (AWS, Azure, Google Cloud, etc.) y cualquier configuración base necesaria para todos los recursos. Aquí solemos definir también el bloque Terraform, con la versión que vamos a utilizar. Podríamos incluir los providers en un fichero específico 'providers.tf', pero, en aras de la sencillez, en este ejemplo los incluimos en el main.
- **B. Variables. TF:** Define todas las variables usadas a lo largo del proyecto, lo que permite parametrizar la configuración.
- **C. Outputs.TF:** Especifica las salidas de Terraform que deseas visualizar posaplicación, como IP, URL, etc.
- **D. Carpeta "resources":** En esta carpeta (podrían también estar en la raíz), definiremos los archivos propios de infraestructura, divididos por tipo de recurso para una mejor organización.
- **E. Carpeta "config":** En esta carpeta (podemos darle el nombre que queramos) guardaremos los ficheros 'tfvars', que contienen los valores de las variables que vamos a utilizar. Tener ficheros separados por entornos nos permite aplicar un mismo código para desplegar un sandbox o producción con sus valores correspondientes. Utilizaremos los ficheros 'backend' para los valores de variables específicos de backend, por ejemplo: definiremos que, si estamos en sandbox, el fichero .tfstate se guardará en el bucket A de la cuenta de AWS A, y, si estamos en producción, el .tfstate se guardará en el bucket B de la cuenta B.

```
terraform {  
  required_version = "= 1.5.7"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 4.0" |  
    }  
  }  
}  
  
provider "aws" {  
  region = "eu-west-1"  
}
```

2.2.2 Ejemplificación archivos

A continuación, un ejemplo de 'variables.tf'

```
variable "instance_type" {  
  description = "El tipo de instancia EC2 a utilizar"  
  type        = string  
  default     = "t2.micro"  
}  
  
variable "public_key" {  
  description = "La clave pública que se usará para la instancia EC2"  
  type        = string  
  default     = ""  
  validation {  
    condition     = length(var.public_key) > 0  
    error_message = "La clave pública no puede estar vacía."  
  }  
}  
  
variable "availability_zones" {  
  description = "Lista de zonas de disponibilidad para desplegar recursos"  
  type        = list(string)  
  default     = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]  
}
```

Figura 13. Ejemplo de variables.tf

Y, por último, un ejemplo de un fichero 'tfvars'

```
instance_type      = "t3.large"  
public_key         = "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCuYJ1..."  
availability_zones = ["eu-west-1a", "eu-west-1b"]
```

Figura 14. Ejemplo de archivo .tfvars

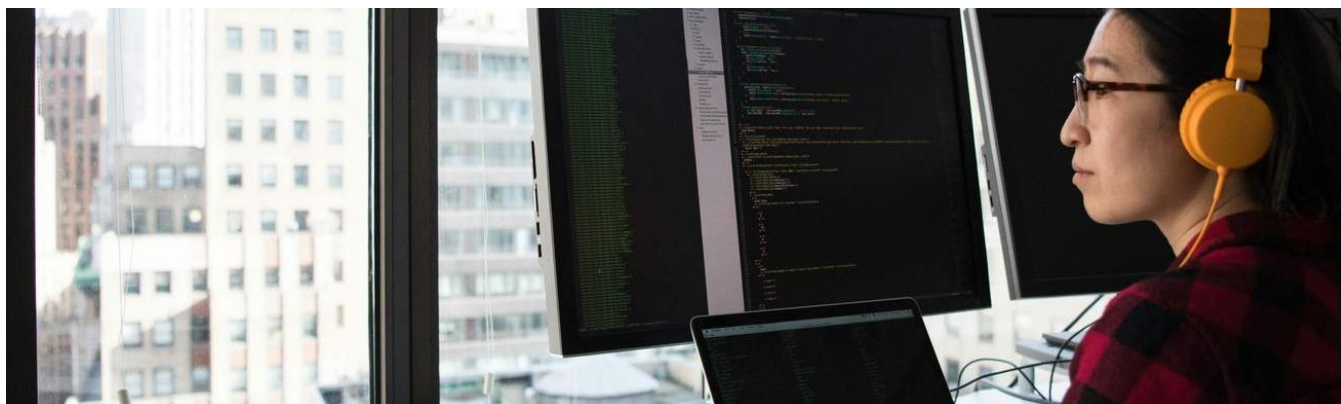
Si nos fijamos, en el variables.tf hemos puesto un valor por defecto para esas variables. No es

Es importante estructurar un proyecto de Terraform de manera lógica y legible, ya sea como os proponemos aquí o con una organización que se ajuste a vuestras necesidades. Aunque no es algo imperativo, podríamos tener, por ejemplo, un 'main.tf' en que el que declaramos providers, recursos, outputs y los valores de las variables hardcoded y funcionaría, pero no se considera buena práctica en absoluto.

2.3 Prácticas avanzadas y automatización

A medida que vayamos cogiendo soltura con Terraform, iremos aprendiendo poco a poco algunas prácticas más avanzadas junto con procesos de automatización que, en ocasiones nos veremos obligados a ejecutar.

Con la adopción de estas prácticas avanzadas y la automatización en Terraform no solo aumentaremos la eficiencia y reduciremos los errores, sino que también **mejoraremos la seguridad y la conformidad con las políticas corporativas**. Al avanzar hacia una gestión de infraestructura como código más sofisticado, podemos asegurar operaciones más ágiles y la optimización de costes.



En cualquier caso, aunque a continuación hagamos una breve descripción de ellas, a la hora de empezar a utilizar una herramienta como Terraform **es recomendable comenzar por las bases**, entendiendo bien su funcionamiento, para ir incorporando mejoras a medida que vas progresando.

2.3.1 Automatización del flujo de trabajo de Terraform

La **integración y la entrega continua (CI/CD)** son necesarias para modernizar y automatizar la gestión de infraestructura. **Al incorporar Terraform en pipelines de CI/CD, podemos automatizar la validación, planificación y aplicación de cambios de infraestructura**, lo que facilita el mantenimiento y asegura que los despliegues sean consistentes y sin errores.

Gracias a herramientas como **GitHub Actions**, **GitLab CI** o **Jenkins** podemos automatizar la ejecución de 'terraform plan' y 'terraform apply' basándonos en cambios de código y políticas de revisión. Además, utilizar **Terraform Cloud** o **Terraform Enterprise** proporciona funcionalidades adicionales, como ejecuciones de Terraform automatizadas y gestión de estado segura, entre otras, todas directamente integradas con nuestros sistemas de control de versiones.

```
# Ejemplo utilizando GitHub Actions
name: 'Terraform CI/CD'

on:
  push:
    branches:
      - main
  pull_request:

jobs:
  terraform:
    name: 'Terraform Apply'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v2

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan

      - name: Terraform Apply
        run: terraform apply -auto-approve
        if: github.ref == 'refs/heads/main'
```

Figura 15. Pipeline de despliegue de Terraform con Github actions que se ejecuta cuando hacemos un push a la rama main.

2.3.2 Gestión de estado avanzada

Configurar un backend remoto, como AWS S3 con bloqueo de estado a través de AWS DynamoDB, nos permite asegurar que el estado de Terraform sea seguro, versionado y solo accesible para operaciones autorizadas. Además, implementar bloqueos de estado previene conflictos que podrían surgir de despliegues simultáneos. Cuando desplegamos un código Terraform, el backend queda bloqueado mientras lo hacemos, a fin de evitar que otra persona efectúe un despliegue al mismo tiempo y se produzcan configuraciones contradictorias o incompatibles.

RECUERDA

La integración y la entrega continua (CI/CD) son necesarias para modernizar y automatizar la gestión de infraestructura.

2.3.3 Modularización y reutilización de código

En ocasiones, será necesario crear módulos personalizados para encapsular configuraciones frecuentes, esto nos permitirá reutilizar código en varios proyectos o entornos, facilitando la gestión y la escalabilidad.

Publicar módulos internos en un registro de Terraform **permite versionar y compartir fácilmente estos recursos dentro de la organización**. Además, nos permite cerrar los módulos con

```
# Módulo para una instancia EC2
module "ec2_instance" {
    source          = "../modules/ec2"
    instance_type  = var.instance_type
    ami             = var.ami
}
```

Figura 16. Ejemplo de módulo

Tenemos muchos módulos oficiales a nuestra disposición ya creados para elementos que se utilizan mucho, como pueden ser las EC2 o las Lambdas en AWS, pero **también es posible crear nuestros módulos propios** y almacenarlos en Github, por ejemplo, o en repositorios de artefactos como Nexus.

2.3.4 Optimización y costes

Con herramientas como **Infracost** o **Terraform Cost** podemos estimar el impacto económico de los cambios en Terraform antes de aplicarlos, **a fin de tomar decisiones más informadas y responsables sobre la infraestructura**. Como ya hemos comentado en este curso, nuestro trabajo no solo se basa en levantar y mantener infraestructura, también **tenemos que ser conscientes de los costes que va a suponer**, y gracias a módulos como estos podemos tomar mejores decisiones.

Además, al regular la revisión de los recursos y su utilización se identifican áreas donde se pueden reducir los costes, por ejemplo, desescalando instancias durante períodos de baja demanda.

Tema 3: Automatización y orquestación de redes en DevOps



Introducción



3.1 Importancia de la automatización en redes



3.2 Automatización de redes con Terraform

Introducción

A lo largo de este tema, exploraremos cómo la automatización de redes facilita la implementación de **cambios rápidos y consistentes**, esenciales para soportar los ciclos de desarrollo y despliegue característicos de DevOps. Abordaremos conceptos clave como la provisión de infraestructura, la escalabilidad y rendimiento, la seguridad y monitorización, y la integración y despliegue continuos. Estos conceptos no solo son fundamentales para la construcción de sistemas robustos y seguros, sino que también sirven como piedra angular para la implementación de prácticas de DevOps eficaces.

Además, este tema destacará la relevancia del conocimiento en redes para los profesionales de DevOps, subrayando cómo una comprensión sólida en configuración de infraestructura, monitoreo de red, implementación de la seguridad y solución de problemas de red, es indispensable en el día a día de estos profesionales. A través de una inmersión profunda en la automatización de redes con herramientas como Terraform, los estudiantes adquirirán habilidades prácticas para desplegar y gestionar redes de manera automática y escalable, garantizando configuraciones coherentes y reduciendo errores humanos.

Este tema no solo es relevante por su aplicación práctica inmediata, sino también por cómo prepara a los estudiantes para enfrentar los desafíos futuros en la gestión de infraestructuras de red dentro de entornos DevOps. Al finalizar este módulo, los estudiantes estarán equipados con el conocimiento y las herramientas necesarias para mejorar significativamente la colaboración entre equipos, optimizar los procesos y elevar la calidad del producto final en sus organizaciones.

3.1 Importancia de la automatización en redes

En el ámbito de DevOps, la automatización de redes **es parte integrante de los procesos que incrementan la eficiencia y escalabilidad de las operaciones**. Mediante la automatización, las organizaciones pueden implementar cambios rápidos y consistentes en la infraestructura de red, algo imprescindible para soportar los ciclos rápidos de desarrollo y despliegue que caracterizan DevOps.

Una configuración de red adecuada es importantísima para que nuestros desarrollos y aplicaciones funcionen según lo previsto, y la infraestructura como código puede servirnos también como documentación, lo que nos vendrá genial, pues en muchos casos la infraestructura de red puede (y suele) ser compleja.



Además, gracias a la automatización de redes, se **elimina la necesidad de intervenciones manuales en tareas rutinarias y repetitivas, pudiendo dedicar los equipos de red más tiempo a iniciativas estratégicas** y menos a la gestión diaria.

De esta manera se reduce drásticamente el tiempo de implementación y el margen de error humano, dos factores que pueden llevar a problemas de red significativos y costosos.



Con herramientas automatizadas, los cambios se pueden implementar con mayor rapidez y precisión, y los sistemas pueden responder de forma proactiva a las condiciones cambiantes del entorno de red.

Los conceptos más importantes son:

- **Provisión de infraestructura:** Los conocimientos en redes son necesarios para la configuración y gestión de entornos virtuales, como plataformas en la nube o centros de datos locales. Organizar y conectar recursos de manera eficaz exige comprender los conceptos de red.
- **Escalabilidad y rendimiento:** El objetivo de los equipos de devops es desarrollar sistemas escalables y de alto rendimiento. Los ingenieros que dominan las redes diseñan y optimizan la arquitectura de red, el equilibrio de carga y la gestión del tráfico para cumplir con estos requisitos.
- **Seguridad y monitorización:** Las redes son la base para implementar medidas de seguridad sólidas y soluciones de monitoreo. Es necesario manejar con soltura conceptos como firewalls, VPN, tablas de rutas, grupos de seguridad y herramientas de monitoreo de red para garantizar la seguridad y la fiabilidad de los sistemas.
- **Integración y despliegue continuos:** La experiencia en redes nos permite diseñar flujos de trabajo eficientes para la integración y despliegue continuos (CI/CD). Lo que implica configurar la conectividad de red entre diversas plataformas y servicios.

Como vemos, el conocimiento de redes es muy importante en un puesto DevOps, por supuesto no se nos exige el mismo conocimiento que a un especialista centrado solamente en ese campo,

pero sí que necesitamos unas bases sólidas. Veamos otros **cuatro puntos sobre por qué estos conocimientos son importantes y qué nos vamos a encontrar en nuestro día a día:**

- **Configuración de infraestructura:** Nos encargaremos de configurar los ajustes de red para servidores, máquinas virtuales, equilibradores de carga y contenedores. Para lograr una comunicación fluida entre estos recursos, es imprescindible comprender la asignación de direcciones IP, el subnetting (dividir redes en subredes) y el enrutamiento.
- **Monitoreo de red:** Supervisar el tráfico de red, el uso de ancho de banda y la latencia sirve para identificar cuellos de botella y optimizar el rendimiento de los sistemas. Poder efectuar un diagnóstico eficaz y llevar a cabo un mantenimiento proactivo requiere conocer las herramientas de monitoreo de red.
- **Implementación de la seguridad:** Configurar redes de manera segura, gestionar las reglas del firewall o de los grupos de seguridad y establecer redes privadas virtuales (VPN) son tareas que contribuyen a garantizar la seguridad e integridad de nuestra infraestructura.
- **Solución de problemas de red:** Los ingenieros de devops solemos enfrentarnos a los desafíos que plantean las redes. Comprender los protocolos de red, diagnosticar problemas de conectividad y realizar tareas de solución de problemas en la red son habilidades imprescindibles.

3.1.1 Conceptos básicos de redes

Visto lo anterior y con el objetivo de entender mejor los conceptos, vamos a repasar a continuación algunos de los conceptos básicos de esta rama con los que tendremos que estar familiarizados:

(A) Asignación de IP y subnetting

La asignación de direcciones IP es el proceso mediante el que **se otorgan identificadores numéricos únicos a los dispositivos de una red**, permitiendo la comunicación entre ellos mediante el protocolo de Internet (IP). A su vez, el *subnetting*, o creación de subredes, **divide una red grande en redes más pequeñas**, mejorando la asignación de direcciones IP, la segmentación de la red, la seguridad y el rendimiento.

Debemos entender las direcciones IP y la creación de subredes para gestionar posteriormente y de manera eficiente los recursos de la red, de manera que podamos aprovechar los rangos de IP, dirigir correctamente el tráfico y aumentar la seguridad mediante la separación de segmentos de la red.

(B) Conjunto de protocolos TCP/IP

El conjunto de protocolos TCP/IP rige la comunicación a través de internet y la mayoría de las redes informáticas, incluyendo protocolos como IP, TCP, UDP e ICMP. Este **conjunto proporciona la base para la transmisión de datos y la comunicación en red, garantizando la entrega de paquetes de manera segura y sin errores**.

Como futuros expertos en DevOps es necesario que comprendamos estos protocolos para diagnosticar problemas, optimizar el rendimiento y diseñar mecanismos de comunicación eficientes.

(C) DNS y DHCP

El sistema de nombres de dominio (DNS) **traduce nombres de dominio legibles por humanos a direcciones IP. Es el sistema que nos permite acceder a sitios web mediante nombres de dominio familiares.** Por otro lado, el protocolo de configuración dinámica de host (DHCP) automatiza la asignación de direcciones IP en una red.

Sin el DNS no se podría navegar por internet tal como lo hacemos, porque es el sistema que nos da una correspondencia entre nombres de dominio y direcciones IP. El DHCP simplifica la gestión de la red al eliminar la necesidad de configurar manualmente las direcciones IP.

(D) Enrutamiento y conmutación

El enrutamiento **decide la ruta que los paquetes de red deben seguir desde el origen hasta el destino**, usando protocolos como OSPF y BGP. La conmutación, por otro lado, transfiere paquetes de datos entre dispositivos dentro de una red.

De esta manera, gracias al enrutamiento se transmiten los datos de manera eficiente y segura, mientras que con **la conmutación se facilita la comunicación interna en la red.**

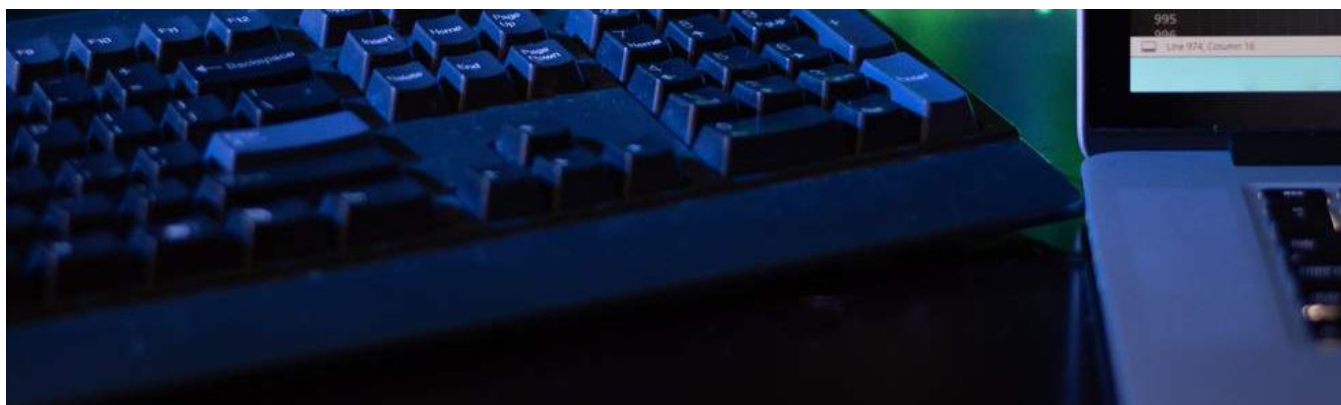
(E) Seguridad de red

La seguridad de red protege la infraestructura, los datos y las comunicaciones de accesos no autorizados, ataques y amenazas, e incluye medidas como firewalls, VPN, protocolos de encriptación como SSL/TLS y prácticas de comunicación segura. Esto es fundamental **para proteger los datos confidenciales, garantizar la privacidad y prevenir actividades maliciosas.**

Resulta por tanto necesario que comprendamos estos conceptos de seguridad de red para poder implementar medidas de seguridad sólidas, detectar y responder a amenazas, y mantener la integridad y disponibilidad de los recursos de la red.

3.2 Automatización de redes con Terraform

Como hemos visto, en la era de la nube, es necesario poseer la capacidad de desplegar y gestionar redes de forma automática y escalable. Con Terraform, podemos garantizar configuraciones coherentes, reducir errores humanos y agilizar el proceso de despliegue y mantenimiento de infraestructuras de red.



3.2.1 Creación de una virtual private cloud (VPC)

La VPC es uno de los principales componentes de la red en la nube, proporcionando un **espacio aislado** donde operan los recursos. Vamos a crear una VPC configurando el bloque CIDR adecuado, que define el rango de direcciones IP privadas. Para estos ejemplos de Terraform utilizaremos AWS como base.

```

✓ resource "aws_vpc" "mi_vpc" {
  cidr_block      = "10.0.0.0/16"
  instance_tenancy = "default"
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = "curso_devops_vpc"
  }
}

```

Figura 17. Código que inicializa una VPC con un bloque CIDR que permite hasta 65 536 direcciones IP privadas.

En el código de ejemplo vemos estas configuraciones:

- **cidr_block:** Define el rango de direcciones IP dentro de la VPC. Este rango es exclusivo y no debe solaparse con otras redes.
- **enable_dns_support y enable_dns_hostnames:** Estas configuraciones permiten la resolución de nombres DNS dentro de la VPC.

Como comentamos anteriormente, la segmentación de una VPC en subredes (o *subnetting*) permite un control detallado sobre el diseño de la red, lo que posibilita la optimización de recursos y mejora la seguridad.

Vamos a ver cómo definir subredes en Terraform.

```
resource "aws_vpc" "mi_vpc" {
  cidr_block      = "10.0.0.0/16"
  instance_tenancy = "default"
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = "curso_devops_vpc"
  }
}

resource "aws_subnet" "publica" {
  vpc_id            = aws_vpc.mi_vpc.id # Hacemos referencia a la VPC que hemos creado en el bloque anterior
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "eu-west-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "publica"
  }
}

resource "aws_subnet" "privada" {
  vpc_id            = aws_vpc.mi_vpc.id # Hacemos referencia a la VPC que hemos creado en el bloque anterior
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "eu-west-1b"
  map_public_ip_on_launch = false

  tags = {
    Name = "privada"
  }
}
```

Figura 18. Ejemplo de definición de subredes

Aquí **map_public_ip_on_launch** indica si las instancias lanzadas en estas subredes deben recibir una dirección IP pública.

Además de redes y subredes, hay otros dos elementos importantes, ya que las redes necesitan acceso a internet. En AWS tenemos el **Internet Gateway** y el **NAT Gateway**. El Internet Gateway proporciona acceso a Internet a las subredes públicas, mientras que los NAT Gateways permiten que las subredes privadas accedan a recursos de Internet sin exponerse directamente.

```

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.mi_vpc.id

  tags = {
    Name = "devops_igw"
  }
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.publica.id

  tags = {
    Name = "devops_nat"
  }
}

resource "aws_eip" "nat" {
  vpc = true
}

```

Figura 19. Internet Gateway e Internet Nat Gateway

Estos conceptos son propios de AWS, pero en proveedores como Azure o Google Cloud vamos a encontrar elementos similares u homólogos. Esto nos pasará siempre, ya que **los proveedores Cloud importantes funcionan de manera semejante y ofrecen servicios muy parecidos**, pero en el cambio de uno a otro nos tocará hacer un poco de investigación.

Ejemplo

Por ejemplo, Azure sí ofrece un servicio de NAT Gateway que se puede usar para permitir que las máquinas virtuales en una red privada accedan a Internet mientras usan una dirección IP pública estática. Este servicio es similar al NAT Gateway en AWS.

En cambio, no hay un Internet Gateway como tal, y cualquier recurso en una subred puede acceder a Internet si no se configura un grupo de seguridad de red (*network security group* [NSG]) con reglas que lo impidan.

3.2.2 Tablas de rutas

Las tablas de rutas son las que definen cómo se dirige el tráfico dentro de la VPC, de forma que el tráfico de las subredes privadas pase a través del NAT Gateway, mientras que el tráfico de las subredes públicas puede ir directamente a Internet.

```
resource "aws_route_table" "rtb_publica" {
  vpc_id = aws_vpc.mi_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id #Aquí haríamos referencia al ID den Internet Gateway creado anteriormente
  }

  tags = {
    Name = "devops_rtb_publica"
  }
}

#Aquí asociamos la tabla de rutas a una subnet concreta mediante el ID de la misma.
resource "aws_route_table_association" "publica_assoc" {
  subnet_id      = aws_subnet.publica.id
  route_table_id = aws_route_table.rtb_publica.id
}

resource "aws_route_table" "rtb_privada" {
  vpc_id = aws_vpc.mi_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id #Aquí haríamos referencia al ID den NAT Gateway creado anteriormente
  }

  tags = {
    Name = "rtb_privada"
  }
}

resource "aws_route_table_association" "privada_assoc" {
  subnet_id      = aws_subnet.privada.id
  route_table_id = aws_route_table.rtb_privada.id
}
```

Figura 20. Tabla de ruta

En la figura anterior podemos ver la configuración de la **ruta 0.0.0.0/0 en 'rtb_publica'** para dirigir todo el tráfico no local hacia el Internet Gateway. Junto a ésta, podemos ver también una configuración similar en 'rtb_privada', pero redirigiendo el tráfico hacia el NAT Gateway.

3.2.3 Grupos de seguridad

Los grupos de seguridad (*security groups* en inglés) actúan como **firewalls virtuales** para las instancias, controlando tanto el tráfico entrante como el saliente, basándose en reglas específicas que nosotros definimos. Son una parte muy importante de la seguridad de nuestra red.

Veamos algunos ejemplos:

1

```
resource "aws_security_group" "sg_web" {
  name           = "sg_devops"
  description    = "Security Group for DevOps Tokio Course"
  vpc_id        = aws_vpc.mi_vpc.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "sg_web"
  }
}
```

Figura 21. Ejemplo de grupo de seguridad

- **ingress:** Reglas que definen cómo se permite el acceso a las instancias desde el exterior.
- **egress:** Reglas que controlan cómo las instancias pueden comunicarse hacia fuera.

```
resource "aws_vpc" "test-vpc" {
  cidr_block = "10.200.0.0/16"
  tags = {
    Name = "devops-test"
  }
}

resource "aws_subnet" "test-public-subnet" {
  vpc_id = aws_vpc.test-vpc.id
  cidr_block = "10.200.200.0/24"
  map_public_ip_on_launch = "true"
  availability_zone = "eu-west-1a"
  tags = {
    Name = "devops-public-subnet"
  }
}

resource "aws_route_table" "public-route" {
  vpc_id = aws_vpc.test-vpc.id
  tags = {
    Name = "devops-public-route"
  }
}

resource "aws_internet_gateway" "test-igw" {
  vpc_id = aws_vpc.test-vpc.id
  tags = {
    Name = "devops-test-igw"
  }
}

resource "aws_route" "internet-access" {
  route_table_id = aws_route_table.public-route.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.test-igw.id
}

resource "aws_route_table_association" "subnet-association" {
  subnet_id = aws_subnet.test-public-subnet.id
  route_table_id = aws_route_table.public-route.id
}
```

Figura 22. Despliegue de VPC y subred

En esta imagen estamos creando un **grupo de seguridad** que permitiría el tráfico SSH y una instancia **EC2**. Revisando los valores de la máquina, vemos que utiliza la subred y el grupo de seguridad que hemos creado anteriormente.

```

resource "aws_security_group" "allow_ssh" {
  name        = "allow_ssh_devops"
  description = "Allow SSH inbound traffic from my IP"
  vpc_id      = aws_vpc.test-vpc.id

  ingress {
    description = "SSH from gp"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["1.1.1.1/32"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    ipv6_cidr_blocks = [ ":::/0" ]
  }

  tags = {
    Name = "devops-test"
  }
}

resource "aws_instance" "test-linux" {
  ami          = "ami-07355fe79b493752d"
  instance_type = "t2.micro"
  subnet_id    = aws_subnet.test-public-subnet.id
  key_name     = "devops-keypair"
  vpc_security_group_ids = [ aws_security_group.allow_ssh.id ]
  associate_public_ip_address = true

  tags = {
    Name = "devops-test-linux"
  }
}

output "public_ip" {
  value = aws_instance.test-linux.public_ip
}

```

Figura 23. Grupo de seguridad que permite tráfico SSH

En este último ejemplo estamos desplegando una **VPC** y una **subred**. A su vez, desplegamos también un **Internet Gateway** (para dar salida a internet a esa subred pública) y una tabla de rutas para la subred.

A esa tabla de rutas le añadimos una **ruta** con su bloque correspondiente y asociamos la tabla de rutas a la subred con el '**aws_route_table_association**'.

3.2.4 Superar complejidades en la automatización de redes

La automatización de redes comprende múltiples sistemas de distintos proveedores para satisfacer una solicitud lógica única, lo que normalmente requeriría conocimiento de cada interfaz de línea de comandos (CLI), interfaz gráfica de usuario (GUI) o API. **Terraform simplifica este proceso mediante el uso de un lenguaje uniforme y fácil de leer, Hashicorp Configuration Language (HCL), que mantiene la coherencia sin importar con qué tecnología interactuemos.**



Por ejemplo, el código para una subred en Terraform puede estar contenido en un único archivo que describa todo lo necesario, facilitando así el trabajo de los operadores que no necesitan aprender a interactuar con cada API por separado y pueden concentrarse en la provisión de infraestructura.

3.2.5 Gestión segura de los datos

Otra cuestión delicada en la automatización propia de cambios de red es la recuperación y traducción manual de atributos, como las subredes de una herramienta IPAM, proceso en el que es fácil cometer errores y consume mucho tiempo.

Terraform facilita la transferencia segura de datos entre recursos de una misma solicitud, como referenciar una subred en la interfaz de capa 3, en los grupos de seguridad o en el firewall, lo que reduce el tiempo dedicado a copiar y pegar información manualmente, **disminuyendo el riesgo de errores operativos**, una causa habitual de interrupciones en la red.

3.2.6 Orden de operaciones



Cuando operamos con redes debemos seguir un orden de operaciones específico para completar las solicitudes, lo que a menudo requiere conocimiento especializado o empírico.

Al ser una herramienta declarativa, Terraform calcula automáticamente las dependencias en estas solicitudes y determina qué recursos se pueden crear en paralelo, lo que no solo agiliza el proceso, sino que también reduce la necesidad de mantener documentación actualizada, aunque, como siempre decimos, la documentación exhaustiva nunca está de más.

3.2.7 Gestión del ciclo de vida de los recursos

Muchos equipos de operaciones se centran en la creación de recursos para satisfacer las necesidades de la empresa, pero a menudo se ignora la gestión del ciclo de vida de estos recursos. Esto, como ya hemos explicado en lecciones previas, **puede resultar en infraestructura obsoleta que consume recursos valiosos.**

Terraform realiza un seguimiento de la infraestructura después de su creación y, si hay cambios en el código (por ejemplo, si de pronto se necesita una subred /25 en lugar de una /26), determinará qué elementos de la infraestructura necesitan actualizarse.

3.2.8 Colaboración más fluida

Los equipos de infraestructura a menudo luchamos por trabajar de manera colaborativa usando sistemas CLI o GUI.

Terraform facilita esta colaboración al integrarse directamente con sistemas de control de versiones o pipelines de CI/CD, implementando las mejores prácticas en torno a la higiene del código.

Además, los equipos con conocimiento del dominio (por ejemplo, el equipo de red, si lo hay) pueden modularizar y publicar el código de Terraform para que lo consuman otros equipos que solo necesitan el resultado (por ejemplo, equipos de desarrollo de aplicaciones), haciendo que la infraestructura sea más predecible, repetible y susceptible de ser probada.

3.2.9 Reducción de riesgos

Adoptar la automatización con demasiada rapidez puede suponer riesgos considerables, como errores a gran escala o abusos del sistema. Terraform aborda estos riesgos mediante un marco de políticas como código que **se puede personalizar para controles de gobernanza y cumplimiento**.

Así se mitigan los riesgos asociados con los cambios automáticos, ya que, al definir políticas en código, **estas se pueden auditar y revisar antes de realizar cualquier cambio**, y Terraform también se integra en los procesos existentes de gestión de cambios para su aprobación.

Tema 4: Introducción a Azure



Introducción



4.1 Inicios y lanzamiento de la plataforma Azure



4.2 Estructura general de Azure: regiones, zonas de disponibilidad y centros de datos



4.3 Suscripciones y administración de costes



4.4 Grupos de recursos



4.5 Principales recursos en Azure

Introducción

Este tema se adentrará en el vasto ecosistema de **Azure**, explorando su **estructura, servicios y capacidades** que permiten a las empresas innovar, escalar y adaptarse a los cambiantes requisitos del mercado con una eficiencia y seguridad sin precedentes.

Azure no es solo una plataforma de servicios en la nube; es un habilitador de posibilidades, que ofrece desde soluciones de infraestructura básica hasta capacidades avanzadas de inteligencia artificial y análisis de big data, pasando por **una amplia gama de servicios PaaS y SaaS**.

La relevancia de este tema radica en la creciente demanda de profesionales capaces de navegar, diseñar e implementar soluciones en entornos de nube. Comprender Azure en profundidad no solo es esencial para los desarrolladores y profesionales de IT, sino también para los líderes de proyecto, analistas de sistemas y cualquier persona involucrada en la toma de decisiones estratégicas en el ámbito tecnológico. A través de este estudio, se abordarán los inicios y la evolución de Azure, su estructura general compuesta por **regiones, zonas de disponibilidad y centros de datos**, y cómo estos elementos se interrelacionan para ofrecer una solución robusta y global.

Además, se examinarán las consideraciones clave en la selección de regiones y zonas de disponibilidad, la gestión de costes y la optimización de recursos, así como la importancia de la administración de suscripciones y el control del gasto. Este conocimiento no solo preparará a los estudiantes para enfrentar los desafíos tecnológicos actuales, sino que también **los posicionará a la vanguardia de la innovación, equipándolos con las herramientas y el entendimiento** necesarios para aprovechar el potencial de Azure en la promoción de la transformación digital.

Al finalizar este tema, los estudiantes estarán capacitados para diseñar e implementar soluciones efectivas en Azure, optimizando la colaboración entre equipos, la eficiencia operativa y la calidad del servicio entregado, elementos todos cruciales para el éxito en el competitivo mercado actual.

4.1 Inicios y lanzamiento de la plataforma Azure

Desarrollada por Microsoft, Azure es una **plataforma de computación en la nube expansiva y versátil** que proporciona una amplia gama de prestaciones, incluyendo infraestructura como servicio (**IaaS**), plataforma como servicio (**PaaS**) y software como servicio (**SaaS**). De esta forma, permite a las empresas construir, desplegar y gestionar aplicaciones y servicios a través de una red global de centros de datos de Microsoft.

El enfoque de Azure en la **seguridad**, la **escalabilidad** y la **flexibilidad** contribuye a que organizaciones de todos los tamaños adopten tecnologías de nube para impulsar su transformación digital, ofreciendo soluciones en áreas como computación, almacenamiento, bases de datos, redes, análisis de datos e inteligencia artificial.



Microsoft Azure se lanzó oficialmente como **Windows Azure** en octubre de **2008**, durante el evento Professional Developers Conference (PDC) de Microsoft. En ese momento, se presentó como una plataforma de "nube" que proporcionaba un conjunto de servicios de computación para ayudar a los desarrolladores a construir, desplegar y administrar aplicaciones a través de los centros de datos globales de Microsoft.



De este modo, Windows Azure representó la entrada de Microsoft en el mercado de la nube, compitiendo con plataformas establecidas como Amazon Web Services (AWS).

Desde su lanzamiento, Azure ha ampliado constantemente su cartera de servicios y su presencia global. La plataforma ha crecido hasta abarcar más de **200 productos y servicios** que cubren todos los ámbitos, tanto IaaS, PaaS y SaaS como capacidades de inteligencia artificial, aprendizaje automático e IoT, entre otras muchas.

Microsoft ha invertido significativamente en la expansión de su infraestructura de nube, operando más de **60 regiones de Azure** en todo el mundo, cada una de ellas con múltiples centros de datos, lo que la convierte en una de las **redes de nube más extensas y confiables**.

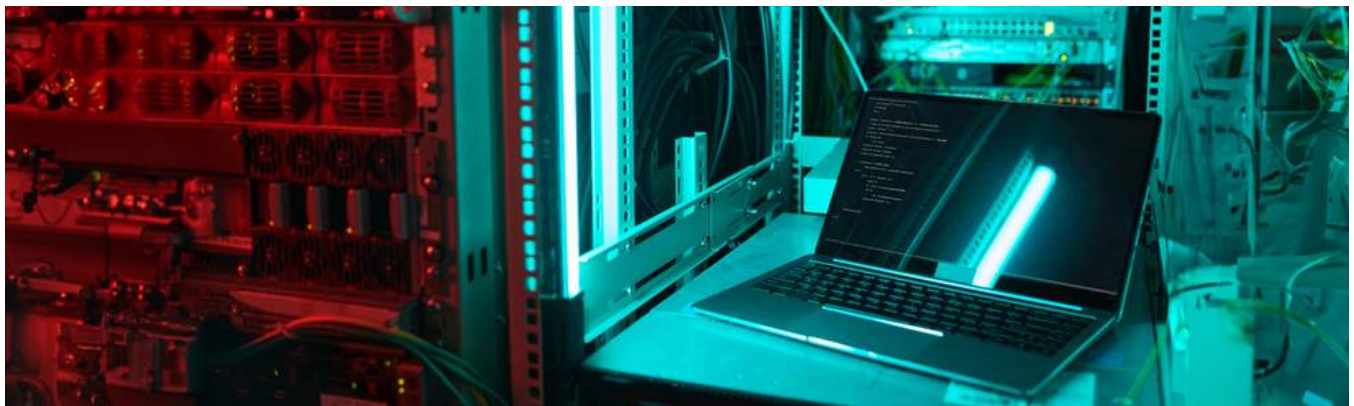
Todo esto ha hecho que Azure se convierta en uno de los principales proveedores de servicios en la nube, ocupando un lugar destacado junto a competidores como Amazon Web Services (AWS) y Google Cloud Platform (GCP).

Asimismo, ha resultado fundamental en la promoción de la **nube híbrida**, permitiendo a las empresas aprovechar tanto los recursos en la nube como los locales, y ha desempeñado un papel crucial en el desarrollo y la popularización de la **computación sin servidor** mediante servicios como Azure Functions.

4.2 Estructura general de Azure: regiones, zonas de disponibilidad y centros de datos

La estructura general de Microsoft Azure ha sido diseñada para ofrecer servicios de computación en la nube altamente disponibles, seguros y escalables a nivel mundial.

Esta infraestructura se organiza en torno a **regiones, zonas de disponibilidad y centros de datos**, proporcionando a los usuarios una amplia gama de opciones a la hora de alojar sus servicios y datos, al tiempo que garantiza la resiliencia, la escasa latencia y la conformidad con las regulaciones locales.



4.2.1 Definición de conceptos

Para poder comprender bien la estructura de Azure, es importante entender previamente los siguientes conceptos, desliza para saber más sobre ello:

Centro de datos

Se trata de instalaciones de vanguardia dirigidas a **alojar la infraestructura física de servidores, almacenamiento y redes** que soporta los servicios de Azure en la nube. Estos centros han sido diseñados con redundancias en sistemas críticos y controles de seguridad física y digital para proteger la infraestructura y los datos de los clientes.

Regiones de azure



Figura 24. Centros de datos y regiones de Azure.

Podemos definir una región de Azure como un **área geográfica que contiene al menos un centro de datos** (aunque normalmente abarca varios) **interconectado con una red de baja latencia.**

Cada región se encuentra separada de las demás por una distancia significativa para reducir el riesgo de fallos simultáneos que podrían repercutir en la continuidad del servicio.

Actualmente, Microsoft tiene desplegadas más de 60 regiones para Azure en todo el mundo, más que cualquier otro proveedor de servicios en la nube, permitiendo a las empresas seleccionar aquella que mejor se ajuste a sus necesidades, una decisión que, como comprobaremos más adelante, resulta especialmente relevante.

Zonas de disponibilidad



Figura 25. Zonas de disponibilidad.

Dentro de las regiones de Azure, una zona de disponibilidad constituye una **capa adicional de separación física y lógica** que consiste en **uno o más centros de datos equipados con su propia alimentación eléctrica, refrigeración y redes independientes**.

Están diseñadas para permanecer aisladas entre sí dentro de la misma región, lo que significa que, si una zona se ve afectada por un desastre natural o un fallo de sistema, las otras zonas de la misma región pueden continuar funcionando normalmente.

Esta arquitectura permite a los usuarios distribuir sus aplicaciones y datos entre varias zonas para lograr una elevada disponibilidad y protección contra fallos.

Como podemos ver, la estructura de Azure, compuesta por regiones, zonas de disponibilidad y centros de datos, ofrece **protección frente a fallos y desastres**, y permite que las empresas pueden diseñar sus aplicaciones y servicios para ser resilientes, distribuyéndolos a través de múltiples zonas de disponibilidad o regiones.

4.2.2 Selección de la región y las zonas de disponibilidad

En Azure, una selección adecuada de la región y de las zonas de disponibilidad representa un factor clave de cara al despliegue de aplicaciones y servicios en la nube. Se trata de una **elección estratégica** que repercute directamente en el **rendimiento**, la **disponibilidad**, la **conformidad con la legislación sobre protección de datos** y la **optimización de costes** para las empresas.

(A) Rendimiento y latencia

La proximidad geográfica de la región elegida a los usuarios finales resulta importante para minimizar la latencia en las aplicaciones y servicios, lo que se traduce en tiempos de carga más rápidos y en una mejor experiencia de usuario, algo fundamental en el caso de usar aplicaciones críticas y sensibles al tiempo de respuesta.

Ejemplo

Una empresa de juegos en línea con base en Europa elige desplegar su infraestructura en la región de Azure "Europa del Norte" para servir a sus usuarios, principalmente europeos.

Al alojar los servidores de juegos y las bases de datos cerca de sus usuarios finales, la empresa puede reducir significativamente la latencia, lo que contribuye a una experiencia de juego más fluida y a una rápida respuesta durante las partidas en línea.

(B) Conformidad y regulaciones de datos

Los diferentes países y regiones presentan distintas regulaciones y requisitos de conformidad en cuanto a la gestión y almacenamiento de datos. Elegir una región de Azure que se alinee con estas legislaciones puede ayudar a las empresas a cumplir con los requisitos legales locales, como el GDPR en Europa, garantizando que los datos se almacenen y procesen de manera adecuada.

Ejemplo

Un hospital en el Reino Unido necesita almacenar datos de pacientes de manera segura de conformidad con el GDPR. Al seleccionar la región de Azure "Reino Unido Sur", el hospital se asegura de que los datos sensibles se almacenen físicamente dentro del territorio del Reino Unido, cumpliendo con las leyes locales de protección de datos y con el GDPR, al tiempo que facilita el acceso rápido y seguro a la información por parte del personal autorizado.

(C) Disponibilidad y recuperación ante desastres

Recurrir a múltiples zonas de disponibilidad dentro de la misma región o a la distribución de cargas de trabajo a través de varias regiones puede aumentar considerablemente la disponibilidad y la resistencia de las aplicaciones. Esta estrategia nos permitiría mantener la continuidad del negocio y minimizar el impacto frente a desastres naturales, cortes de energía o posibles fallos de red.

Ejemplo

Una plataforma de comercio electrónico decide utilizar múltiples zonas de disponibilidad dentro de la región de Azure "Este de EE. UU." para su infraestructura crítica. Al distribuir a través de estas zonas sus cargas de trabajo, como bases de datos y servidores web, la plataforma puede mantener su operación aun cuando una zona experimente un fallo, asegurando así una experiencia de compra ininterrumpida para sus clientes.

(D) Costes

Los precios de los servicios de Azure pueden variar entre las distintas regiones como consecuencia de las diferencias en los costes de gestión. Por tanto, seleccionar determinada región puede incidir en la estrategia de costes de la infraestructura en la nube, de forma que las empresas deben evaluar cuidadosamente los precios de los servicios en las diferentes regiones para optimizar sus gastos.

Ejemplo

Una startup tecnológica con operaciones globales, pero con un presupuesto limitado, realiza un análisis de costes y averigua que los precios de las máquinas virtuales son más reducidos en la región de Azure "Centro-Sur de EE. UU." que en la región "Oeste de EE. UU." Al desplegar su ambiente de desarrollo y pruebas en la región más económica, la startup consigue reducir sus costes operativos sin comprometer el rendimiento o la disponibilidad de sus servicios.

4.3 Suscripciones y administración de costes

Una suscripción de Azure funciona como un **acuerdo de facturación con Microsoft** y representa el principal nivel en el que se administran sus recursos y servicios. Cada suscripción se asocia a una **cuenta de facturación** y puede disponer de uno o varios grupos que contienen los recursos de Azure.

En este sentido, las organizaciones pueden mantener múltiples suscripciones bajo la misma cuenta para separar los entornos, departamentos o proyectos y, de este modo, facilitar la gestión y la asignación de costes.



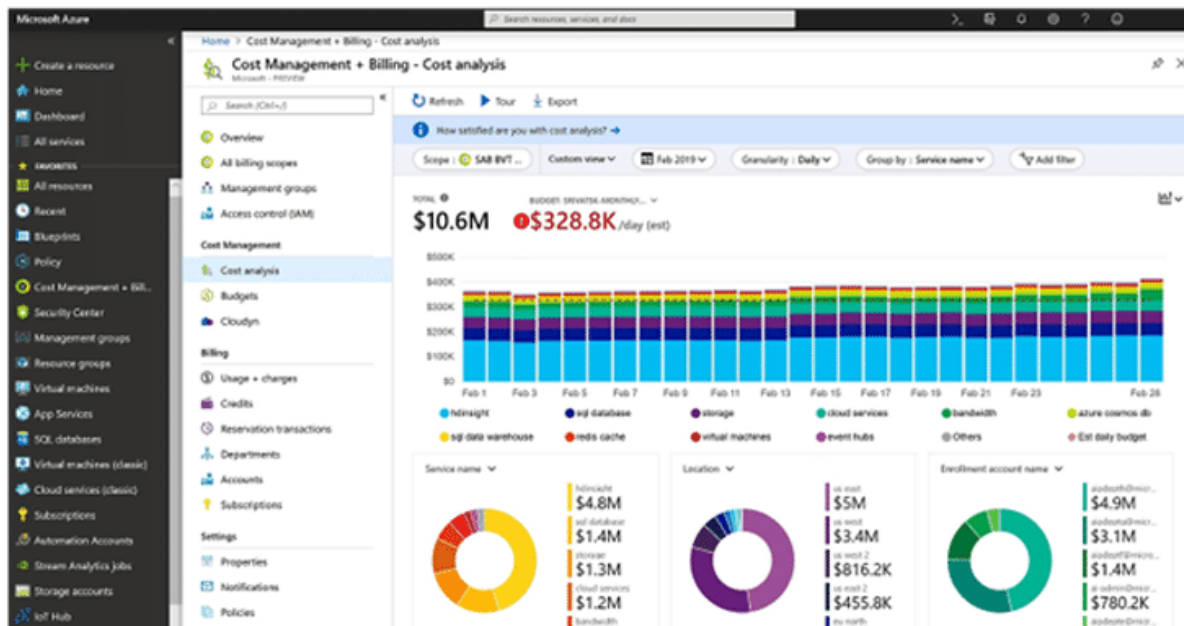
Suscripciones			
Directorio predeterminado			
+ Agregar Opciones avanzadas			
Buscar cualquier ca... Suscripciones : Filtrado (1 de 1) Mi rol == todo Estado == todo Agregar filtro			
Nombre de la suscripción ↑↓	Id. de la suscripción ↑↓	Mi rol ↑↓	Costo actual
Evaluación gratuita	06af7cc4-e1d5-488e-bd1d-40dec72876b5	Administrador de cuenta	Sin autorización

Figura 26. Captura de una suscripción.

4.3.1 Manejo de costes

Azure nos permite establecer presupuestos para nuestras suscripciones o grupos de recursos, lo que nos ayuda a **monitorear y controlar el gasto en la nube**. Recibimos alertas automáticas cuando alcanzamos ciertos umbrales, lo que previene gastos inesperados y nos permite tomar decisiones proactivas para ajustar recursos o escalas según nuestras necesidades.

La herramienta de análisis de costes de Azure nos proporciona una vista detallada de nuestro consumo y gastos, permitiéndonos **explorar y analizar estos costos a lo largo del tiempo**. Podemos identificar tendencias, detectar ineficiencias y comprender el desglose de costos por servicio, ubicación y otros criterios, lo que facilita la optimización de nuestro gasto.



4.3.2 Herramientas para el control del gasto

En Azure disponemos de una serie de herramientas que nos facilitaran el control del gasto. Pulsa los botones para saber que estas herramientas son:

- **Azure Advisor.** Ofrece **recomendaciones personalizadas para optimizar los recursos de Azure**. En términos de costes, Advisor puede sugerir acciones como reducir o eliminar recursos desaprovechados, reservar recursos para servicios que se utilizan constantemente (lo que puede representar ahorros significativos en comparación con los precios de pago por uso) u optimizar las configuraciones de los servicios para equilibrar el rendimiento y el coste
- **Reservas de Azure.** Las reservas de Azure permiten a los usuarios obtener un **descuento considerable** en ciertos servicios de Azure al comprometerse a un uso constante (por ejemplo, una máquina virtual) durante un período de uno o tres años.

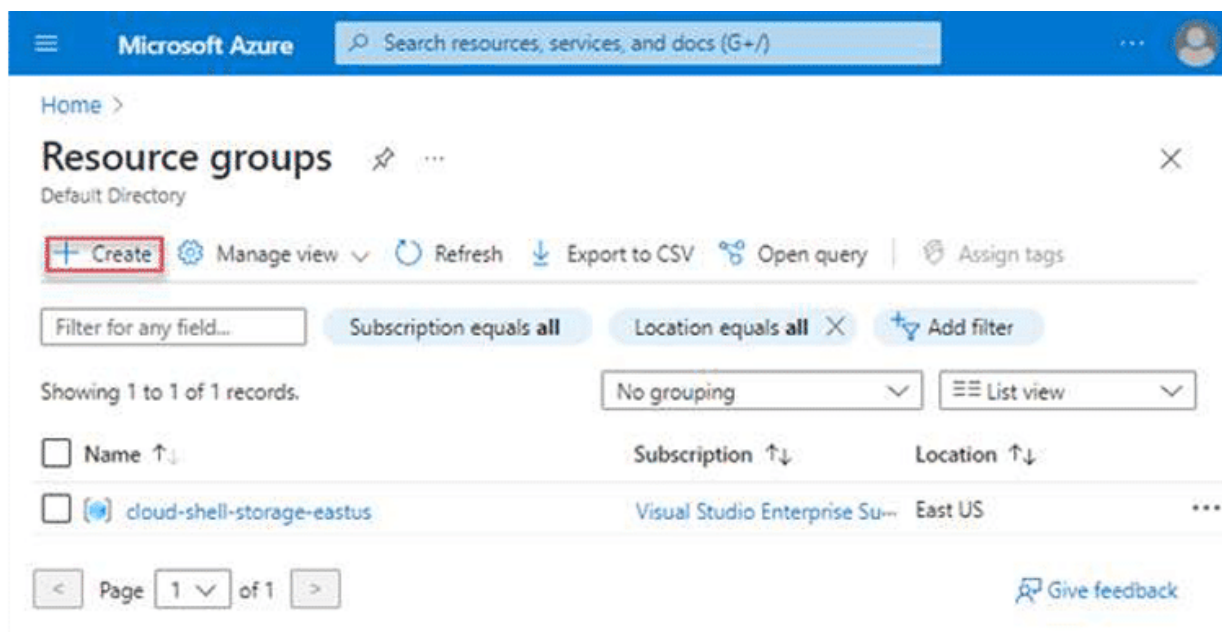
Este modelo resulta idóneo para **cargas de trabajo estables y predecibles** donde el compromiso a largo plazo puede traducirse en ahorros sustanciales en comparación con los precios bajo demanda.

- **Azure Hybrid Benefit.** Es una oferta de ahorro de costes para los clientes de Windows Server y *SQL Server con Software Assurance* que les permite utilizar sus licencias para ejecutar instancias de Windows Server o SQL Server en Azure con tarifas reducidas, adaptando la oferta a sus necesidades, lo que representa un ahorro considerable.

4.4 Grupos de recursos

Podemos imaginar un grupo de recursos como una caja donde guardamos todas las herramientas necesarias para un proyecto específico. En el contexto de la construcción, por ejemplo, podríamos disponer de una caja de herramientas para la electricidad, de otra para la fontanería y de una tercera para la carpintería. Por tanto, cada caja contiene todas las herramientas (recursos) requeridas para realizar tareas específicas dentro de ese proyecto.

De manera similar, un grupo de recursos en Azure agrupa **todos los recursos digitales** (como máquinas virtuales, aplicaciones web, bases de datos...) que necesitamos para **ejecutar y gestionar una aplicación o servicio en la nube**. Esta agrupación facilita la administración, ya que podemos tratar la caja (el grupo de recursos) como un **único elemento** de cara al despliegue, la gestión y la facturación.



The screenshot displays the Microsoft Azure portal interface for managing resource groups. The top navigation bar includes the Microsoft Azure logo and a search bar. The main heading is 'Resource groups' with a 'Default Directory' subtitle. Below the heading, there are several action buttons: 'Create' (highlighted with a red box), 'Manage view', 'Refresh', 'Export to CSV', 'Open query', and 'Assign tags'. A filter bar allows users to filter by subscription and location, both currently set to 'all'. The table below shows one resource group: 'cloud-shell-storage-eastus'. The table has columns for Name, Subscription, and Location. The bottom of the page shows pagination controls indicating 'Page 1 of 1' and a 'Give feedback' link.

Name	Subscription	Location
cloud-shell-storage-eastus	Visual Studio Enterprise Su...	East US

Figura 28. Grupo de recursos.

Cuando agrupamos recursos que comparten un mismo ciclo de vida en un grupo, simplificamos su **administración, despliegue, actualización y eliminación**. Esto es especialmente útil en escenarios donde necesitamos desplegar repetidamente entornos similares, como los de desarrollo, pruebas y producción. Los grupos de recursos también facilitan la implementación de **políticas de seguridad y control de acceso**.



Azure nos permite asignar permisos a nivel de grupo mediante el control de acceso basado en roles (RBAC), garantizando que solo los usuarios autorizados tengan acceso a los recursos del grupo.

Además, agrupar recursos relacionados nos facilita el seguimiento de los costes ya que Azure permite visualizar y gestionar el gasto a nivel de grupo, lo cual es útil para aquellas organizaciones que necesitan asignar gastos a departamentos, proyectos o iniciativas específicos.

Y de la misma forma, los grupos de recursos también nos permiten aplicar políticas y cumplimiento de manera consistente a todos los recursos del grupo, algo indispensable para organizaciones que deben adherirse a estrictas regulaciones de seguridad y conformidad, asegurando que todos los recursos cumplan con las mismas normativas.

En el caso de que trabajamos con grupos de recursos es recomendable mantener unas buenas prácticas para agilizar su uso:

- **Según el ciclo de vida y la gestión.** Agrupar recursos que se gestionan y se actualizan conjuntamente.
- **Por entorno.** Crear grupos de recursos separados para desarrollo, pruebas y producción.
- **Conformidad y seguridad.** Utilizar grupos de recursos para aplicar políticas de seguridad y conformidad de manera coherente.
- **Optimización de costes.** Agrupar recursos por departamento o proyecto para facilitar la asignación y el seguimiento de costes.

4.5 Principales recursos en Azure

En Azure, disponemos de una variedad de recursos que facilitan la creación, gestión y escalado de aplicaciones y servicios. Los principales recursos que encontramos:

- Máquinas virtuales (VMs).
- Almacenamiento.
- Redes.
- Bases de datos.
- Azure Active Directory.

Aunque a continuación entraremos en detalle en cada uno de estos, debemos señalar también que además de los recursos citados, Azure dispone también de servicios de contenedores, de IA y de monitorización y gestión.

4.5.1 Máquinas virtuales

Las máquinas virtuales (VM), es uno de los principales servicios que ofrece Azure, y que nos permite **desplegar y gestionar servidores virtuales en la nube**, proporcionándonos la flexibilidad de ejecutar una amplia gama de aplicaciones y sistemas operativos.

(A) Tipos de VM

(A) Tipos de VM

Dentro de Azure podemos encontrar diversos tipos de máquinas virtuales que se adaptan a diferentes cargas de trabajo:

1

Series A, B, D, E y F. Optimizadas para propósitos generales, ofrecen un equilibrio entre memoria, CPU y recursos de almacenamiento.

2

Series H. Especializadas en cargas de trabajo de alto rendimiento como simulaciones científicas, modelado financiero y análisis de datos.

3

Series N. Con GPU integradas, han sido diseñadas para aplicaciones intensivas en gráficos y visualización, renderizado 3D, así como para el aprendizaje automático y la inteligencia artificial.

Seleccionar un tipo de carga de trabajo

Uso general (serie D) predeterminado	Optimizada para memoria (serie E)	Optimizada para proceso (serie F)
 Example sizes DS2_v2: 2 CPU, 7 GB DS3_v2: 4 CPU, 14 GB	 Example sizes E2s_v3: 2 CPU, 16 GB E4s_v3: 4 CPU, 32 GB	 Example sizes F2s_v2: 2 CPU, 4 GB F4s_v2: 4 CPU, 8 GB
 CPU rápidas con configuración de CPU a memoria óptima	 Relación elevada de memoria a núcleo, optimizada para aplicaciones en memoria de uso intensivo	 Relación elevada entre CPU y memoria, optimizada para cargas de trabajo con muchos procesos
 Workload types Aplicaciones empresariales, bases de datos relacionales, análisis	 Workload types SAP HANA, Hekaton de SQL, otras cargas de trabajo en memoria de gran tamaño	 Workload types Procesamiento por lotes, servidores web, juegos

Figura 29. Selección de carga de trabajo para una VM.

Las configuraciones de las VM pueden personalizarse ajustando la **cantidad de vCPU** (versión virtual de un núcleo del procesador físico), la **memoria RAM**, el **almacenamiento** y las **opciones de red** con el objeto de satisfacer las necesidades específicas de sus aplicaciones.

Azure nos proporciona una **amplia biblioteca de sistemas operativos**, incluyendo Windows Server, Linux y versiones optimizadas de SQL Server y SAP HANA. Además de estos, también podemos recurrir a extensiones para configurar, administrar y mejorar las VM, automatizando tareas como la instalación de software y la configuración de seguridad.

(B) Optimización de máquinas virtuales

- **Escalado.** Denominamos escalado a la capacidad de adaptarse al rendimiento o a la carga de trabajo en diferentes momentos. Para manejar variaciones en la carga de trabajo, Azure ofrece capacidades de escalado automático, lo que permite que las VM se escalen de forma vertical u horizontal en función de las necesidades.
- **Escalado horizontal.** Capacidad de ampliar el hardware del servidor, como la RAM, el vCPU o el almacenamiento.
- **Escalado vertical.** Se consigue ampliando el número de servidores que atienden una aplicación.
- **Reservas de instancias.** Los usuarios pueden reservar instancias de VM por un período de uno o tres años y, de este modo, obtener un descuento significativo en comparación con los precios de pago por uso, optimizando así los costes para cargas de trabajo estables y predecibles.
- **Monitoreo y diagnóstico.** Azure proporciona herramientas integradas de monitoreo y diagnóstico para evaluar el rendimiento de las VM, ayudando a identificar y resolver problemas rápidamente. Profundizaremos en este aspecto más adelante.

(C) Escenarios de uso

Las **máquinas virtuales (VM)** nos permiten configurar rápidamente entornos aislados y reproducibles para desarrollo y pruebas, lo que acelera nuestro ciclo de desarrollo de software. También resultan útiles para alojar aplicaciones web y bases de datos, ya que nos brindan control total sobre la configuración del servidor y el entorno de ejecución.

Además, para cargas de trabajo que requieren alto rendimiento en el procesamiento de grandes volúmenes de datos y análisis en tiempo real, las VM nos ofrecen la potencia computacional necesaria. Por otro lado, las empresas que buscan migrar aplicaciones a la nube sin modificar su

arquitectura pueden recurrir a las VM para facilitar este proceso, aprovechando la flexibilidad y escalabilidad que Azure nos ofrece.

Datos básicos	
Suscripción	Evaluación gratuita
Grupo de recursos	Tokio-VM
Nombre de máquina virtual	Tokio
Región	North Europe
Opciones de disponibilidad	Zona de disponibilidad
Zona de disponibilidad	1
Tipo de seguridad	Estándar
Imagen	Ubuntu Server 20.04 LTS - Gen2
Arquitectura de VM	Arm64
Tamaño	Standard B2pts v2 (2 vcpu, 1 GiB de memoria)
Tipo de autenticación	Clave pública SSH
Nombre de usuario	azureuser
Nombre de par de claves	Tokio_key
Azure de acceso puntual	No

[Crear](#) [< Anterior](#) [Siguiente >](#) [Descargar una plantilla para la automatización](#)

Figura 30. Ejemplo de configuración básica en Azure.

4.5.2 Almacenamiento en Azure

En Azure, el almacenamiento proporciona **soluciones flexibles, escalables y seguras** para satisfacer una amplia gama de necesidades. Así, ofrece diversas opciones para almacenar desde archivos simples hasta datos masivos y altamente accesibles con la finalidad de optimizar el rendimiento y garantizar la seguridad de la información.

Dentro de Azure podemos encontrar diferentes opciones de almacenamiento, pulsa los botones para verlas:

- **Azure Blob Storage.** Resulta idóneo para almacenar **cantidades masivas de datos no estructurados**, como textos o archivos binarios. Se trata de la opción más generalista de almacenamiento en Azure y es compatible con cientos de aplicaciones y proveedores para consumir o añadir datos.

Además, es altamente escalable y accesible desde cualquier lugar del mundo y añade multitud de opciones avanzadas como replicación, seguridad de los datos, eliminación automática, etc.

- **Azure Files.** Ofrece servicios de carpetas compartidas en la nube, accesibles mediante el protocolo SMB o NFS. Azure Files resulta perfecto para **migraciones de aplicaciones que dependen de sistemas de archivos compartidos**, así como para compartir datos entre **diferentes servicios de Azure**.

Se trata de una opción ampliamente adoptada en aquellas empresas que necesitan carpetas compartidas entre todos los equipos de la empresa.

- **Azure Queues.** Proporciona almacenamiento de **colas de mensajes para la comunicación entre componentes de aplicación**, facilitando la creación de aplicaciones distribuidas y escalables.

- **Azure Table Storage.** Base de datos NoSQL que almacena **datos no estructurados y semiestructurados**. Representa una solución eficiente para desarrollar aplicaciones que requieren una base de datos con un esquema flexible y la capacidad de escalar de manera elástica.

- **Azure Disk Storage.** Ofrece discos duros virtuales, que representan la solución de almacenamiento para las **máquinas virtuales de Azure**. Los discos pueden ser de HDD estándar, SSD estándar, SSD premium y SSD ultra, adecuados para cargas intensivas de trabajo de lectura y escritura.

Según el servicio utilizado y la configuración establecida el rendimiento del almacenamiento puede variar. Así, por ejemplo, Azure Blob Storage puede optimizarse para acceso frecuente o infrecuente, cuyos precios y rendimiento difieren.

Seguridad en los datos: —

Azure implementa múltiples capas de seguridad para proteger los datos almacenados:

- **Cifrado:** todos los datos almacenados en los servicios de Azure Storage se encuentran cifrados por defecto mediante Azure Storage Service Encryption (SSE) para protegerlos en reposo. Además, los usuarios pueden optar por un cifrado del lado del cliente para obtener una seguridad adicional.
- **Control de acceso:** azure utiliza el control de acceso basado en roles (RBAC) y sus políticas para gestionar el acceso a los datos de almacenamiento. Las claves y los tokens de acceso compartido disponen de controles muy detallados sobre quién cuenta con acceso a los datos. También ampliaremos este aspecto más adelante.
- **Red:** las opciones de red, como los puntos de conexión privados y el servicio Azure Private Link, permiten a los usuarios controlar cómo se accede a los datos desde la red, reduciendo la exposición a amenazas externas.
- **Protección de datos:** azure ofrece capacidades robustas de copia de seguridad y recuperación ante desastres, incluyendo redundancia de datos en múltiples ubicaciones geográficas para garantizar su durabilidad y disponibilidad.

Como vemos, el almacenamiento proporciona una solución completa para una amplia variedad de necesidades, desde archivos simples hasta datos complejos y de alto rendimiento. Aprovechando las diferentes opciones de almacenamiento, junto con configuraciones avanzadas de rendimiento y sólidas medidas de seguridad, podemos diseñar soluciones que no solo cumplan con los requisitos operativos, sino que también protejan los datos contra amenazas y garanticen su disponibilidad en cualquier circunstancia.

4.5.3 Redes en Azure

Azure Networking abarca un extenso conjunto de servicios y características diseñados para facilitar la creación, gestión y optimización de redes en la nube. De este modo, esta herramienta nos permite conectar aplicaciones de manera segura, tanto dentro de Azure como entre Azure, locales y otros entornos de nube, obteniendo una elevada disponibilidad, seguridad y rendimiento. Principales servicios que ofrece Azure Networking son:

- **Azure Virtual Network (VNet).** Las VNet nos permiten crear redes privadas en la nube donde poder desplegar recursos de Azure, como máquinas virtuales (VM) y bases de datos, garantizando una comunicación segura y aislada.

Además, dentro de una VNet, se pueden definir espacios de direcciones IP, subredes, políticas de seguridad y rutas personalizadas.

- **Azure Load Balancer.** Este servicio ofrece alta disponibilidad y distribuye el tráfico entrante entre los recursos de Azure, mejorando el rendimiento de las aplicaciones. Soporta tanto el balanceo de carga de nivel 4 (TCP, UDP) como configuraciones más avanzadas para gestionar el tráfico eficientemente.
- **Azure Application Gateway.** Se trata de un balanceador de carga que proporciona enrutamiento basado en URL, SSL y otras características de gestión de tráfico de aplicaciones web. Es ideal cuando se requiere un enrutamiento basado en contenido y una gestión de sesión más compleja como, por ejemplo, segmentar el tráfico por país de origen.
- **Azure VPN Gateway.** Permite la creación de conexiones seguras entre una red de Azure y otras locales a través de internet utilizando túneles VPN. Resulta útil para escenarios de red híbrida, ya que permite a las empresas extender sus redes locales a la nube de Azure de manera segura.
- **Azure ExpressRoute.** Ofrece conexiones privadas entre las redes de Azure y la infraestructura de TI local o en un centro de datos de la organización. ExpressRoute es ideal para aplicaciones críticas que requieren mayor seguridad, velocidad y fiabilidad que las conexiones a través de internet.
- **Azure DNS.** Proporciona alojamiento de DNS para dominios, permitiendo su resolución en direcciones IP de recursos de Azure, lo que resulta útil de cara a la accesibilidad y el rendimiento de las aplicaciones y servicios hospedados en la plataforma.

Así, por ejemplo, podemos exponer una máquina virtual a través de un dominio a internet y automatizar la redirección de DNS si la IP de la máquina virtual cambia.

- **Azure Front Door.** Se trata de un servicio global de entrega de contenido y balanceo de carga que permite definir, gestionar y monitorizar el

enrutamiento conjunto del tráfico hacia los recursos de Azure, optimizando la velocidad y la fiabilidad del acceso a aplicaciones y datos.

- **Azure Network Security Group (NSG).** Los NSG se utilizan para filtrar el tráfico de red hacia y desde los recursos de Azure dentro de una VNet en base a reglas de seguridad definidas, lo que resulta útil para proteger las aplicaciones y servicios en la nube contra accesos no autorizados y ataques.
- **Azure Firewall.** Servicio de seguridad de red administrado que proporciona protección avanzada contra amenazas, permitiendo a las empresas asegurar sus recursos de Azure mediante políticas centralizadas y registro detallado.

Gracias a todos estos servicios, Azure Networking ofrece un marco completo y avanzado de cara a la construcción de redes en la nube, ya que proporciona las herramientas y servicios necesarios para crear **soluciones de red seguras, escalables y altamente disponibles**.

De esta forma, se facilita a las organizaciones la implementación de arquitecturas de red que soportan sus necesidades operativas y estratégicas, maximizando el rendimiento y la eficiencia en un entorno de nube global.

4.5.4 Bases de datos en Azure

Paralelamente, Azure proporciona una amplia gama de bases de datos para satisfacer diversas necesidades de almacenamiento, desde aplicaciones empresariales hasta soluciones modernas e innovadoras.

(A) Servicios de BBDD

Servicios de BBDD —

- **Azure SQL Database:** base de datos relacional como servicio (DBaaS) basada en SQL Server.

- **Azure Cosmos DB:** base de datos NoSQL distribuida globalmente para aplicaciones de alta disponibilidad y escalabilidad.
- **Azure Database for MySQL y PostgreSQL:** servicios gestionados que ofrecen bases de datos relacionales basadas en MySQL y PostgreSQL.
- **Azure Table Storage:** almacenamiento NoSQL para datos no estructurados y semiestructurados.
- **Azure SQL Managed Instance:** instancia de SQL Server totalmente gestionada en la nube.
- **Azure Cache for Redis:** almacenamiento de datos en memoria orientado a aquellos escenarios que requieren un rápido acceso a la información.

(B) Aplicaciones de los servicios

Veamos a continuación algunos casos de uso y ejemplos de estos servicios.

1

Aplicaciones empresariales tradicionales

Para un sistema de gestión de recursos humanos (HRMS) que necesita transacciones consistentes y confiables, Azure SQL Database sería una solución ideal.

Por ejemplo, una empresa puede migrar su base de datos local de HRMS a Azure SQL Database, aprovechando la alta disponibilidad y el ajuste automático de rendimiento. Esto facilita un acceso remoto seguro para su equipo, que está disperso geográficamente.

Aplicaciones móviles y web globales

Imaginemos ahora una aplicación móvil de redes sociales que necesita almacenar y recuperar grandes volúmenes de datos no estructurados distribuidos globalmente, Azure Cosmos DB sería el servicio adecuado.

Por ejemplo, una startup de tecnología puede utilizar Azure Cosmos DB para su aplicación de redes sociales, beneficiándose de la replicación global y del acceso de baja latencia a los datos por parte de los usuarios desde cualquier parte del mundo.

Migración de aplicaciones a la nube

Otro caso sería migrar una aplicación basada en LAMP (Linux, Apache, MySQL, PHP) a la nube sin modificar la arquitectura de la aplicación, para este supuesto Azure Database for MySQL sería una excelente opción.

Por ejemplo, un sitio web de comercio electrónico puede migrar su base de datos MySQL a Azure Database for MySQL para mejorar la escalabilidad y la seguridad, manteniendo la compatibilidad con su pila tecnológica actual.

Almacenamiento de datos de telemetría

Supongamos ahora la recopilación y análisis de grandes volúmenes de datos de telemetría de dispositivos IoT. En este caso Azure Table Storage sería la opción adecuada.

Por ejemplo, una empresa de fabricación de dispositivos inteligentes puede utilizar Azure Table Storage para almacenar datos de telemetría enviados por millones de dispositivos, facilitando su análisis y seguimiento en tiempo real.

Modernización de aplicaciones empresariales

Para empresas que buscan modernizar sus aplicaciones SQL Server efectuando las mínimas modificaciones posibles, Azure SQL Managed Instance ofrece una solución eficiente.

Por ejemplo, un banco puede modernizar su aplicación de procesamiento de transacciones migrando a Azure SQL Managed Instance, beneficiándose de las características de la nube, como la escalabilidad y la recuperación ante desastres, sin necesidad de cambiar el código de la aplicación.

Mejora del rendimiento de aplicaciones

Pongamos ahora el caso de aplicaciones web que requieren tiempos de carga rápidos y baja latencia en el acceso a los datos, para un supuesto como este Azure Cache for Redis sería una excelente opción.

Por ejemplo, un portal de noticias puede utilizar Azure Cache for Redis para almacenar en caché los contenidos más populares, lo que reduce significativamente los tiempos de carga de la página y mejora la experiencia del usuario durante picos de tráfico.

4.5.5 Azure Active Directory

Azure Active Directory (AAD) es el servicio de Microsoft basado en la nube para la **gestión de identidades y accesos**. Ofrece un conjunto de capacidades destinadas a facilitar la administración de usuarios y grupos, así como a asegurar el acceso a aplicaciones dentro de la organización y en la nube.

AAD permite a las empresas implementar un **control de acceso basado en roles (RBAC)**, **autenticación multifactor (MFA)** y **políticas de seguridad condicional**, entre otras características avanzadas.

A continuación, exploraremos el uso avanzado de AAD, sus integraciones y cómo contribuye a reforzar la seguridad de cara a la gestión de identidades.

**USO AVANZADO DE AZURE
ACTIVE DIRECTORY.**

INTEGRACIONES

**SEGURIDAD DE CARA A LA
GESTIÓN DE IDENTIDADES**

Azure AD, mejora considerablemente la seguridad al aplicar la **autenticación multifactor (MFA)**, es decir, requiere de **dos o más métodos de verificación** antes de conceder acceso a los usuarios. Dichos métodos pueden contemplar un elemento que el usuario conoce (una contraseña), un elemento que el usuario posee (un teléfono o token de hardware) o un elemento que forma parte del usuario (biometría).

Además, este servicio también permite implementar un **control de acceso basado en roles (RBAC)**, proporcionando a los administradores la capacidad de asignar permisos a usuarios, grupos y aplicaciones a un nivel granular basado en roles específicos.

De este modo, se garantiza que los usuarios solo obtengan el acceso que realmente necesitan para desempeñar sus funciones. Por ejemplo, podemos crear un rol para que los usuarios que cuenten con él solo dispongan de acceso a un grupo de recursos específico en nuestra suscripción de Azure.

A estas medidas, se le suman también las **políticas de seguridad condicionales** que permiten a los administradores definir y forzar reglas y procedimientos de acceso basados en condiciones específicas, como la ubicación del usuario, el dispositivo que está utilizando o el riesgo de la sesión.

**USO AVANZADO DE AZURE
ACTIVE DIRECTORY.**

INTEGRACIONES

**SEGURIDAD DE CARA A LA
GESTIÓN DE IDENTIDADES**

Azure AD ofrece integración con **miles de aplicaciones SaaS** (Software as a Service) como Salesforce o Google Apps, entre otras muchas, permitiendo a los usuarios acceder a todas sus aplicaciones con una sola identidad.

Además, soporta **protocolos de identidad estándar** como SAML 2.0, OAuth 2.0 y OpenID Connect, lo que facilita la integración con una amplia variedad de aplicaciones modernas y personalizadas.

También es el sistema de identidad detrás de Microsoft 365 y Office 365, proporcionando autenticación y autorización para todos los productos de Microsoft 365, incluyendo Exchange Online, SharePoint Online y OneDrive.

**USO AVANZADO DE AZURE
ACTIVE DIRECTORY.**

INTEGRACIONES

**SEGURIDAD DE CARA A LA
GESTIÓN DE IDENTIDADES**

Azure AD Identity Protection emplea aprendizaje automático para detectar anomalías y actividades sospechosas relacionadas con las identidades de los usuarios, ofreciendo capacidades de evaluación de riesgos y remediación automática.

Además, las herramientas de gobierno de identidades implementadas en Azure AD permiten a las organizaciones garantizar que únicamente los usuarios correctos obtengan acceso a los recursos correspondientes, lo que contempla la administración de privilegios y revisiones de acceso, así como la de privilegios excesivos.

Por último, Azure AD también proporciona informes detallados y herramientas de análisis para que los administradores puedan entender los patrones de acceso, investigar incidentes y mejorar el nivel de seguridad general de la organización.

Tema 5: Despliegue en Azure



Introducción



5.1 Qué consideramos un despliegue



5.2 Herramientas y estrategias



5.3 Introducción a Azure Portal



5.4 Introducción a Azure CLI



5.5 Plantillas ARM: automatización y gestión de infraestructuras en Azure

Introducción

A continuación, nos adentraremos en el proceso de despliegue en Azure, explorando desde los fundamentos de qué consideramos un despliegue hasta las estrategias y herramientas específicas que Azure pone a nuestra disposición para llevar a cabo esta tarea de manera efectiva.

El despliegue, en el contexto de **desarrollo de software y operaciones de TI (DevOps)**, es un proceso clave que no solo involucra la distribución y entrega de aplicaciones o sistemas de software, sino también su puesta en funcionamiento de manera que permanezcan disponibles y operativos para los usuarios finales.

Este proceso abarca una serie de pasos que van desde la preparación y envío del código desde el entorno de desarrollo hasta su implementación en el entorno de producción. En este módulo, nos centraremos en el despliegue de infraestructura y herramientas dentro del ecosistema de Azure, una plataforma que, gracias a su versatilidad, se adapta a una amplia gama de necesidades y escenarios de despliegue.

La relevancia de este tema radica en la creciente necesidad de las organizaciones de desplegar rápidamente aplicaciones y servicios en la nube para responder a las demandas del mercado y mantenerse competitivas. Azure ofrece la flexibilidad y las capacidades necesarias para adaptarse a diversos escenarios de despliegue, desde proyectos pequeños hasta aplicaciones empresariales complejas.

A través de este tema, los estudiantes adquirirán una comprensión profunda de las herramientas y estrategias de despliegue en Azure, incluyendo Azure Portal, Azure CLI, Plantillas ARM, integraciones con servicios de terceros y la API de Azure.

Este conocimiento no solo es fundamental para los profesionales de IT y desarrolladores, sino también para aquellos en roles de gestión y operaciones, ya que la implementación de despliegues eficientes afecta a toda la cadena de valor del software.

Al finalizar el tema, los estudiantes estarán equipados con las herramientas y conocimientos necesarios para planificar e implementar despliegues en Azure, mejorando la colaboración entre equipos, la eficiencia en los procesos y la calidad del producto final.

5.1 Qué consideramos un despliegue

Dentro del contexto de desarrollo de software y operaciones de TI (DevOps), denominamos despliegue al **proceso mediante el cual se distribuye, entrega y pone en funcionamiento una aplicación o un sistema de software.**



Este proceso abarca tanto la preparación y el envío del código desde el entorno de desarrollo como su implementación en el entorno de producción asegurando que la aplicación permanezca disponible y operativa para los usuarios finales. En este módulo, nos centraremos en el **despliegue de infraestructura** (los distintos servicios de Azure) **y de posibles herramientas.**

En Azure, dada la versatilidad y el amplio espectro de herramientas que ofrece, los despliegues pueden adaptarse a una gran variedad de necesidades y escenarios: desde un simple despliegue de una aplicación web hasta la creación de infraestructuras complejas que involucran múltiples servicios y recursos. A continuación, profundizaremos en las principales herramientas y estrategias que proporciona para facilitar dichos despliegues

5.2 Herramientas y estrategias

Al considerar la implementación de despliegues en Azure, es importante que seleccionemos la estrategia que mejor se alinee con los objetivos de nuestro proyecto, el nivel de complejidad de la infraestructura y las preferencias del equipo de desarrollo.

Tanto para proyectos pequeños como para aplicaciones empresariales, Azure nos ofrece la flexibilidad y las capacidades necesarias para adaptarse a una amplia gama de escenarios de despliegue.



5.2.1 Azure Portal

Azure Portal es la **interfaz gráfica web** que permite a los usuarios **visualizar, crear y administrar los recursos de Azure**. Es una herramienta idónea para aquellos usuarios que están introduciéndose en Azure o para aquellos que prefieren una interacción visual de cara a la gestión de sus recursos.

Por medio del portal, los usuarios pueden realizar **despliegues manuales, configurar servicios y monitorear el estado y el rendimiento de sus aplicaciones y servicios en tiempo real** desde una interfaz web amigable.

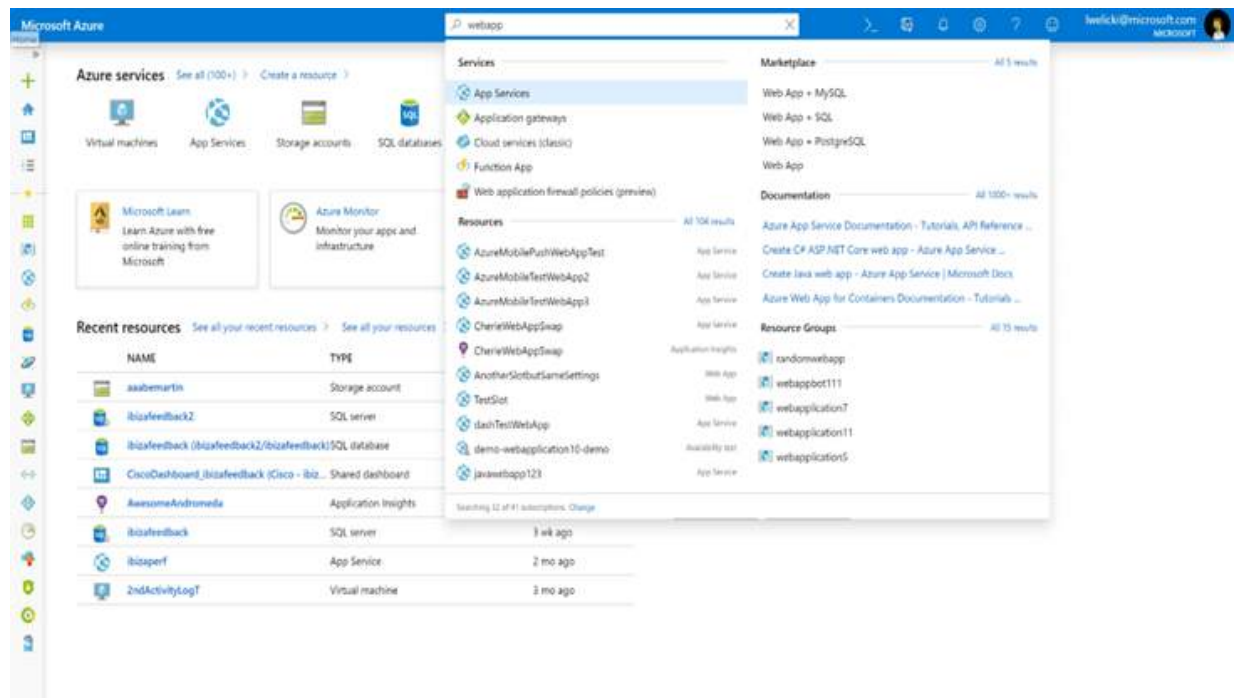


Figura 31. Captura de Azure Portal.

5.2.2. Azure CLI

Azure CLI es una **herramienta de línea de comandos** para operar en Azure a través de la terminal o de scripts automatizados. Aquellos usuarios que prefieren una experiencia más automatizada o trabajan dentro de entornos de desarrollo integrados (IDE), Azure Command Line Interface (CLI) les ofrece un poderoso conjunto de comandos para administrar los recursos de Azure.

Se trata de una buena opción para **scripting y automatización**, pues permite ejecutar secuencias de comandos destinados a desplegar, configurar y administrar recursos de Azure de manera programática.



Welcome to the cool new Azure CLI!

Use `az --version` to display the current version.

Here are the base commands:

account	: Manage Azure subscription information.
acr	: Manage private registries with Azure Container Registries.
ad	: Manage Azure Active Directory Graph entities needed for Role Based Access Control.
advisor	: Manage Azure Advisor.
afd	: Manage Azure Front Door Standard/Premium.
aks	: Manage Azure Kubernetes Services.
ams	: Manage Azure Media Services resources.
apim	: Manage Azure API Management services.
appconfig	: Manage App Configurations.
appservice	: Manage App Service plans.

5.2.3 Plantillas ARM

Azure Resource Manager (ARM) proporciona una forma **declarativa** (similar a Ansible o Terraform) de definir y administrar todos los recursos de Azure necesarios para nuestras aplicaciones. En este sentido, recurre a **plantillas JSON**, que permiten **definir infraestructuras complejas como código** (IaC), lo que facilita el despliegue repetible y consistente de entornos completos.

Utilizando ARM, los equipos pueden versionar su infraestructura junto con su código de aplicación, simplificando los procesos de integración y entrega continua (CI/CD).

Así, por ejemplo, si contamos con un servicio de Azure definido como una plantilla ARM, podemos desplegarla de forma automática siempre que lo necesitemos.



Azure presenta la particularidad de que permite generar la plantilla de cualquier servicio ya desplegado, lo que favorece la adopción de este método.

```

1 {
2   "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
3   "contentVersion": "1.0.0.0",
4   "parameters": {
5     "storageAccounts_test18f17cterraform_name": {
6       "defaultValue": "test18f17cterraform",
7       "type": "String"
8     }
9   },
10  "variables": {},
11  "resources": [
12    {
13      "type": "Microsoft.Storage/storageAccounts",
14      "apiVersion": "2023-01-01",
15      "name": "[parameters('storageAccounts_test18f17cterraform_name')]",
16      "location": "northeurope",

```

Figura 33. Ejemplo de JSON.

5.2.4 Integraciones con servicios de terceros

Azure se integra con una amplia gama de herramientas y servicios de terceros, como GitHub, Jenkins y Terraform, ofreciendo **flexibilidad** a los equipos de desarrollo a la hora de implementar sus flujos de trabajo de CI/CD mediante aquellas herramientas con las que ya se encuentran familiarizados.

Estas integraciones facilitan la **automatización del ciclo de vida del *software***, desde el desarrollo hasta la producción, mejorando la eficiencia y la consistencia de los despliegues.

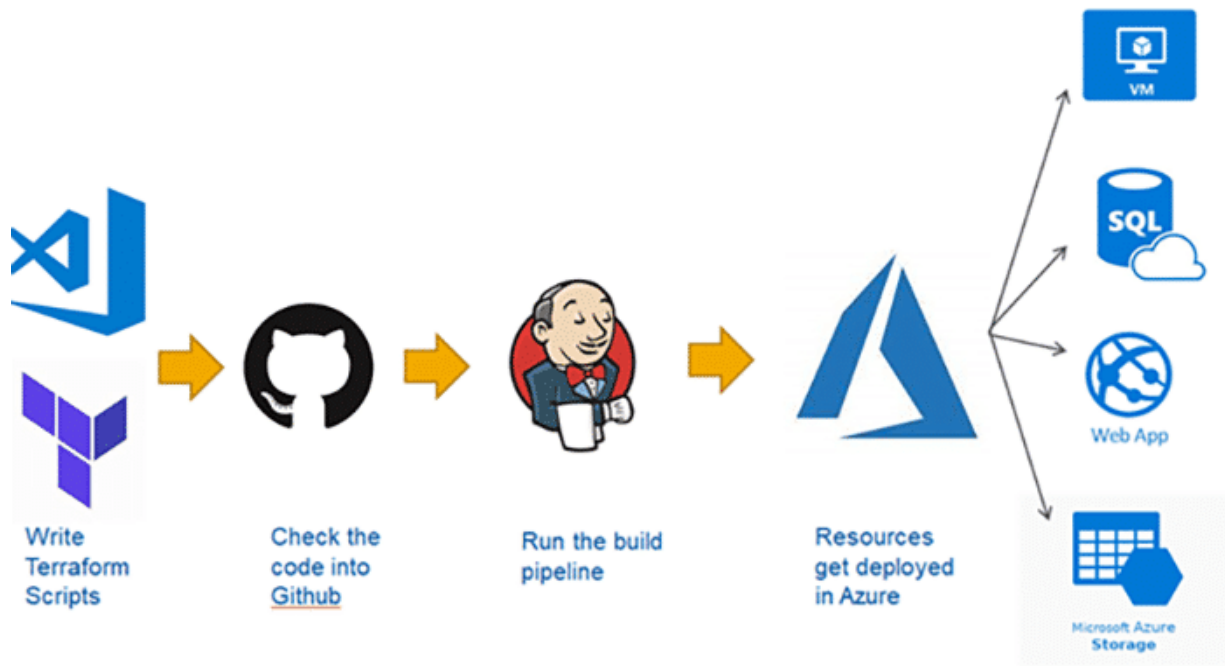


Figura 34. Integraciones con terceros.

5.2.5 API de Azure

La API de Azure proporciona **acceso por código** a todas las funcionalidades de la plataforma, permitiendo a los desarrolladores integrar su gestión directamente en sus aplicaciones. Por tanto, resulta útil para casos de uso avanzados que requieren **interacciones personalizadas con los servicios de Azure** o bien de cara a la **automatización de tareas específicas** dentro de aplicaciones empresariales.

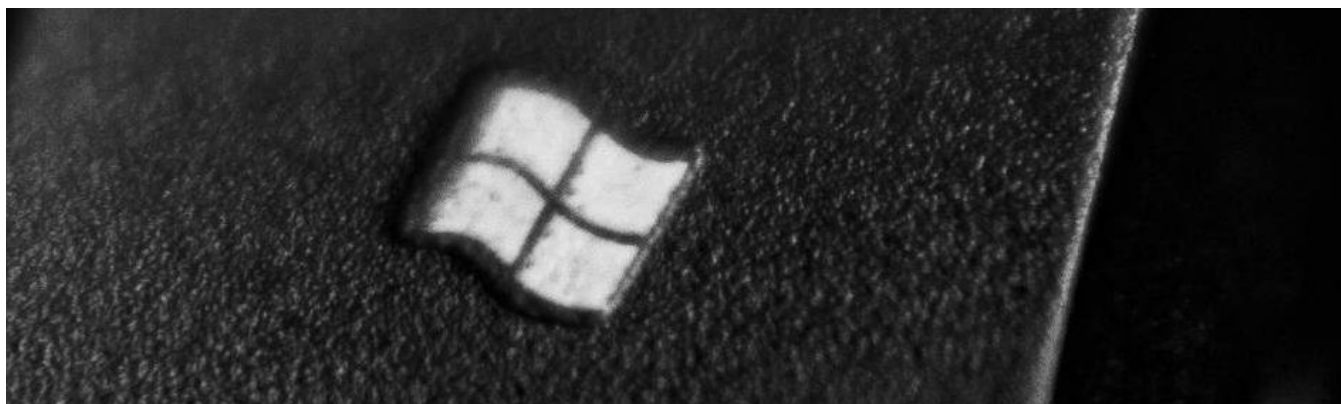
Cuando valoramos la implementación de despliegues en Azure, resulta importante seleccionar la estrategia y las herramientas que se alineen en mayor medida con los objetivos de nuestro proyecto, con el nivel de complejidad de la infraestructura y con las preferencias del equipo de desarrollo.

Ya sea para proyectos pequeños a través de **despliegues manuales con Azure Portal** o para la implementación de infraestructuras complejas mediante IaC (infraestructura como código) con Terraform o plantillas ARM en aplicaciones empresariales, Azure proporciona la flexibilidad y las capacidades necesarias para adaptarse a una amplia gama de escenarios de despliegue.

5.3 Introducción a Azure Portal

Como hemos mencionado anteriormente, Azure Portal es una interfaz de usuario basada en la web que ofrece un acceso centralizado a todos los servicios y recursos de Microsoft Azure. Como centro neurálgico para la administración de la infraestructura de la nube, el portal simplifica la gestión de aplicaciones, servicios y dispositivos.

Para acceder a Azure Portal, simplemente nos dirigiremos a <https://portal.azure.com> y utilizaremos nuestras credenciales de Microsoft para iniciar sesión. Al ingresar, nos encontraremos con el **panel de control principal**, que nos proporciona una **vista general de nuestros recursos, métricas de servicio y accesos directos a herramientas de administración**.



5.3.1 Navegación y personalización del panel de control

El dashboard del Azure Portal es absolutamente personalizable. En este sentido, podemos añadir, eliminar o reorganizar los mosaicos para adaptar la vista a nuestras necesidades específicas.

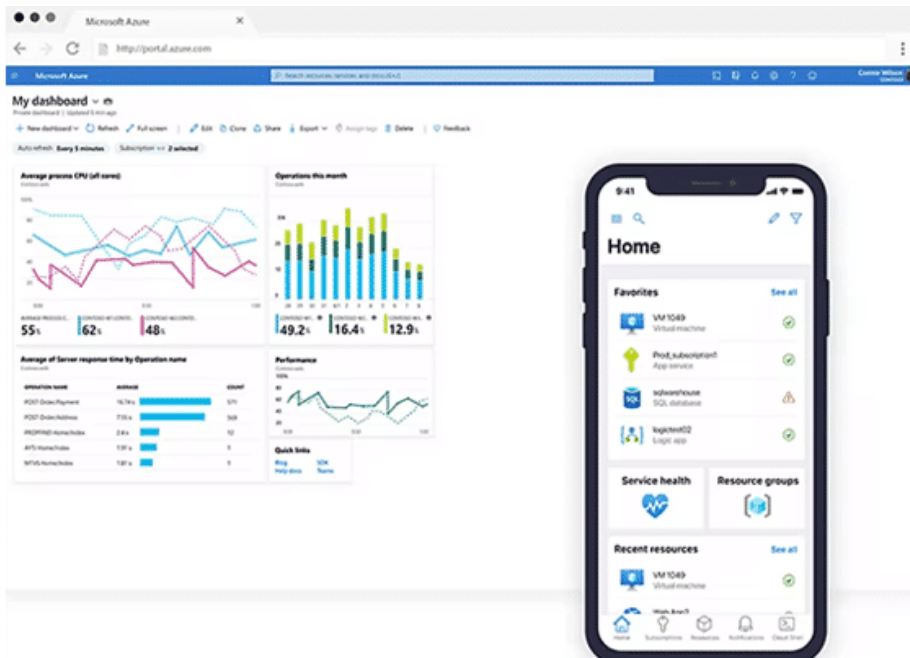


Figura 35. Dashboard de Azure Portal.

5.3.2 Creación y gestión de recursos

Crear y gestionar recursos a través del portal de Azure es un proceso sumamente intuitivo. Así, para crear un nuevo recurso, seguiremos los pasos que se indican a continuación.

(A) Crear un recurso

Clicamos en el botón + Crear un recurso, situado en la esquina superior izquierda del portal, lo que nos dirigirá al *marketplace* de Azure, donde podemos buscar y seleccionar el tipo de recurso que deseamos crear.



Figura 36. Crear un recurso.

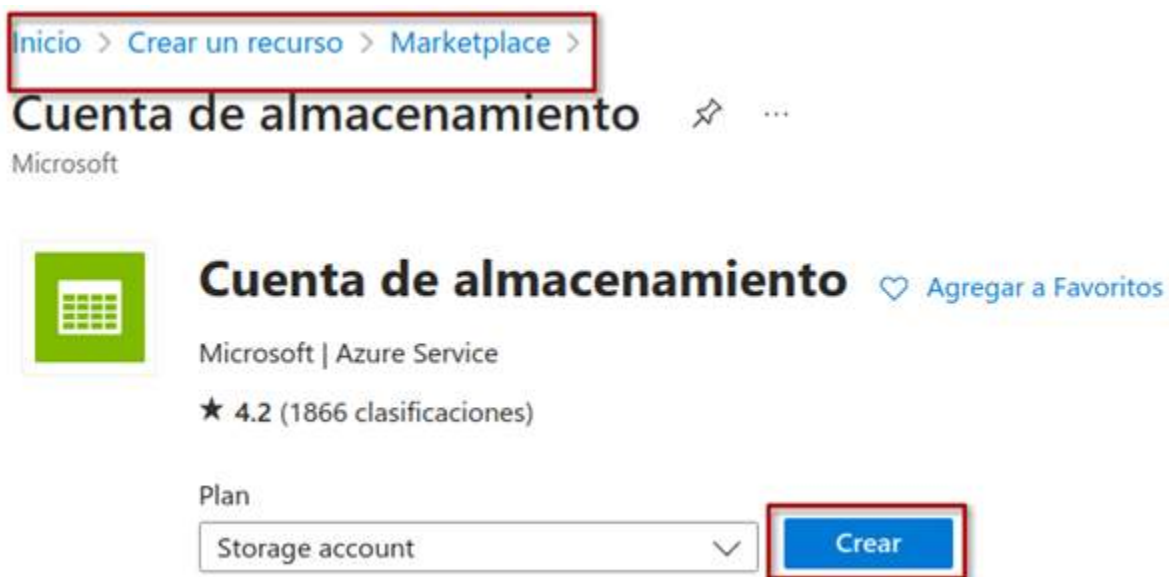


Figura 37. Selección del recurso.

(B) Configuración del recurso

Cada tipo de recurso posee su propio formulario de configuración. En este sentido, debemos proporcionar información básica, como el nombre del recurso, la región (ubicación del centro de datos) y cualquier configuración específica del servicio.

Este paso resulta fundamental, ya que es donde se definen los **atributos esenciales** que determinarán **cómo se desplegará y operará el recurso dentro de Azure**. Destacan campos obligatorios como el nombre del recurso, la región (o ubicación del centro de datos) y, posiblemente, otras opciones específicas del servicio que se está configurando. Se trata de campos esenciales por los siguientes motivos:

1

Nombre del recurso. Funciona como identificador único dentro de nuestra suscripción de Azure y ha de ser descriptivo, pero también fácil de recordar.

2

Región. Como ya hemos señalado, la elección de la región afecta tanto a la latencia de la aplicación como a la conformidad con las regulaciones de datos. Por consiguiente, resulta importante elegir una región cercana a nuestros usuarios finales o que reúna los requisitos específicos de residencia de datos.

3

Configuración específica del servicio. En Azure, cada tipo de servicio posee su propio conjunto de configuraciones que pueden incidir en diversos aspectos, desde el rendimiento hasta la seguridad.

Antes de finalizar, revisaremos la configuración y clicaremos en Crear para desplegar nuestro recurso.

Crear una cuenta de almacenamiento

[Datos básicos](#)[Opciones avanzadas](#)[Redes](#)[Protección de datos](#)[Cifrado](#)[Etiquetas](#)[Revisar](#)

Datos básicos

Suscripción	Evaluación gratuita
Grupo de recursos	Tokio-Resource-Group
Ubicación	northeurope
Nombre de la cuenta de almacenamiento	tokioschoolstorage
Modelo de implementación	Resource Manager
Rendimiento	Standard
Replicación	Almacenamiento con redundancia local (LRS)

Opciones avanzadas

Habilitar el espacio de nombres jerárquico	Deshabilitado
Habilitar el sistema de archivos de red v3	Deshabilitado
Permitir replicación entre espacios empresariales	Deshabilitado
Nivel de acceso	Hot
Habilitar SFTP	Deshabilitado
Recursos compartidos de archivos grandes	Disabled

Crear

< Anterior

Siguiente >

[Descargar una plantilla para la automatización](#)

Figura 38. Vista previa antes de desplegar el recurso.

Una vez finalizada la configuración de un recurso, se accede a una pantalla como la que se muestra en la imagen superior, que es la **vista de resumen** o el **panel de detalles del recurso recién desplegado en Azure Portal**.

Este panel resulta útil para los administradores y desarrolladores, ya que ofrece una visión general inmediata de la configuración clave y del estado actual del recurso, facilitando su gestión y monitoreo desde el momento en que se completa su creación. La visualización incluye:

- **Nombre del recurso y tipo.** En la parte superior, se muestra el nombre único asignado durante la fase de configuración junto con el tipo de recurso, proporcionando visibilidad inmediata sobre qué servicio específico se ha configurado.
- **Detalles del recurso.** Se indican atributos básicos como la suscripción bajo la cual se creó el recurso, el grupo de recursos al que pertenece y la región en la que se encuentra desplegado. Estos detalles son esenciales para la gestión de costes, el cumplimiento de políticas y la optimización de la latencia.

- **Estado y métricas de estado.** Podemos visualizar el estado operativo del recurso, donde se indica si se encuentra en funcionamiento, junto con métricas clave de estado o rendimiento en función de su tipo. De este modo, podemos detectar rápidamente cualquier problema o verificar si dicho recurso funciona como debería.

- **Configuración y gestión.** La interfaz proporciona accesos directos a las opciones de configuración más relevantes y a herramientas de gestión para el recurso, como ajustes de tamaño, seguridad y conectividad. De este modo, las configuraciones críticas siempre permanezcan al alcance, facilitando modificaciones rápidas cuando resulte necesario.

- **Acciones y herramientas de diagnóstico.** Además, se proporcionan herramientas y opciones para realizar diagnósticos, establecer políticas de *backup* y configurar alertas, entre otros muchos aspectos, desde una ubicación centralizada, lo que mejora la eficiencia de la administración del recurso.

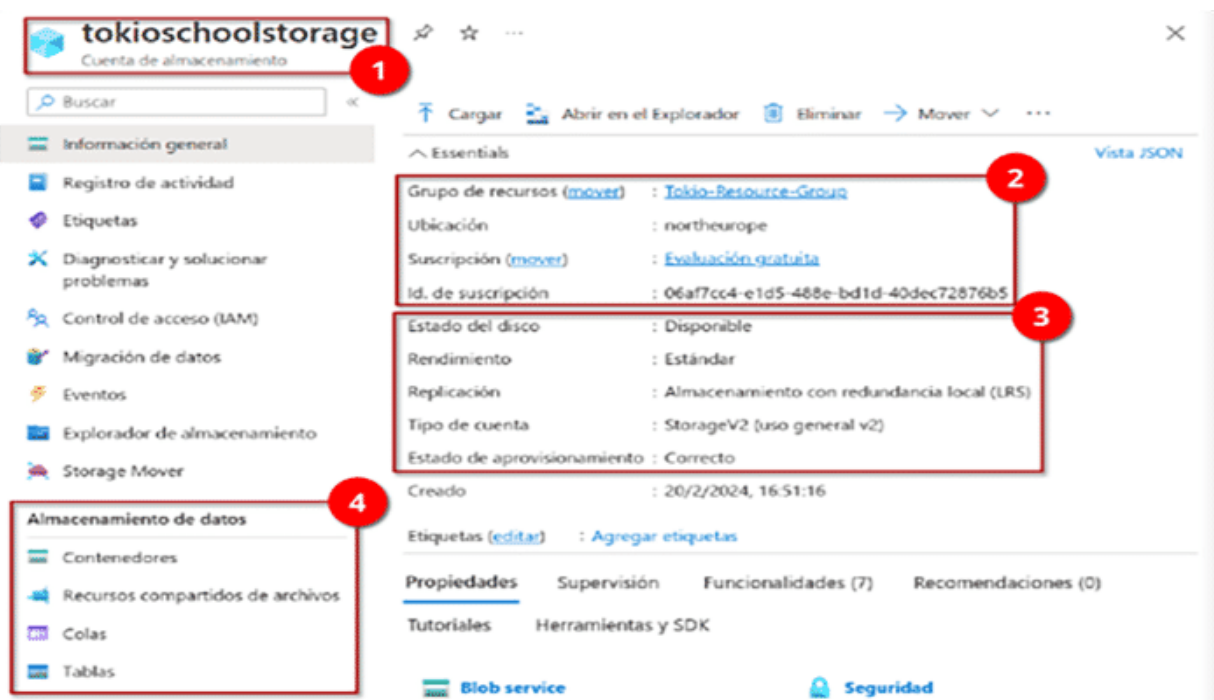
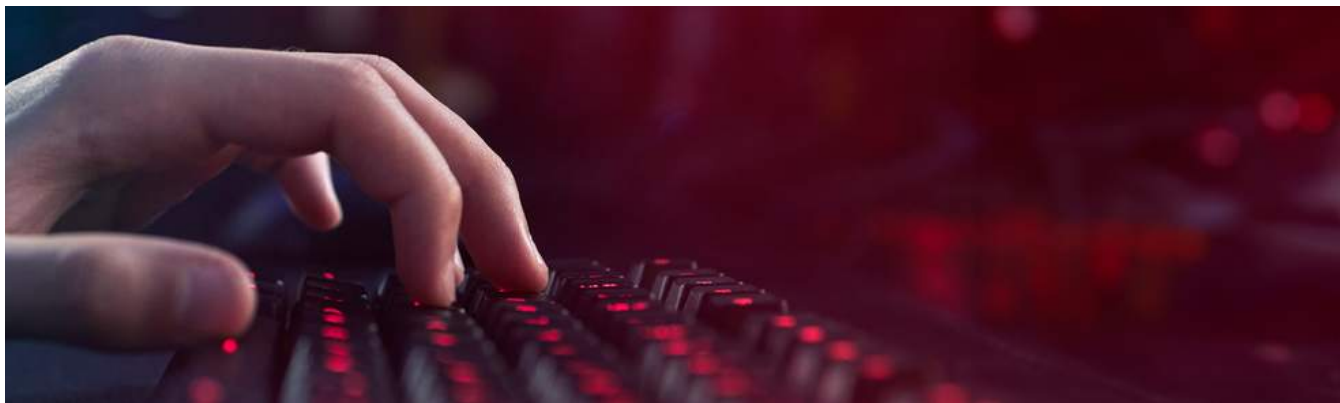


Figura 39. Vista general del recurso.

5.4 Introducción a Azure CLI

Azure Command-Line Interface (Azure CLI) es una herramienta de código abierto que permite a desarrolladores y administradores de sistemas **automatizar las tareas de administración de Azure mediante scripts y comandos.**

Proporciona una interfaz para ejecutar comandos que permiten crear, gestionar y monitorizar cualquier servicio de Azure desde la terminal, tanto en sistemas operativos Windows como en Unix.



5.4.1 Instalación de Azure CLI

Azure CLI puede instalarse tanto en Windows y macOS como en Linux. Generalmente, la instalación implica una **descarga directa del paquete adecuado a nuestro sistema operativo desde la página oficial de Microsoft o mediante gestores de paquetes** como APT en Ubuntu, Brew en macOS o Choco en Windows.

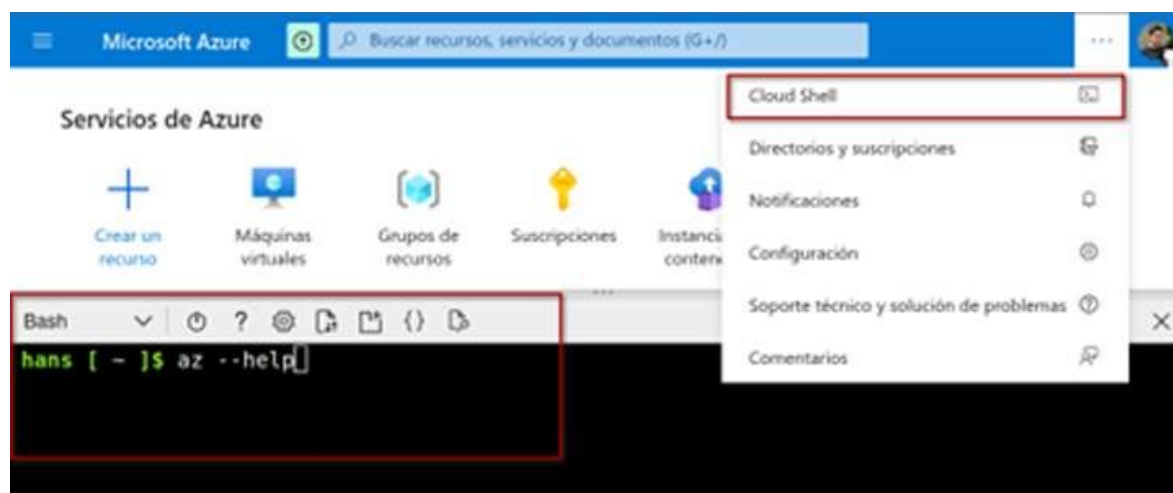
Una vez instalada, podemos abrir la terminal y ejecutar az login para iniciar sesión en nuestra cuenta de Azure, lo que abrirá una ventana de navegador donde podremos ingresar nuestras credenciales. Alternativa de instalación usando Azure Cloud Shell:

Alternativa de instalación usando Azure Cloud Shell:

Además de la interfaz de línea de comandos tradicional, Azure CLI se integra perfectamente con Azure Cloud Shell, una terminal interactiva accesible desde nuestro navegador.

Azure Cloud Shell ofrece un **entorno preconfigurado** donde Azure CLI ya se encuentra instalado, de forma que no resulta necesario llevar a cabo ninguna configuración local, permitiendo a los usuarios ejecutar comandos de Azure CLI desde cualquier lugar o dispositivo. Con solo acceder al portal de Azure y lanzar Cloud Shell, podemos empezar a gestionar nuestros recursos de Azure de inmediato y aprovechando las ventajas de un entorno de terminal completo basado en la nube.

Cloud Shell también proporciona **herramientas adicionales y soporte para lenguajes de scripting** como Bash y PowerShell, ampliando las capacidades de automatización y de gestión directas desde el navegador. De este modo, no solo facilita el acceso y la portabilidad, sino que también asegura que siempre trabajemos con la versión más reciente de Azure CLI, despreocupándonos de las actualizaciones o las compatibilidades del sistema operativo.



5.4.2 Conceptos básicos y comandos esenciales

Antes de comenzar, es necesario tener claros los siguientes conceptos básicos:

- **Comandos y argumentos.** Los comandos de Azure CLI siguen el patrón `az <grupo de comandos> <subgrupo> <acción>`, siempre encabezado por el comando `az` y donde especificamos en cada parte lo que deseamos llevar a cabo. Los argumentos proporcionan detalles adicionales, como nombres de recursos, ubicaciones y configuraciones.
- **Grupos de comandos.** Se corresponden con los servicios de Azure, como `vm` para máquinas virtuales, `storage` para cuentas de almacenamiento y `group` para grupos de recursos de Azure, entre otros muchos. Permiten organizar los comandos relacionados con la gestión de estos servicios.
- **Output.** Azure CLI soporta varios formatos de salida para los comandos ejecutados, incluyendo `json`, `yaml`, `table` y `tsv`, que pueden especificarse con el argumento `--output` u `-o`. De esta forma, si la información va a ser visualizada por un usuario, probablemente resulte más cómodo un formato "tabla"; en cambio, si está destinada a un script o a un programa, el formato "json" es más fácil de manipular y resulta más sencillo acceder a la información.

```
hans [ ~ ]$ az group list --output table
Name                                     Location    Status
-----
Tokio-VM                               northeurope Succeeded
Tokio-Resource-Group                   northeurope Succeeded
cloud-shell-storage-west europe         west europe Succeeded
```

Figura 41. Ejemplo 1: Punto 1, grupo, Punto 2, subgrupo; Punto 3, acción

```
hans [ ~ ]$ az storage account list --output json
[
  {
    "accessTier": "Hot",
    "accountMigrationInProgress": null,
    "allowBlobPublicAccess": false,
    "allowCrossTenantReplication": false,
    "allowSharedKeyAccess": null,
    "allowedCopyScope": null,
```

Figura 42. Ejemplo 2: Punto 1, grupo, Punto 2, subgrupo; Punto 3, acción.

Una vez conocidos, es necesario aprender una serie de Comandos esenciales, como, por ejemplo:

1

Iniciar sesión en Azure: az login. Este comando abre una ventana del navegador para que iniciemos sesión con nuestra cuenta de Azure. Una vez autenticados, la CLI obtiene acceso para operar con nuestros recursos de Azure.

2

Listar suscripciones: az account list --output table. Muestra todas las suscripciones disponibles en nuestra cuenta de Azure en un formato de tabla legible.

3

Establecer una suscripción activa: az account set --subscription "Nombre o ID de la suscripción". Configura la suscripción con la que trabajaremos, lo que resulta importante si contamos con acceso a múltiples suscripciones.

4

Crear un grupo de recursos: az group create --name MyResourceGroup --location eastus. Los grupos de recursos nos permiten agrupar nuestros recursos de Azure, facilitando su gestión y facturación.

5

Listar todos los recursos en un grupo de recursos: az resource list --resource-group MyResourceGroup. Proporciona una lista de todos los recursos incluidos dentro de un grupo específico.

Eliminar un grupo de recursos y todos sus componentes: `az group delete -name MyResourceGroup`. Elimina el grupo especificado junto con todos los recursos que contiene

5.4.3 Automatización con scripts

Azure CLI es especialmente potente cuando se utiliza para automatizar tareas mediante scripts. Así, por ejemplo, podemos escribir un script en Bash o en PowerShell que use los comandos de Azure CLI para automatizar el despliegue de infraestructura, configurar redes o desplegar aplicaciones en Azure, **simplificando procesos complejos y repetitivos**.

5.4.4 Ejecución de un contenedor en Azure con Azure CLI

En el siguiente ejemplo práctico, vamos a comprobar cómo utilizar Azure CLI con el propósito de ejecutar un contenedor dentro de la plataforma aprovechando el plan gratuito de Azure y Azure Container Instances (ACI), una solución para aquellas tareas que requieren contenedores que nos permite prescindir de orquestación compleja (Kubernetes, Docker Swarm, etc.).

En primer lugar, creamos un grupo de recursos en la ubicación que deseemos:

In [1]

```
az group create --name MyContainerGroup --location eastus
```

```
hans [ ~ ]$ az group create --name MyContainerGroup --location eastus -o table
Location      Name
-----
eastus        MyContainerGroup
```

Figura 43. Creación del grupo de recursos.

A continuación, vamos a desplegar un contenedor utilizando Azure Container Instances. Para este ejemplo, recurriremos a una imagen genérica de Docker Hub, como hello-world. Dicho contenedor simplemente imprime un mensaje de bienvenida y no requiere configuraciones complejas, demostrando la facilidad y la rapidez con las que se pueden ejecutar contenedores en Azure.

In [1]

```
az container create --resource-group MyContainerGroup --name myhellocontainer --image
hello-world --os-type Linux --location eastus -o table
+
```

Name	ResourceGroup	Status	Image	CPU/Memory	OsType	Location
myhellocontainer	MyContainerGroup	Running	hello-world	1.0 core/1.5 gb	Linux	eastus

Figura 44. Desplegado de contenedor.

Este comando crea y ejecuta un contenedor basado en la imagen hello-world, donde --os-type especifica el sistema operativo del contenedor y --location indica la región en la que se desplegará.

De forma adicional, podemos visualizar el contenedor en Azure Portal:

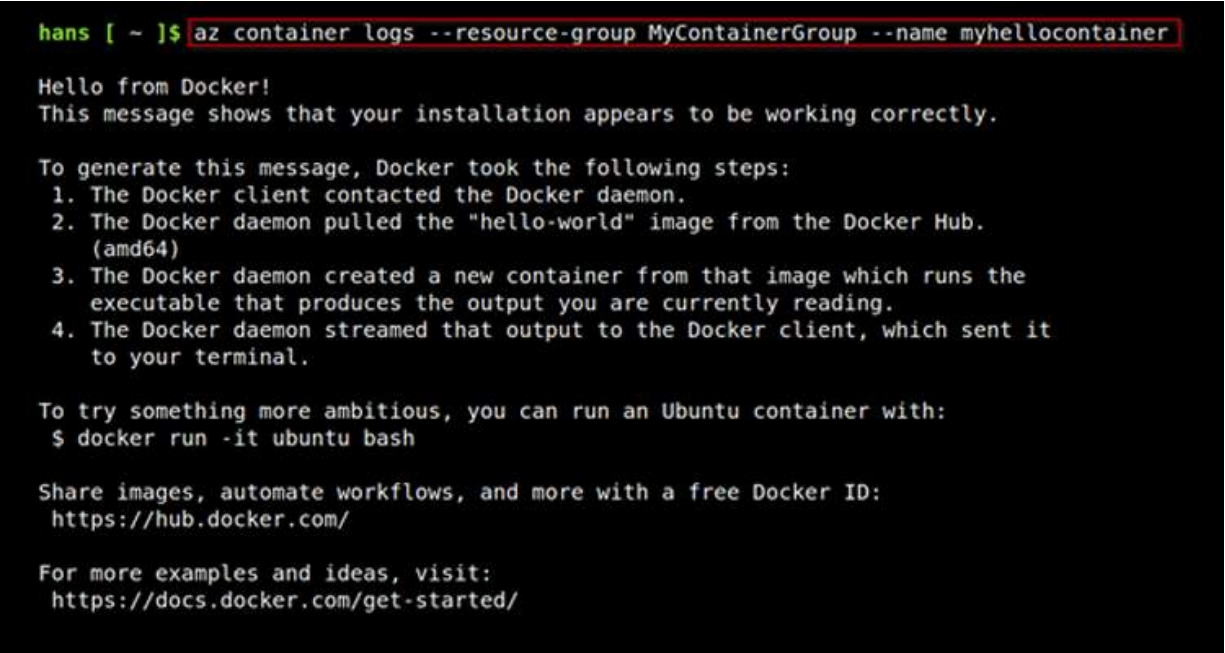


Figura 45. Visualización del contenedor en Azure Portal.

Para obtener los logs y ver el mensaje de salida del contenedor, ejecutaremos el siguiente comando:

In [1]

```
az container logs --resource-group MyContainerGroup --name myhellocontainer
```



```
hans [ ~ ]$ az container logs --resource-group MyContainerGroup --name myhellocontainer

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

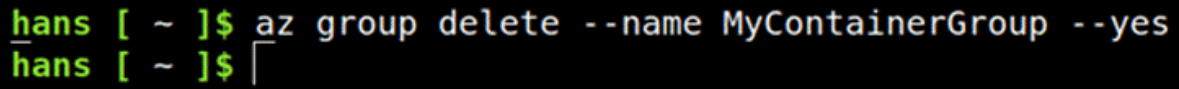
Figura 46. Obtención de visualización.

Finalmente, para evitar incurrir en gastos tras experimentar con Azure Container Instances, podemos eliminar el grupo y todos los recursos asociados a él:

In [1]

```
az group delete --name MyContainerGroup -yes
```

Este comando no devuelve respuesta en terminal.



```
hans [ ~ ]$ az group delete --name MyContainerGroup --yes
hans [ ~ ]$
```

Figura 47. Comando resultante.

Este ejemplo demuestra la eficiencia y simplicidad de Azure CLI de cara a la gestión de recursos. Con una sola línea de comando, podemos automatizar la creación de infraestructura en Azure, lo que resulta especialmente útil para scripts de automatización y para quienes prefieren trabajar desde la línea de comandos o necesitan realizar despliegues programados.

5.5 Plantillas ARM: automatización y gestión de infraestructuras en Azure

Las plantillas de Azure Resource Manager (ARM) representan una eficaz herramienta para **automatizar el despliegue y la gestión de recursos** en Azure. Dado que recurren a un enfoque de infraestructura como código, nos permiten definir de manera declarativa el entorno de nuestra infraestructura en archivos JSON.



5.5.1 Introducción a las plantillas ARM

Las plantillas ARM son **archivos JSON o BICEP** que definen uno o más recursos de Azure para desplegarlos en un grupo. Dichos archivos describen de manera declarativa los recursos, las dependencias entre estos y su configuración.

Al usar plantillas ARM, los desarrolladores y operadores pueden desplegar consistentemente infraestructuras completas por medio de un solo comando, lo que reduce los errores manuales y mejora la eficiencia operativa. Ventajas de usar plantillas ARM:

- **Automatización.** Posibilitan la creación y la configuración automáticas de recursos, permitiendo prescindir de configuraciones manuales repetitivas.
- **Consistencia.** Aseguran que cada despliegue resulte consistente, reduciendo los errores humanos y facilitando la replicación de entornos.
- **Control de versiones.** Las plantillas pueden almacenarse en sistemas de control de versiones, permitiendo el seguimiento de cambios y una colaboración efectiva entre equipos.
- **Reutilización.** Las plantillas son reutilizables y pueden parametrizarse para usarlas en diferentes escenarios o entornos.

Cuando implementamos una plantilla ARM, se desencadena un proceso complejo y sumamente optimizado que traduce nuestras definiciones declarativas en JSON o BICEP a una serie de peticiones a la API de Azure. Este proceso garantiza que los recursos se creen, configuren y relacionen tal y como se especifica en la plantilla. Desliza para saber más sobre los distintos pasos:

1

Paso 1: Interpretación de la plantilla

Al enviar una plantilla ARM para su implementación, Azure Resource Manager (ARM) la valida y, posteriormente, la procesa.

En este sentido, **verifica la sintaxis de la plantilla**, asegurándose de que incluye todos los campos requeridos y de que estos resultan válidos. Asimismo, resuelve las dependencias entre recursos para determinar el orden correcto de creación.

2

Paso 2: Conversación a peticiones de API

Una vez validada la plantilla, ARM convierte las definiciones que contiene a peticiones API individuales para cada recurso especificado.

Así, cada definición de recurso de la plantilla se traduce en **una o más llamadas a la API de Azure** que incluyen instrucciones detalladas sobre cómo configurarlo y desplegarlo.

Paso 3: Ejecución secuencial y paralela

ARM ejecuta las peticiones API siguiendo el **orden de dependencia calculado**.

Las peticiones para recursos sin dependencias pueden enviarse en paralelo, acelerando el proceso de implementación, mientras que los recursos que dependen de otros se crean secuencialmente para garantizar la integridad de la infraestructura.

Este enfoque optimiza el tiempo de implementación sin comprometer la precisión.

Paso 4: Manejo de respuestas

Para cada petición a la API, ARM recibe una **respuesta que indica el éxito o el fracaso de la creación del recurso**, monitoreándolas con el objeto de gestionar el estado de la implementación.

Si una petición falla, ARM puede revertir cambios previos o detener el proceso de implementación en función de la configuración de la plantilla y de las políticas del usuario.

Paso 5: Confirmación de la implementación



Figura 48. Proceso del uso de plantillas ARM.

Cuando todas las peticiones API se han ejecutado exitosamente y el conjunto de los recursos se hallan configurados y en funcionamiento, ARM establece la **implementación de la plantilla como completada**.

Entonces, el usuario puede verificar el estado de los recursos desplegados a través del portal de Azure, Azure CLI o Azure PowerShell.

Este proceso evidencia la importancia de las plantillas ARM como **capa de abstracción que simplifica la interacción con la API de Azure**, lo que permite a los usuarios concentrarse en la arquitectura y en el diseño de sus soluciones, despreocupándose de los detalles técnicos de las peticiones API.

5.5.2 Lenguaje Bicep en Azure

Bicep es un **lenguaje de descripción de infraestructura como código** (IaC) introducido por Microsoft para **simplificar la creación de plantillas de Azure Resource Manager** (ARM), pues ha

Bicep actúa como un **transpilador**, convirtiendo las definiciones Bicep en plantillas ARM JSON estándar. En comparación con el JSON utilizado en las plantillas ARM, presenta una sintaxis más sencilla y concisa, facilitando la definición, el despliegue y el mantenimiento de infraestructuras en Azure.

(A) Características clave de bicep

- **Sintaxis simplificada.** Bicep reduce la complejidad y la redundancia del código necesarias para definir recursos en Azure, contribuyendo a que las plantillas resulten más fáciles de escribir y mantener.
- **Transpilación a ARM JSON.** Bicep se compila en plantillas ARM JSON, lo que significa que se pueden usar todas las características y capacidades de ARM sin restricciones.
- **Soporte completo de recursos de Azure.** Como ocurre con las plantillas ARM, Bicep soporta todos los tipos y versiones de recursos disponibles en Azure.
- **Integración con herramientas de desarrollo.** Integrado con Visual Studio Code, Bicep proporciona características como resaltado de sintaxis, autocompletado y validación en tiempo real, lo que mejora significativamente la experiencia de desarrollo.

(B) Componentes de una plantilla Bicep

- 1 **Parámetros.** Los parámetros permiten la personalización de las plantillas Bicep en el momento del despliegue, representando un medio para trasladar valores externos a la plantilla. Resultan útiles para especificar configuraciones que pueden variar entre despliegues, como nombres de recursos, tamaños de máquinas virtuales o ubicaciones geográficas.

2

Variables. En Bicep, las variables se utilizan para simplificar las plantillas y reutilizar valores en múltiples lugares. En este sentido, pueden contribuir a reducir la redundancia y a que la plantilla resulte más legible.

3

Recursos. Los recursos constituyen el núcleo de las plantillas Bicep, representando los objetos de Azure que deseamos crear, como máquinas virtuales, cuentas de almacenamiento o redes virtuales. En Bicep, cada recurso se define con un tipo, un nombre y un conjunto de propiedades que especifican su configuración.

4

Salidas (outputs). Las salidas son valores que se devuelven una vez desplegada una plantilla Bicep. Resultan útiles para obtener información sobre los recursos creados, como direcciones IP, identificadores o URL de endpoints.

```
1 param location string = 'eastus'
2
3 var storageAccountName = 'biceptokiostorage'
4
5 resource storageAccount 'Microsoft.Storage/storageAccounts@2021-04-01' = {
6   name: storageAccountName
7   location: location
8   sku: {
9     name: 'Standard_LRS'
10  }
11   kind: 'StorageV2'
12 }
13
14 output storageAccountId string = storageAccount.id
15
```

Figura 49. Punto 1, parámetro; Punto 2, variable; Punto 3, recursos; Punto 4, salida.

En la imagen anterior, podemos apreciar un ejemplo de cómo se define una plantilla Bicep con el propósito de crear una cuenta de almacenamiento en Azure. A tal efecto, muestra cómo se utilizan los parámetros, los recursos y las salidas.

Al desplegar esta plantilla, se crea una cuenta de almacenamiento con las especificaciones proporcionadas y se devuelve el identificador de la cuenta de almacenamiento como salida.

(C) Integración de lógica en plantillas Bicep

Bicep permite a los usuarios incorporar lógica directamente en sus plantillas recurriendo a **expresiones integradas** que pueden realizar operaciones sobre cadenas, generar valores únicos o manipular listas, entre otros aspectos. Al utilizar estas capacidades, podemos crear plantillas dinámicas y flexibles sin depender de **herramientas externas como scripts de generación de plantillas**.

(D) Ejemplo práctico: generación de nombres únicos

En el siguiente ejemplo, podemos apreciar cómo se utilizan expresiones integradas en Bicep con el objeto de generar un nombre único para una cuenta de almacenamiento en Azure.

En este sentido, se toma en consideración el ID del grupo de recursos con el propósito de garantizar la unicidad y se modifica el resultado para cumplir con las restricciones de nombres de Azure.

```
1 param location string = 'eastus'
2
3 // Genera un sufijo basado en el ID del grupo de recursos con 4 caracteres en minúsculas
4 var uniqueSuffix = toLower(take(uniqueString(resourceGroup().id), 4))
5 var storageAccountName = 'tokio${uniqueSuffix}'
6
7 resource storageAccount 'Microsoft.Storage/storageAccounts@2021-04-01' = {
8   name: storageAccountName
```

Figura 50. Declaración de variables.

VARIABLE UNiquesuffix.

VARIABLE STORAGEACCOUNTNAME.

Utiliza una combinación de las funciones `toLowerCase()`, `take()` y `uniqueString()` para generar un sufijo único de cuatro caracteres en minúsculas basado en el ID del grupo de recursos, lo que garantiza que el nombre de la cuenta de almacenamiento sea único y cumpla con las restricciones de Azure, que requiere que los nombres se encuentren en minúsculas.

VARIABLE UNIQUESUFFIX.

VARIABLE STORAGEACCOUNT NAME.

Concatena un texto estático (`tokio`) con el sufijo único para generar el nombre completo de la cuenta de almacenamiento. Esta técnica garantiza que cada despliegue presente un nombre distintivo, evitando conflictos con otros.

En nuestro ejemplo, se crearía el siguiente grupo de recursos:



Figura 51. Vista del grupo de recursos.

El uso de expresiones integradas en Bicep para incluir lógica directamente en las plantillas proporciona una forma eficaz y eficiente de automatizar y personalizar la creación de infraestructuras en Azure.

El ejemplo anterior ilustra cómo las capacidades de Bicep pueden utilizarse para simplificar significativamente el desarrollo y despliegue de recursos, al tiempo que se conservan las plantillas claras, mantenibles y en conformidad con las políticas de Azure.

5.5.3 Despliegue de plantillas BICEP en Azure

Una vez creada nuestra plantilla BICEP, el siguiente paso consiste en implementarla en Azure para materializar nuestra infraestructura, un proceso directo y eficiente gracias a herramientas como Azure CLI y Azure Cloud Shell.

Azure CLI ofrece soporte completo para trabajar directamente con **plantillas BICEP**, eliminando la necesidad de compilar manualmente a JSON. Debemos asegurarnos de que nos encontramos en el directorio donde se ubica nuestro archivo BICEP. La extensión de BICEP se instalará automáticamente cuando ejecutemos un comando relacionado.

Para desplegar la plantilla BICEP, utilizaremos el siguiente comando de Azure CLI asegurándonos de reemplazar MyResourceGroup por el nombre de nuestro grupo de recursos y path/to/your/template.bicep por la ruta a nuestro archivo BICEP.

In [1]

```
az deployment group create --resource-group MyResourceGroup --template-file
```

Out[1]



```
hans [ ~ ]$ az deployment group create --resource-group Tokio-Resource-Group --template-file TokioStorage.bicep -o table
Name                State      Timestamp                               Mode      ResourceGroup
-----
TokioStorage_       Succeeded  2024-02-25T20:27:45.086346+00:00      Incremental  Tokio-Resource-Group
```

Figura 52. Cómo desplegar la plantilla BICEP.

Este comando iniciará el despliegue de los recursos definidos en nuestra plantilla dentro del grupo de recursos especificado y nos devolverá el resultado en pantalla.

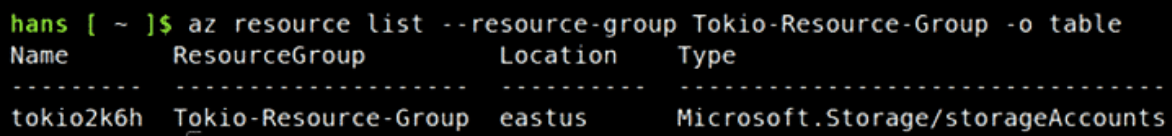
(A) Verificar con Azure CLI

Para verificar que el despliegue se ha completado correctamente, podemos listar los recursos en nuestro grupo mediante el siguiente comando:

In [1]

```
az resource list --resource-group MyResourceGroup -o table
```

Out[1]



```
hans [ ~ ]$ az resource list --resource-group Tokio-Resource-Group -o table
Name      ResourceGroup      Location      Type
-----
tokio2k6h  Tokio-Resource-Group  eastus        Microsoft.Storage/storageAccounts
```

Figura 53. Verificación de Azure Cli.

De este modo, se muestra una tabla con todos los recursos que contiene el grupo especificado, lo que nos permite revisar rápidamente los resultados del despliegue.

(B) Verificar en el portal de azure

De cara a una visualización más gráfica, podemos verificar los recursos creados directamente en el portal de Azure.

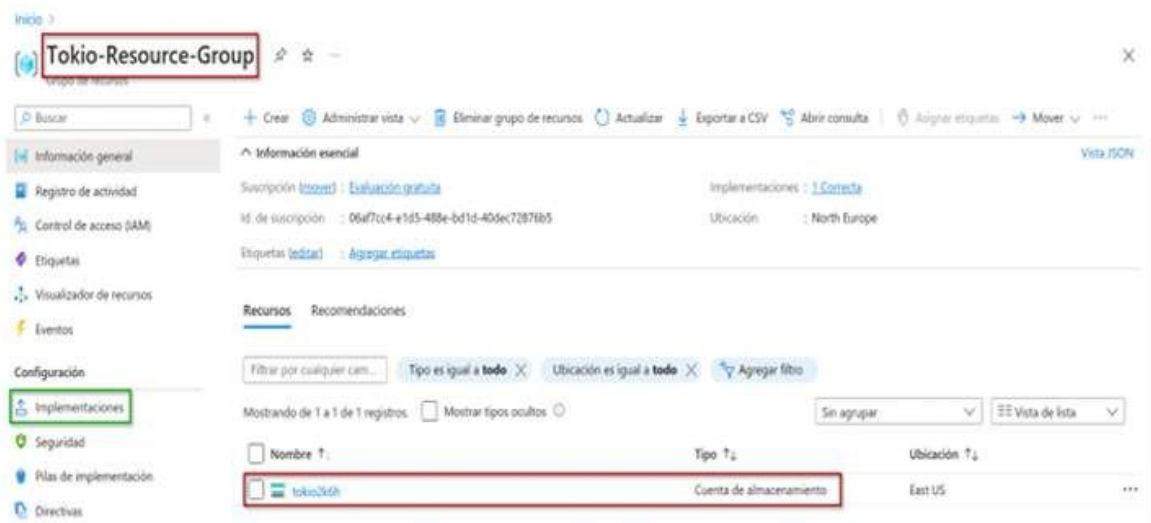


Figura 54. Visualización en Portal Azure.

Así, además de visualizar aquellos recursos que se han creado en el grupo, podemos consultar cualquier configuración, su estado y los detalles relevantes. Además, navegando a la opción Implementaciones del menú izquierdo (marcado en verde en la imagen) podemos ver el detalle de todas las plantillas desplegadas.



Figura 55. Vista detalla de una implementación.

Asimismo, al clicar en una implementación, podemos consultar todos sus detalles, como la plantilla, las salidas (output), descargarla, realizar modificaciones o incluso desplegarla de nuevo desde el propio portal de Azure.

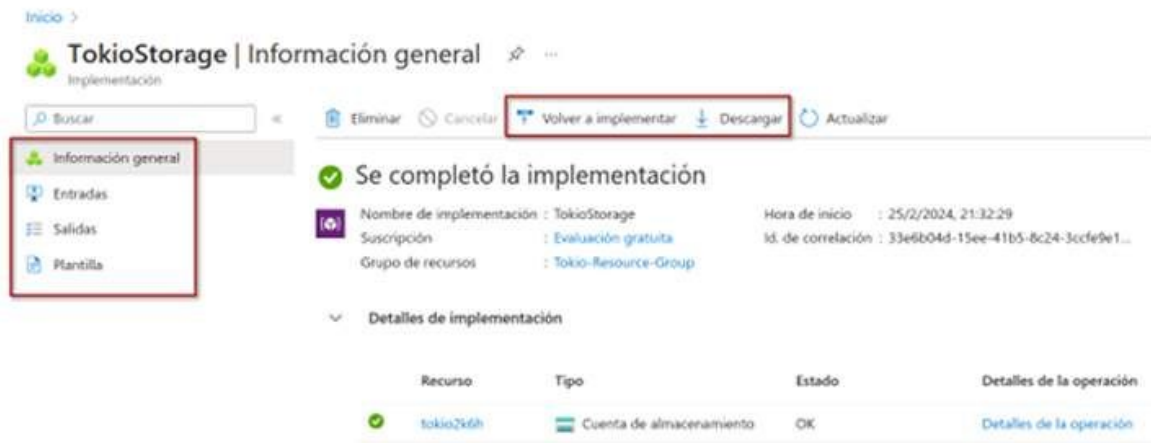


Figura 56. Vista de información general tras la implementación.

Desplegar plantillas BICEP directamente en Azure simplifica notablemente el proceso de materialización de infraestructuras complejas en la nube. Mediante Azure CLI y el portal de Azure, desarrolladores y operadores pueden implementar, verificar y gestionar recursos de manera eficaz y eficiente.

Tema 6: Azure DevOps Services



-  Introducción
-  6.1 Introducción a la plataforma de Azure DevOps
-  6.2 Azure Boards: gestión de proyectos ágiles
-  6.3 Azure Repos: gestión avanzada del código fuente
-  6.4 Introducción a Azure Pipelines
-  6.5 Azure Artifacts
-  6.6 Introducción a Azure Test Plans

Introducción

En este tema nos adentraremos en la plataforma Azure DevOps, explorando sus servicios, beneficios y componentes clave, y cómo estos pueden ser aplicados para transformar el ciclo de vida del desarrollo de software en su organización.

Se trata de un tema importante dada la creciente necesidad de las organizaciones de adaptarse rápidamente a los cambios del mercado, manteniendo al mismo tiempo altos estándares de calidad en sus productos de software.

Azure DevOps no solo ofrece **herramientas para automatizar y optimizar procesos como la integración y entrega continuas (CI/CD)**, sino que también proporciona capacidades para la gestión de proyectos, control de versiones, gestión de paquetes y pruebas automatizadas, entre otros. Estas herramientas, cuando se utilizan de manera efectiva, pueden acelerar el tiempo de entrega, mejorar la calidad del software y fomentar una cultura de colaboración y mejora continua.

A lo largo de este tema, se presentarán los componentes principales de **Azure DevOps Services, incluyendo Azure Boards, Azure Repos, Azure Pipelines, Azure Artifacts y Azure Test Plans**. Se explorará cómo cada uno de estos servicios desempeña un papel crucial en el ciclo de vida del desarrollo de software, desde la planificación y el seguimiento de proyectos hasta la integración, entrega y pruebas de software.

Además, se analizarán casos prácticos y estrategias para implementar eficazmente Azure DevOps en proyectos de software, proporcionando así una comprensión profunda de cómo esta plataforma puede ser utilizada para enfrentar los desafíos actuales en el desarrollo de software y posicionar a las organizaciones a la vanguardia de la innovación tecnológica.

Al finalizar este tema, los estudiantes estarán equipados con las herramientas y conocimientos necesarios para aplicar prácticas DevOps en sus organizaciones, mejorando así la colaboración entre equipos, la eficiencia en los procesos y la calidad del producto final. Este tema promete ser una exploración enriquecedora y transformadora, preparando a los estudiantes para los desafíos y oportunidades del desarrollo de software moderno.

6.1 Introducción a la plataforma de Azure DevOps

En el ámbito del desarrollo de software, la capacidad de adaptarse rápidamente a los cambios y de entregar productos de alta calidad en tiempos reducidos se ha convertido en una necesidad. Azure DevOps Services, anteriormente conocido como **Visual Studio Team Services (VSTS)**, ofrece un conjunto de herramientas de desarrollo basadas en la nube que permiten a los equipos implementar prácticas de DevOps de manera eficiente y efectiva.



Azure DevOps se centra en mejorar la colaboración entre los equipos de desarrollo y operaciones con el propósito de automatizar el proceso de entrega de software y contribuir a una integración y entrega continuas (CI/CD). Al proporcionar servicios integrados para el seguimiento de proyectos, control de versiones, CI/CD, gestión de paquetes y pruebas automatizadas, facilitando a las organizaciones la gestión de todo el ciclo de vida del desarrollo de software.

6.1.1 Beneficios de Azure DevOps

Esta herramienta, al tratarse de una solución integral, ofrece una serie de beneficios significativos

- Colaboración mejorada: Herramientas como Azure Boards contribuyen a que la planificación, el seguimiento y la discusión de proyectos resulten más accesibles para todos los miembros del equipo independientemente de su ubicación geográfica.
- Automatización de procesos: Azure Pipelines permite la automatización de compilaciones, pruebas y despliegues, reduciendo el riesgo de errores humanos y acelerando los tiempos de entrega.
- Flexibilidad: La capacidad de integrarse con una amplia gama de herramientas y servicios, tanto de Microsoft como de terceros, convierte a Azure DevOps en una solución sumamente flexible.
- Gestión eficiente del código fuente: Con Azure Repos, los equipos disponen de acceso a repositorios Git ilimitados y privados que cuentan con políticas de seguridad y búsquedas de código, favoreciendo una gestión del código fuente segura y eficiente.
- Entrega continua: La integración y entrega continuas se simplifican, contribuyendo a que los equipos desplieguen software de alta calidad de forma más rápida y frecuente.
- Visibilidad y control: La amplia gama de herramientas orientadas a informes y análisis proporciona a los equipos una visión clara del estado y del progreso de sus proyectos, ayudándoles a adoptar decisiones informadas rápidamente.
- Manejo eficiente de dependencias: A la hora de compartir y gestionar paquetes de código dentro de equipos o con la comunidad, Azure Artifacts se integra perfectamente con Azure Pipelines de cara a una eficaz gestión y distribución de artefactos.
- Calidad y control: Azure Test Plans permite la creación y ejecución de pruebas manuales y automatizadas para garantizar la calidad del software, integrándose perfectamente en el proceso de CI/CD con el objeto de detectar y corregir problemas desde etapas tempranas del desarrollo.

Todo esto hace que Azure DevOps pueda configurarse como una plataforma integral y robusta que encapsula todas las herramientas necesarias para adoptar y ejecutar prácticas DevOps con éxito.

6.1.2 Componentes de Azure DevOps Services

Cada uno de los principales servicios de Azure DevOps desempeña un papel crucial en el ciclo de vida del desarrollo de software, favoreciendo la implementación de prácticas DevOps.



Figura 57. Logo de Azure DevOps

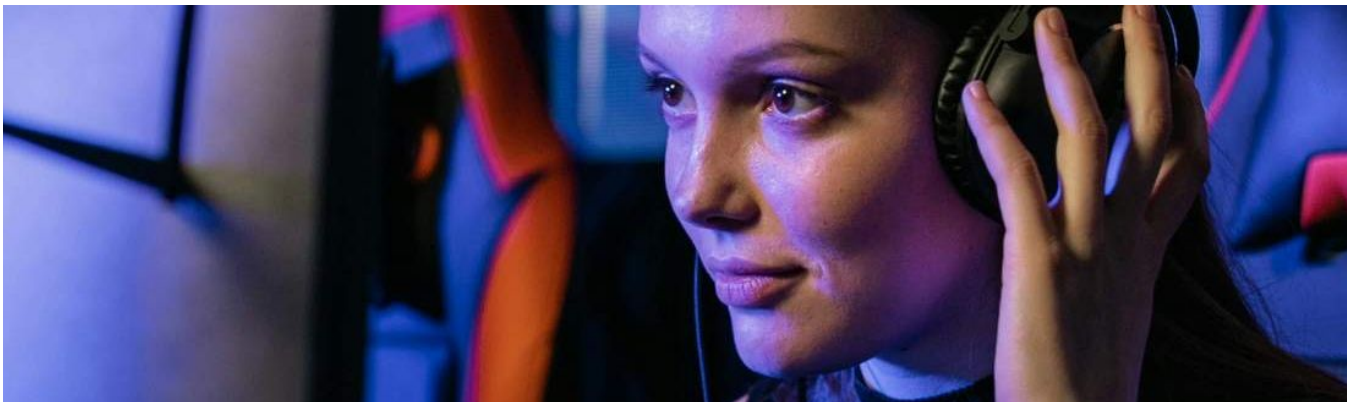
- Azure Boards: Azure Boards es una herramienta de seguimiento de proyectos que proporciona capacidades de planificación, seguimiento y discusión del trabajo entre equipos.
- Azure Repos: Azure Repos ofrece un servicio de repositorios de código fuente gestionado, soportando tanto Git como Team Foundation Version Control (TFVC).
- Azure Pipelines: Azure Pipelines es un servicio de integración y entrega continuas (CI/CD) que permite a los equipos automatizar la compilación, la prueba y el despliegue de sus aplicaciones en cualquier nube o de forma local.

- Azure Artifacts: Azure Artifacts permite a los equipos crear, alojar, compartir y consumir paquetes de software dentro de sus organizaciones y con la comunidad de desarrolladores en general.
- Azure Test Plans: Azure Test Plans ofrece una solución completa para la prueba manual y automatizada de aplicaciones.

En los próximos apartados, abordaremos con más detalle cada uno de estos componentes clave.

6.2 Azure Boards: gestión de proyectos ágiles

Azure Boards es una herramienta potente y versátil integrada en Azure DevOps y **especializada en la gestión ágil de proyectos**. En este sentido, suministra a los equipos las capacidades necesarias para **planificar, rastrear y discutir** el trabajo a lo largo del ciclo de vida del desarrollo de software. A continuación, nos aproximaremos a sus principales características y analizaremos cómo contribuye a la gestión ágil de proyectos.



6.2.1 Principales características

A continuación, se enumeran las principales características de Azure Boards:

(A) Trabajo con sprints, backlogs y tableros kanban

- Sprints: Azure Boards permite a los equipos organizar su trabajo en sprints, facilitando la planificación iterativa y el seguimiento del progreso de cara a la consecución de los objetivos a corto plazo. Así, los equipos

pueden asignar tareas y bugs a sprints específicos, estableciendo compromisos claros y fechas límite realistas para cada iteración.

- Backlogs: Se trata de listas de trabajo pendiente donde los equipos pueden gestionar y priorizar las características, tareas y bugs. Los backlogs de Azure Boards ofrecen una visión clara de todo el trabajo requerido, contribuyendo a que los equipos se centren en aquellos aspectos que revisten más importancia y planifiquen eficientemente las próximas iteraciones.
- Tableros Kandan: Los tableros kanban ofrecen una visualización en tiempo real del flujo de trabajo, permitiendo que los equipos comprueben el estado de cada tarea a lo largo de las diferentes etapas del proceso de desarrollo. Los tableros son completamente personalizables, soportando columnas, límites de las tareas en curso (WIP) y políticas de columnas para optimizar el flujo de trabajo y mejorar la eficiencia.

(B) Personalización y flexibilidad

- Campos personalizables y formularios: Los equipos pueden personalizar los elementos de trabajo para que se ajusten en mayor medida a sus necesidades específicas, añadiendo campos particularizados, modificando formularios y adaptando las vistas. Esta flexibilidad asegura que Azure Boards pueda adecuarse a una amplia variedad de metodologías de trabajo y a los requisitos específicos del proyecto.
- Reglas de negocio y automatización: Azure Boards se orienta a la creación de reglas de negocio que automatizan ciertos comportamientos relativos a los elementos de trabajo, por ejemplo, modificar automáticamente el estado de estos cuando se cumplen determinadas condiciones. De este modo, contribuye a optimizar los flujos de trabajo.

6.2.2 Facilitación de la gestión ágil de proyectos: Integración con GitHub y Azure Repos

Una de las ventajas más significativas de Azure Boards **en comparación con otras herramientas** de gestión de proyectos reside en su capacidad de **integrarse con herramientas de control de versiones** como GitHub y Azure Repos, lo que permite a los equipos vincular directamente *commits*, solicitudes de extracción (*pull requests*) y ramas a elementos de trabajo en Azure Boards, proporcionando un contexto completo de desarrollo y favoreciendo la trazabilidad desde los requisitos hasta el código implementado.

- Configuración de la integración: Para vincular Azure Boards con GitHub o Azure Repos, debe definirse previamente la conexión en Azure DevOps, lo que implica autorizar el acceso de este a los repositorios específicos de GitHub o de Azure Repos, así como configurar los proyectos de Azure Boards que recibirán los enlaces de los elementos de trabajo.
- Uso de identificadores en commits y pull requests: Los desarrolladores han de incluir el identificador del elemento de trabajo (tareas, sprints, etc.) de Azure Boards en sus mensajes de commit o en las descripciones de pull requests por medio de una sintaxis específica, como AB#123, donde 123 representa el número del elemento de trabajo. Este procedimiento se puede llevar a cabo tanto en GitHub como en Azure Repos.
- Automatización y reglas personalizadas: A través de la API de Azure DevOps, los equipos pueden crear reglas personalizadas o automatizaciones más complejas. Así, por ejemplo, podrían desarrollar scripts que vinculen automáticamente todos los commits de una rama con un sprint o que actualicen tareas en Azure Boards en función de eventos específicos como pull requests en GitHub o Azure Repos.
- Visualización y seguimiento: Una vez establecida la integración, los elementos de trabajo mostrarán en Azure Boards enlaces directos a los commits, pull requests y ramas relevantes en GitHub o Azure Repos. De esta forma, los equipos pueden visualizar rápidamente el contexto del código asociado a tareas, bugs o características, lo que mejora la capacidad de seguimiento y evaluación del progreso del proyecto.

Esta integración refuerza la metodología ágil y las prácticas DevOps al proporcionar una **conexión fluida entre la planificación del proyecto y el desarrollo del código**. En este sentido, permite a los equipos mantener una visión integral del progreso del proyecto, asegurando que las actualizaciones de código se encuentren alineadas con sus objetivos y requisitos, además de facilitar la gestión eficaz de los ciclos de vida del desarrollo de software.

Además, este servicio no solo se integra con GitHub y Azure Repos, sino también con una amplia gama de herramientas y servicios como Jira, Slack y Microsoft Teams, permitiendo a los equipos utilizar Azure Boards junto con otras herramientas ya en uso, manteniendo la coherencia y eficiencia en los flujos de trabajo. Además, el servicio ofrece control de acceso basado en roles, permitiendo a los administradores configurar detalladamente el acceso de los usuarios a diferentes áreas y funciones, garantizando que los miembros del equipo solo accedan a la información y funcionalidades relevantes para sus roles.

6.3 Azure Repos: gestión avanzada del código fuente

Azure Repos proporciona un servicio de control de versiones basado en la nube, sumamente seguro y escalable, que permite a los equipos de desarrollo colaborar en el código de manera eficiente.

```
11         self.x = x
12         self.y = y
13     }
14
15     /// Returns the surrounding points in range of
16     /// the current one
17     func points(inRange range: Int = 1) -> [Point] {
18         var results = [Point]()
19
20
21         let lowerBoundOfXRange = x - range
22         let upperBoundOfXRange = x + range
23
24         let lowerBoundOfYRange = y - range
25         let upperBoundOfYRange = y + range
```

(A) Componentes

Como componente de Azure DevOps, soporta tanto Git como Team Foundation Version Control.

- Soporte Git completo: Azure Repos se encuentra completamente integrado con Git, el sistema de control de versiones distribuido más extendido, de manera que los equipos pueden utilizar todas sus funciones estándar, así como características adicionales que mejoran la colaboración y la seguridad.
- Repositorios ilimitados: Los equipos pueden crear un número ilimitado de repositorios privados, lo que contribuye a organizar el código de manera

efectiva y segura y permite despreocuparse de las restricciones de espacio o de costes adicionales. Resultan útiles para gestionar el código de las aplicaciones basadas en microservicios, pues cada uno de ellos puede desarrollarse y gestionarse en su propio repositorio.

- Revisión de código: Azure Repos favorece un proceso de revisión de código robusto mediante pull requests, comentarios en línea y políticas de rama para asegurar su calidad y controlar el flujo de cambios hacia las ramas principales.
- Integración con Azure Pipelines: La integración directa con Azure Pipelines permite implementar flujos de trabajo de CI/CD directamente desde los repositorios en Azure Repos, automatizando la construcción, prueba y despliegue de aplicaciones.
- Seguridad y políticas de rama: Azure Repos también permite configurar políticas de rama para solicitar revisiones de pull request o comprobaciones de construcción, entre otros aspectos, antes de que los cambios puedan fusionarse. De esta manera, contribuye a mantener la calidad y seguridad del código. Así, por ejemplo, se pueden bloquear ramas específicas después de una release (lanzamiento de versión) y permitir únicamente los cambios supervisados por un administrador.
- Herramientas de búsqueda de código: Las capacidades de búsqueda de código avanzadas permiten a los desarrolladores localizar rápidamente aquello que necesitan dentro de grandes bases de código, mejorando la eficiencia y reduciendo los tiempos a la hora de comprender y modificar el código existente.

(B) Integraciones y extensiones

- Integración con herramientas externas: Azure Repos se integra con una amplia gama de herramientas de desarrollo, incluyendo IDE populares y herramientas de CI/CD, lo que posibilita que los desarrolladores trabajen en entornos familiares sin necesidad de sacrificar la eficiencia.

- Soporte para Git LFS (Large File Storage): Azure Repos soporta Git LFS, lo que permite a los equipos manejar grandes archivos binarios sin afectar al rendimiento del repositorio. Configurar y usar Git LFS en Azure Repos puede resultar especialmente relevante para aquellos equipos que trabajan con activos digitales de gran tamaño, como videojuegos o aplicaciones donde confluyen numerosos medios.
- Extensiones: El marketplace de Azure DevOps ofrece numerosas extensiones que pueden añadirse a Azure Repos para ampliar sus funcionalidades, como integraciones con herramientas de análisis de código o gestión de proyectos, entre otras.

(C) Workflows de git en azure repos

Los *workflows* de Git son **métodos utilizados por los equipos de desarrollo para colaborar en proyectos de software** y están destinados a gestionar cómo se comparte, revisa y despliega el código. La elección de un *workflow* adecuado resulta crucial en la cultura DevOps, ya que facilita la integración y entrega continuas (CI/CD), mejora la colaboración entre los equipos de desarrollo y los de operaciones y contribuye a automatizar el proceso de despliegue de software.

Dado que se trata de una solución de control de versiones basada en Git y compatible con herramientas de internet y externas de gestión y CI/CD, Azure Repos soporta una *numerosos workflows*, de manera que los equipos pueden seleccionar el que se adapte en mayor medida a sus necesidades y aporte mayor flexibilidad para ajustarlo a las circunstancias del proyecto.

6.4 Introducción a Azure Pipelines

Azure Pipelines es una componente de servicio de Azure DevOps que permite a los equipos de desarrollo **automatizar el proceso de integración continua (CI) y entrega continua (CD)**. Se trata de una herramienta que proporciona una plataforma robusta y flexible y que ha sido diseñada para compilar, probar y desplegar software en cualquier lenguaje y a cualquier plataforma o nube. El propósito principal de Azure Pipelines consiste en **facilitar el flujo de trabajo de desarrollo de software**, contribuyendo a que los equipos implementen actualizaciones de manera más rápida, eficiente y segura.



Este componente se integra perfectamente con el resto de los servicios de Azure DevOps, ofreciendo una experiencia de usuario cohesiva y eficiente. Así, por ejemplo, los despliegues pueden rastrearse a través de Azure Boards, mientras que el código fuente se gestiona en Azure Repos y los artefactos de compilación se almacenan y comparten por medio de Azure Artifacts.

- Automatización completa: Azure Pipelines automatiza todas las fases del ciclo de vida del software, desde la compilación hasta el despliegue, reduciendo significativamente el potencial de errores humanos y acelerando el tiempo de lanzamiento al mercado.

- Plataforma agnóstica: Soporta una amplia variedad de lenguajes de programación, plataformas y nubes, entre otras, Windows, Linux, macOS, Azure, AWS o Google Cloud Platform, lo que concede una flexibilidad sin precedentes a la hora de desplegar aplicaciones en cualquier entorno.
- Integración con una amplia gama de herramientas: Azure Pipelines se integra con numerosas herramientas y servicios externos, como GitHub, contenedores Docker o Kubernetes, lo que permite a los equipos usar las herramientas que prefieran sin comprometer la eficiencia.
- Escalabilidad y seguridad: Posee capacidad para manejar proyectos de cualquier tamaño, desde startups hasta grandes empresas, contando con políticas de seguridad y control de acceso para proteger el código y los datos.
- Mejora continua: Mediante capacidades avanzadas de monitoreo y análisis, los equipos pueden medir el rendimiento de sus pipelines, identificar cuellos de botella y realizar ajustes continuos para mejorar la calidad y eficiencia del proceso de entrega.

6.4.1 Conceptos básicos de Azure Pipelines

Para comprender el funcionamiento de Azure Pipelines, resulta necesario aclarar previamente dos conceptos básicos:

- Integración continua (CI): La integración continua es una práctica de desarrollo de software que requiere que los miembros del equipo integren su trabajo con frecuencia. Cada integración se verifica automáticamente mediante la construcción (compilación) y las pruebas del proyecto, lo que permite detectar y solucionar errores rápidamente, mejorar la calidad del software y reducir el tiempo necesario para validar y lanzar nuevas actualizaciones. Azure Pipelines automatiza este proceso al compilar el código y ejecutar pruebas cada vez que se realiza un commit en el repositorio, proporcionando feedback instantáneo sobre el estado de la última integración.

- Entrega continua (CD): La entrega continua supone una extensión de la integración continua, dirigida a asegurar que se pueda lanzar una calidad renovada en cualquier momento. En el contexto de Azure Pipelines, la CD automatiza la entrega del software en varios entornos, desde el desarrollo hasta la producción, mediante el uso de pipelines. En este sentido, abarca la automatización de todas las fases de lanzamiento, desde las pruebas hasta la entrega real, y permite a los equipos realizar despliegues frecuentes con el mínimo esfuerzo manual. La entrega continua se centra en que el proceso de despliegue resulte lo más eficiente y fiable posible, asegurando que la aplicación permanezca siempre en un estado desplegable a lo largo de su ciclo de vida.

Escenario de ejemplo

Consideremos un proyecto de software donde el equipo está desarrollando una aplicación web. Su objetivo consiste en **automatizar el proceso de integración y entrega** para asegurar que las nuevas características, correcciones de errores y actualizaciones se validen y desplieguen rápidamente y de manera confiable.

- Configuración del pipeline de CI/CD: Creamos un pipeline en Azure Pipelines que se activa con cada push en cualquier rama del repositorio y que ha sido diseñado para manejar tanto la integración continua (CI) como la entrega continua (CD) en un flujo de trabajo unificado.
- CI - Compilación y pruebas: Justo a continuación de un push, el pipeline ejecuta tareas de compilación para construir la aplicación web y, posteriormente, ejecuta pruebas unitarias y de integración. De esta manera, se asegura de que el código nuevo o modificado no introduzca errores y preserve la calidad del software.
- CD - Despliegue automático: Si el paso de CI se completa con éxito, el pipeline inicia automáticamente la fase de CD, donde se despliega en primer lugar la versión actualizada de la aplicación en un entorno de pruebas. Asimismo, se pueden realizar pruebas adicionales, como las de aceptación del usuario.

- Aprobación y despliegue en producción: Tras la validación en el entorno de pruebas, se solicita una aprobación manual de cara al despliegue en producción. Una vez aprobado, el pipeline despliega la aplicación en el entorno de producción, donde permanece disponible para los usuarios finales.

Este enfoque CI/CD asegura que el software se construya, pruebe y despliegue de forma eficiente, manteniendo elevados estándares de calidad y permitiendo entregas rápidas y seguras.

RECUERDA

Para la configuración del pipeline de CI/CD creamos un pipeline en Azure Pipelines que se activa con cada push en cualquier rama del repositorio y que ha sido diseñado para manejar tanto la integración continua (CI) como la entrega continua (CD) en un flujo de trabajo unificado.

6.4.2 Agentes en Azure Pipelines

Los agentes son **servidores físicos o virtuales** que incorporan el software de agente de Azure Pipelines y permiten la ejecución de tareas de *build* y *release*, como compilación, pruebas y

despliegues. Cada ejecución de pipeline requiere de un agente que lleve a cabo las tareas definidas en el pipeline.

Microsoft-hosted agents.	Self-hosted (autohospedados) agents.
Azure DevOps proporciona agentes hospedados por Microsoft y preconfigurados con varias imágenes de sistemas operativos (Windows, Linux, macOS) que cuentan con múltiples herramientas preinstaladas. Estos agentes son mantenidos, actualizados y administrados por Microsoft y resultan idóneos para proyectos que requieren una configuración rápida sin necesidad de gestionar infraestructura. Además, disponen de una opción gratuita para proyectos con pocas necesidades.	Los equipos pueden definir sus propios agentes en servidores dentro de su infraestructura, lo que permite un control más riguroso de la configuración del entorno, versiones de software/herramientas y acceso a recursos internos de la red. Resultan preferibles cuando se necesitan configuraciones específicas o recurrir a recursos internos a los que no se puede acceder desde el cloud.

Microsoft-hosted	Self-hosted.
Idóneo para proyectos que demandan flexibilidad y que no requieren configuraciones de entorno específicas, así como para equipos que desean ponerse en marcha inmediatamente sin necesidad de invertir en hardware.	Se trata de la opción perfecta cuando se requiere una configuración específica del entorno y acceso a recursos internos o bien se desea optimizar cargos a largo plazo manteniendo los agentes en servidores propios.

A medida que crece el equipo y el código de un proyecto, se pueden requerir más agentes para cubrir las necesidades, en cuyo caso resulta posible usar agentes Microsoft-hosted o self-hosted de forma simultánea o incluso opciones más avanzadas como un conjunto de máquinas virtuales que se escalen automáticamente para cubrir una demanda elevada de forma flexible.

Los **entornos** representan una **colección de recursos donde se despliegan las aplicaciones y servicios**. Pueden incluir servidores de desarrollo, de pruebas, de staging y de producción y se utilizan para definir el despliegue de pipelines. Ofrecen características como aprobaciones y verificaciones de seguridad, proporcionando un control detallado acerca de cómo y cuándo se despliega en estos entornos.

(C) Ejemplos habituales de configuración de entornos

- Entorno de desarrollo: Configurado para despliegues dinámicos y frecuentes, posiblemente automatizando la integración continua sin necesidad de aprobaciones para que los desarrolladores puedan probar cambios rápidamente.
- Entorno de pruebas/QA: Podría configurarse con políticas de aprobación para garantizar que solo se desplieguen aquellas versiones que han superado ciertas pruebas automatizadas, de forma que los equipos de QA puedan realizar comprobaciones más exhaustivas.
- Entorno de staging/preproducción: A menudo, se trata de un entorno lo más cercano posible al de producción que se destina a realizar las pruebas finales previas al despliegue en este. Puede requerir configuraciones de seguridad más estrictas, así como aprobaciones manuales de los stakeholders del proyecto.
- Entorno de producción: Entorno final donde el software permanece disponible para los usuarios finales y donde los despliegues suelen controlarse cuidadosamente, con aprobaciones manuales de alta jerarquía y verificaciones automáticas para asegurar que todo funcione según lo previsto antes de que el despliegue se considere exitoso.

La utilización de entornos en Azure Pipelines transforma la manera en que se gestionan los despliegues de software, ofreciendo una **segmentación clara del proceso** desde el desarrollo hasta la producción, mejorando la seguridad y el control mediante **políticas de aprobación**, potenciando la **automatización** de cara a una mayor eficiencia y rapidez en la entrega,

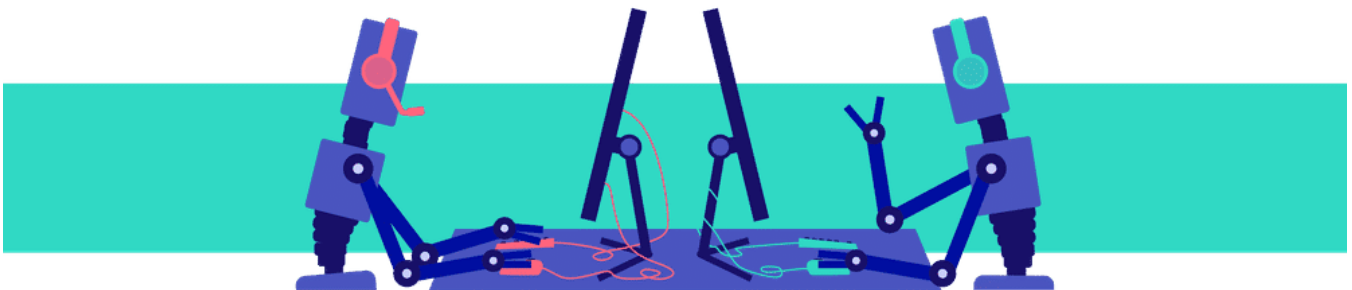
permitiendo una **flexibilidad** sin precedentes en la configuración de despliegues y mejorando significativamente la **visibilidad** de todo el proceso, lo que contribuye a la detección y resolución de problemas.

(D) Parámetros en tiempo de ejecución

Los parámetros en tiempo de ejecución permiten **personalizar las ejecuciones de pipelines en Azure Pipelines**. Así, podemos definir parámetros y ajustarlos en el momento de ejecutar el pipeline llevando a cabo configuraciones de entorno, versiones de aplicaciones o cualquier otro valor requerido.

Gracias a estos parámetros, los pipelines son más flexibles y reutilizables, permitiendo ajustes dinámicos sin necesidad de modificar su código. Resultan especialmente útiles en escenarios donde se requiere desplegar la misma aplicación en múltiples entornos con configuraciones ligeramente diferentes o bien cuando se han de probar diferentes configuraciones de forma rápida y sencilla.

¿Sabes cuáles son las fases y tareas en Azure Pipelines? Avanza para descubrirlo.



6.4.3 Fases y tareas en Azure Pipelines

En Azure Pipelines, el proceso de CI/CD se organiza mediante fases y tareas, que podemos considerar los **bloques de construcción** de nuestros pipelines. Esta estructuración contribuye a

organizar y automatizar de manera efectiva el flujo de trabajo de desarrollo de software, desde el código fuente hasta el despliegue en producción.

Podemos concebir las fases como **etapas clave o pasos** en nuestro proceso de CI/CD. Así, cada fase comprende un **conjunto de tareas relacionadas** que representan una sección significativa de nuestro pipeline, como la compilación, las pruebas o el despliegue. Las fases se pueden ejecutar una tras otra (**secuencialmente**) o simultáneamente (**en paralelo**) en función de cómo deseemos organizar nuestro flujo de trabajo. Por ejemplo, podemos dedicar una fase a compilar el código y, seguidamente, abordar una fase destinada a ejecutar pruebas automatizadas para, a continuación, iniciar el despliegue del software.

En cambio, las tareas, comprendidas dentro de cada fase, son las **acciones específicas** que se llevan a cabo, **pasos individuales** como compilar el código, ejecutar pruebas unitarias o desplegar una aplicación. Azure Pipelines proporciona una amplia variedad de **tareas predefinidas** dirigidas a realizar operaciones comunes y, además, nos permite definir tareas personalizadas para adaptarlas a necesidades específicas. Cada tarea se configura mediante **parámetros** que establecen cómo se debe ejecutar, por ejemplo, la ruta al código fuente o los argumentos de línea de comandos para las herramientas de compilación.



Al estructurar nuestro pipeline en fases y tareas, mantenemos un control detallado sobre cada paso del proceso de CI/CD, lo que contribuye a automatizar completamente desde la integración hasta el despliegue. Esta organización no solo facilita la gestión y el mantenimiento de nuestros pipelines, sino que también mejora la eficiencia al permitirnos identificar y solucionar problemas en determinadas etapas del proceso.

6.4.4 Desencadenantes en Azure Pipelines

Los desencadenantes son **mecanismos que inician la ejecución de un pipeline**. En Azure Pipelines, pueden configurarse para responder a una variedad de eventos, lo que permite automatizar el proceso de CI/CD y asegurar que el software se integra y entrega de manera oportuna y eficiente.

- Manual: Nos permite iniciar pipelines manualmente desde la interfaz de usuario de Azure DevOps o mediante la API. Resulta útil para despliegues ad-hoc o pruebas de pipelines.
- Basado en eventos: Inicia la ejecución del pipeline en respuesta a eventos específicos, como el push de cambios a un repositorio Git o la actualización de un paquete en Azure Artifacts. De este modo, garantiza que el código se integra y prueba continuamente, facilitando la detección temprana de errores.
- Pull Request (PR): Se activa cuando se crea o actualiza un pull request en el repositorio vinculado, lo que resulta esencial para validar cambios antes de que se fusionen, proporcionando un proceso de revisión de código más riguroso y automatizado.
- Programado: Permite ejecutar pipelines en momentos específicos, por ejemplo, para despliegues nocturnos o pruebas regulares. Por consiguiente, resulta especialmente útil para aquellas tareas que no necesitan ser ejecutadas con cada cambio en el repositorio.

6.4.5 Trabajo avanzado con Azure Pipelines

Para profundizar en el uso de Azure Pipelines, resulta crucial comprender cómo manejar variables y secretos, cómo implementar estrategias de despliegue avanzadas y cómo controlar el flujo de ejecución con condiciones y expresiones. Estos elementos permiten personalizar y asegurar nuestros pipelines, así como adoptar prácticas de despliegue modernas para una entrega de software eficiente y confiable.

A. Usos de variables

En Azure Pipelines, las variables son una **forma de almacenar y reutilizar valores a lo largo de nuestro pipeline**. Así, podemos definir variables a nivel de pipeline, de fase o de tarea, lo que nos permite transferir información y configuraciones de manera dinámica. Las variables se pueden

establecer manualmente a través de la interfaz de usuario de Azure DevOps o bien calcular y asignar durante la ejecución del pipeline.

Esta última opción resulta especialmente útil para versiones de software, rutas de acceso o cualquier otro dato que pueda variar entre ejecuciones o entornos.

B. Gestión de secretos y credenciales

La gestión segura de secretos y credenciales resulta fundamental para proteger nuestro código e infraestructura. Azure Pipelines ofrece varias alternativas para manejar información sensible, como **variables secretas**, **grupos de variables vinculadas a Azure Key Vault** o la integración con sistemas de gestión de secretos.

Resulta vital que los secretos nunca se expongan en logs o se almacenen en repositorios de código; siempre debemos emplear referencias a secretos almacenados de forma segura y asignar los permisos mínimos necesarios de cara a su uso.

C. Condiciones y expresiones

Las condiciones y las expresiones nos permiten **controlar el flujo de ejecución dentro de nuestros pipelines**. En este sentido, podemos especificar condiciones para ejecutar fases, tareas e incluso pipelines enteros basados en el resultado de pasos anteriores, variables de entorno o cualquier otra lógica personalizada. Esta última opción resulta particularmente útil para crear pipelines que se adapten dinámicamente a cambios en el código, a entornos o a la necesidad de intervención manual.

6.5 Azure Artifacts

Azure Artifacts es un servicio de Azure DevOps que permite a los equipos **compartir, publicar y consumir paquetes de *software* en un entorno seguro y privado**. Funciona como un repositorio de paquetes que se integra perfectamente con el sistema de CI/CD (integración continua / entrega continua), facilitando la gestión de dependencias y la automatización de la entrega de *software*.



Los paquetes pueden presentar varios formatos, incluyendo NuGet para .NET, NPM para JavaScript, Maven para Java o Python para paquetes PyPI, entre otros muchos, lo que convierte a Azure Artifacts en una **solución versátil para aquellos equipos que trabajan en una amplia variedad de tecnologías y lenguajes de programación**. Además de almacenar paquetes propios, Azure Artifacts permite acceder a paquetes públicos desde repositorios externos, centralizando así la gestión de dependencias.

Una de las principales ventajas derivadas del uso de Azure Artifacts reside en su capacidad para **crear y gestionar** feeds privados que pueden configurarse para un proyecto específico, para un equipo dentro de una organización o incluso a nivel de toda la organización, proporcionando un control granular acerca de quién puede acceder y consumir los paquetes. De este modo, resulta especialmente útil para aquellas empresas que necesitan asegurar que sus dependencias de

software sean seguras, confiables y permanezcan disponibles exclusivamente para sus proyectos y equipos.

6.5.1 Ventajas de usar feeds privados vs. *feeds* públicos

Los *feeds* de paquetes son **repositorios donde se almacenan y se comparten paquetes de *software***. Pueden albergar bibliotecas, herramientas, scripts y otros componentes reutilizables que los desarrolladores utilizan en sus aplicaciones. De este modo, permiten a los equipos centralizar y gestionar las dependencias de sus proyectos, facilitando el acceso y la distribución de paquetes dentro de una organización o con el público en general.

Feeds privados	Tipos de feed
Riqueza de recursos. Ofrecen una amplia gama de paquetes y dependencias ya existentes, lo que permite a los desarrolladores localizar y utilizar rápidamente soluciones para problemas comunes sin necesidad de reinventar la rueda.	Gestión de dependencias propias. Facilitan la compartición y gestión de paquetes internos desarrollados específicamente para los proyectos de la organización.
Colaboración y comunidad. Fomentan la colaboración y el intercambio de conocimientos dentro de la comunidad de desarrollo, de forma que los equipos puedan beneficiarse de las contribuciones de otros y contribuir a proyectos de código abierto.	Integración y automatización. Se integran estrechamente con Azure DevOps y otros sistemas de CI/CD, automatizando el flujo de trabajo de publicación y consumo de paquetes, lo que aumenta la eficiencia y reduce los errores.

Feeds privados	Tipos de feed
Accesibilidad. Los feeds públicos son accesibles para cualquier desarrollador, lo que permite compartir y consumir paquetes de software de código abierto y otros recursos disponibles públicamente.	Personalización y consistencia. Permiten personalizar los paquetes y garantizar que los equipos utilicen versiones específicas de las dependencias, manteniendo la consistencia en los entornos de desarrollo, prueba y producción.
	Control de acceso. Los feeds privados restringen el acceso a los paquetes, asegurando que solo los miembros autorizados del equipo o de la organización puedan consumirlos, lo que resulta crucial para preservar la seguridad y la privacidad del código, especialmente en proyectos sensibles o propietarios.

En resumen, los **feeds privados** ofrecen **seguridad, control y personalización** para los equipos que necesitan gestionar sus dependencias de forma segura y eficiente, mientras que los **feeds públicos** promueven **la accesibilidad, la colaboración y el uso compartido de recursos** dentro de la comunidad de desarrollo. Optar por un feed privado o público es una decisión que depende de las necesidades específicas del proyecto, de los requisitos de seguridad y de la estrategia de colaboración de la organización o del equipo de desarrollo.

RECUERDA

Una de las principales ventajas derivadas del uso de Azure Artifacts reside en su capacidad para crear y gestionar feeds privados que pueden configurarse para un proyecto específico, para un equipo dentro de una organización.

6.5.2 Integración entre Azure Artifacts y Azure Pipelines

La integración entre Azure Artifacts y Azure Pipelines refuerza los flujos de trabajo de CI/CD, permitiendo una **gestión automatizada y eficiente** tanto de la publicación como del consumo de paquetes. Mediante esta integración, se simplifica la publicación automática de paquetes tras cada *build* exitoso, lo que garantiza que las versiones más recientes permanezcan siempre disponibles.

Además, facilita la gestión de dependencias, permitiendo que los pipelines consuman paquetes de Azure Artifacts de manera controlada y segura, lo que asegura la consistencia y estabilidad de los entornos de desarrollo. Este enfoque automatizado y centralizado mejora significativamente la eficiencia, la seguridad y la gestión de versiones en los proyectos de software.

6.6 Introducción a Azure Test Plans

Azure Test Plans es un componente clave de la suite Azure DevOps que proporciona un **conjunto integral de herramientas para la planificación, ejecución y seguimiento de pruebas de software**. Diseñado para adaptarse tanto a metodologías ágiles como a enfoques más tradicionales, Azure Test Plans ofrece una solución robusta a la hora de gestionar pruebas manuales y automatizadas, permitiendo a los equipos asegurar la calidad del software a lo largo del ciclo de vida del desarrollo.



Este componente ofrece una gama amplia de capacidades de prueba, entre otras:

- Pruebas manuales: Herramientas para la creación y ejecución de casos de prueba manuales que permiten a los testers registrar los resultados y observaciones de forma detallada.
- Pruebas automatizadas: Integración con marcos de pruebas automatizadas para ejecutar comprobaciones como parte de pipelines de CI/CD, apoyando la entrega continua.

- Gestión de planes y suites de prueba: Organización de pruebas en planes y suites para asegurar una mejor cobertura de los requisitos y características del software.
- Trazabilidad y colaboración: Integración con Azure Boards de cara a la trazabilidad completa, desde requisitos hasta defectos, y la colaboración entre equipos de desarrollo y pruebas.
- Análisis e informes: Generación de informes detallados sobre el progreso y la calidad de las pruebas, lo que contribuye a identificar tendencias, áreas problemáticas y oportunidades de mejora.

La integración de **Azure Test Plans** en el ciclo de vida del desarrollo de software contribuye significativamente al éxito de los proyectos al asegurar que los productos finales reúnen los estándares de calidad esperados y satisfacen las necesidades del usuario final. En este sentido, permite a los equipos:

- Detectar y corregir errores tempranamente en el ciclo de desarrollo.
- Mejorar la comunicación y colaboración entre desarrolladores, testers y partes interesadas.
- Adaptarse rápidamente a cambios en los requisitos mediante pruebas iterativas y continuas.
- Reducir los tiempos y los costes asociados mediante la reparación de defectos detectados tardíamente en el desarrollo o después del lanzamiento.

6.6.1 Elementos fundamentales de Azure Test Plans

Azure Test Plans consta de varios elementos fundamentales que trabajan conjuntamente para proporcionar un **enfoque estructurado y eficiente de cara a la gestión de pruebas** dentro del ciclo de vida del desarrollo de software. Dichos elementos comprenden planes de prueba, suites de prueba, casos de prueba y puntos de prueba, desempeñando cada uno de ellos una función específica en el proceso.

- Planes de prueba: Un plan de prueba es un contenedor que agrupa todo cuanto necesitamos para el proceso de pruebas en un ciclo de lanzamiento específico. Funciona como un proyecto de pruebas donde podemos definir el alcance de estas, los objetivos, el calendario y los recursos necesarios. Los planes de prueba contribuyen a organizar y a seguir el progreso de las actividades de pruebas, asegurando que se cubran todos los aspectos necesarios para validar la calidad del software.
- Conjuntos de pruebas: Agrupan casos de prueba relacionados dentro de un plan de prueba y pueden organizarse en base a criterios como características del software, requisitos o funcionalidades. Los conjuntos de pruebas facilitan su gestión detallada, permitiendo asignar pruebas específicas a testers y seguir el progreso de su ejecución por áreas de interés.
- Casos de prueba: Se trata de las unidades individuales de prueba que detallan los pasos específicos que se han de seguir, los datos de entrada requeridos y los resultados esperados para una determinada prueba. Resultan fundamentales para asegurar una evaluación exhaustiva y sistemática de cada función del software, contribuyendo eficientemente a identificar y corregir defectos.

6.6.2 Gráficos de seguimiento del estado de prueba y configuraciones

Distinguimos entre:

Gráficos de seguimiento de estado de prueba	Configuraciones de prueba
Proporcionan una visualización en tiempo real del progreso y de los resultados de las pruebas, de forma que los equipos puedan monitorizar la calidad del software a lo largo del ciclo de desarrollo.	Permiten definir distintos entornos o configuraciones de cara a la ejecución de los casos de prueba, como diferentes sistemas operativos o especificaciones de hardware. De este modo, se garantiza que el software funcione correctamente en diversas condiciones, reflejando una variedad de entornos de usuario final.

Tema 7: Monitoreo y logging en Azure



Tokio.
School



Introducción



7.1 Conceptos básicos



7.2 Tipos de datos recopilados en monitoreo y logging



7.3 Herramientas de monitoreo y logging en Azure



Resumen



Bibliografía

Introducción

El monitoreo y el *logging* en Azure no solo permiten a los equipos de IT y DevOps mantener un pulso constante sobre el **estado y el rendimiento** de sus aplicaciones e infraestructura, sino que también habilitan la capacidad de actuar proactivamente ante cualquier anomalía detectada. Desde la optimización de recursos hasta la garantía de seguridad y el cumplimiento de regulaciones, estas prácticas son indispensables para cualquier estrategia de gestión de IT que busque no solo reaccionar ante los problemas, sino anticiparse a ellos mediante el análisis de tendencias y patrones en los datos recopilados.

A lo largo de este tema, se desglosarán los tipos de datos recopilados en el monitoreo y el *logging*, desde métricas y registros hasta trazas, y se examinarán las herramientas de Azure específicas como Azure Monitor, **Azure Application Insights** y **Azure Log Analytics**. Cada sección está diseñada para ofrecer una visión integral de cómo estas herramientas y prácticas se aplican en escenarios reales, proporcionando a los estudiantes las habilidades y conocimientos necesarios para implementar soluciones de monitoreo y logging efectivas en Azure.

Estos conceptos no solo son imprescindibles para los profesionales técnicos que buscan optimizar y asegurar sus entornos de nube, sino también para aquellos en roles de gestión y operaciones, ya que la implementación efectiva de estas prácticas afecta directamente la calidad, seguridad y eficiencia del producto final. Al final de este tema, los estudiantes estarán equipados con una comprensión sólida de cómo el monitoreo y el *logging* se integran en una estrategia de DevOps, permitiéndoles aplicar estos principios para mejorar la colaboración entre equipos, la eficiencia en los procesos y la calidad del producto final en el contexto de Azure.

7.1 Conceptos básicos

En el ecosistema de Azure, el monitoreo y el *logging* representan los pilares del mantenimiento y la optimización de aplicaciones e infraestructura en la nube. Estas herramientas no solo proporcionan una visión clara del estado de los recursos en la nube, sino que también facilitan una comprensión profunda de cómo se comportan las aplicaciones en el entorno de producción, lo que permite a los equipos de IT y DevOps actuar rápidamente ante cualquier anomalía detectada.



7.1.1 Procesos principales

En primer lugar, es necesario delimitar que entendemos por monitoreo y *logging*.

- A . Monitoreo: El monitoreo (o monitorización) se centra en la recopilación y análisis de métricas que reflejan el rendimiento y la salud de los servicios y las aplicaciones en la nube. Mediante herramientas como Azure Monitor, los equipos pueden acceder a datos en tiempo real para evaluar el estado de su infraestructura. Supongamos que estamos supervisando un sistema de comercio electrónico. Por medio del monitoreo, podemos rastrear la

latencia, el tiempo de carga de las páginas, así como el uso de la CPU y de la memoria, en tiempo real. Si una métrica excede los umbrales definidos —por ejemplo, un aumento repentino en el tiempo de carga de la página—, podemos intervenir rápidamente para identificar y solucionar el problema, minimizando así el impacto en la experiencia del usuario final.

- B. Logging: El logging (o registros), por su parte, se orienta a la creación y gestión de registros (logs) que documentan eventos, transacciones y errores dentro de un sistema o aplicación. Estos registros resultan esenciales para el análisis posterior de incidentes, la auditoría de seguridad y el diagnóstico y solución de problemas. A diferencia del monitoreo, que ofrece una panorámica en tiempo real, el logging proporciona un historial detallado de lo que ha ocurrido en el sistema, lo que resulta imprescindible de cara a la comprensión profunda de los eventos una vez se han producido. Consideremos una aplicación de mensajería donde los usuarios y las usuarias reportan problemas esporádicos al enviar mensajes. Mediante el análisis de logs, podemos determinar que los errores se producen únicamente bajo ciertas condiciones, como picos de tráfico o fallos de conexión específicos. Estos detalles, registrados en los logs, resultan esenciales para diagnosticar y resolver el problema de manera efectiva, permitiendo ajustes precisos en la configuración, en el código de la aplicación o en la infraestructura.

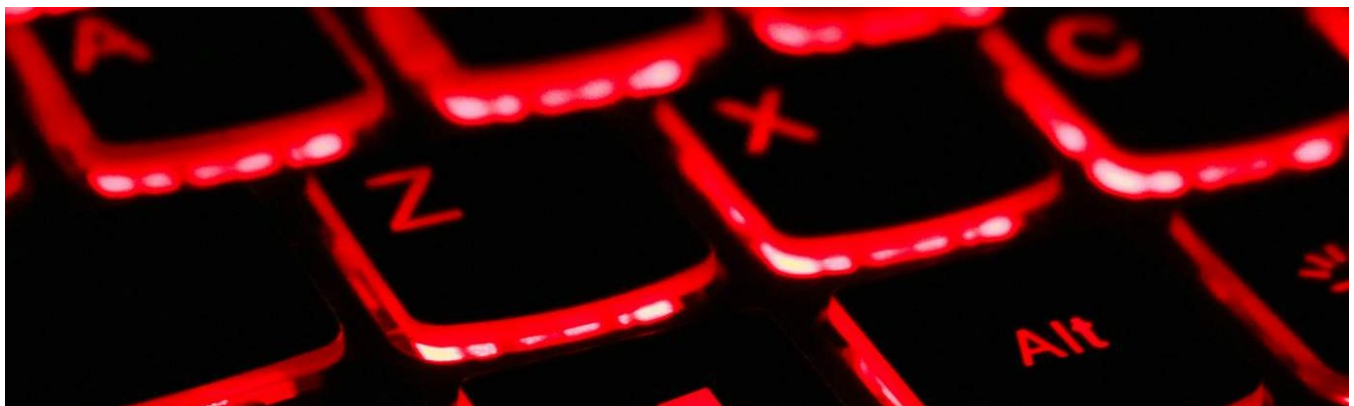
7.1.2 Importancia del monitoreo y el *logging*

Ambas prácticas, aun cuando difieren en su enfoque y aplicación, son complementarias y vitales de cara a la gestión y la operación eficientes de sistemas complejos. En entornos de nube como Azure, donde la infraestructura y las aplicaciones pueden escalar de forma dinámica y cambiar rápidamente, el monitoreo y el *logging* todavía resultan más críticos.

- Proactividad: Detectar y resolver problemas antes de que repercutan en los usuarios finales.
- Optimización de recursos: Adoptar decisiones informadas para la mejora continua del rendimiento y la eficiencia.
- Seguridad y cumplimiento: Asegurar la integridad de los sistemas y cumplir con las regulaciones mediante el seguimiento detallado de las actividades.

7.2 Tipos de datos recopilados en monitoreo y logging

En el ámbito del monitoreo y el *logging*, se reúnen **diversos tipos de datos** que presentan distintos propósitos y utilidades. Estos datos resultan útiles para comprender el comportamiento, el rendimiento y los posibles problemas que pueden producirse dentro de una infraestructura o aplicación. En los siguientes apartados, nos ocuparemos de los principales tipos de datos.



7.2.1 Métricas

Las métricas son **indicadores cuantitativos** que proporcionan información vital sobre el rendimiento, la salud y la disponibilidad de sistemas y aplicaciones. Se recogen en **tiempo real** mediante **herramientas de monitoreo**, contribuyendo a que los equipos técnicos puedan evaluar el estado operativo de sus entornos digitales.

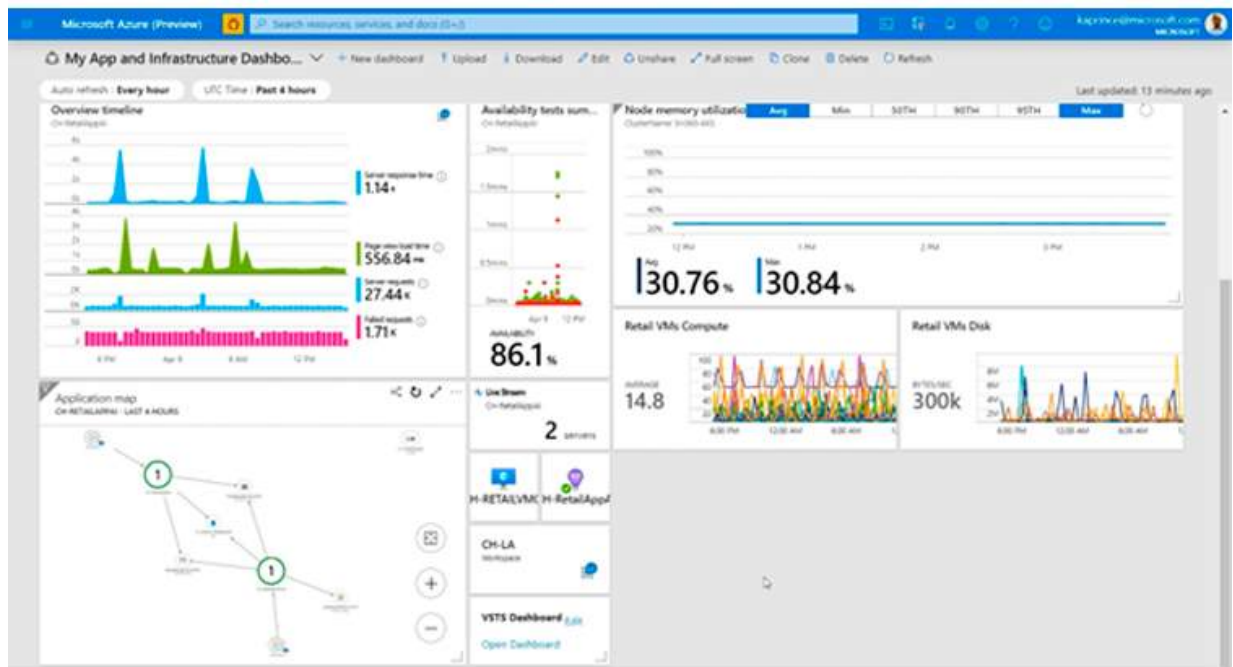


Figura 58. Dashboard de monitoreo

(A) Ejemplos de métricas comunes

A continuación veremos algunos ejemplos:

- **Uso de CPU:** Porcentaje de uso de CPU utilizado por una aplicación o proceso. Un uso elevado de forma sostenida puede indicar una carga de trabajo excesiva o un bucle infinito en el código.
- **Memoria usada:** Cantidad de memoria RAM utilizada por una aplicación o sistema. Resulta útil para detectar fugas de memoria o configuraciones inadecuadas de asignación de recursos.
- **Latencia de red:** Tiempo que tardan en viajar los datos desde el origen hasta el destino. Se trata de una métrica importante para evaluar la experiencia del usuario en aplicaciones web.
- **Errores por segundo:** Frecuencia de los errores generados por una aplicación. Contribuye a identificar problemas de estabilidad o bugs en el software.

- Tasa de transacciones: Número de transacciones completadas en un período específico. Resulta fundamental para sistemas de comercio electrónico o aplicaciones de procesamiento de transacciones.

(B) Casos de uso habitual

En la siguiente tabla veremos representados los casos de uso habitual

Monitoreo de aplicaciones web	Infraestructura en la nube	Sistemas de bases de datos
La latencia de red, el tiempo de respuesta del servidor o el uso de CPU son métricas críticas para mantener la experiencia del usuario final. Así, por ejemplo, un aumento en la latencia de red puede indicar problemas con el proveedor de servicios de internet (ISP) o una configuración inadecuada del balanceador de carga.	El monitoreo del uso de recursos, como CPU, memoria y almacenamiento, resulta esencial para optimizar costes y rendimiento en entornos de nube. Así, por ejemplo, detectar patrones de uso puede ayudar a ajustar las instancias reservadas y las escalas automáticas con el objeto de manejar las cargas de trabajo de manera eficiente.	El monitoreo de métricas como la latencia de las consultas, el tiempo de cada petición y la conexión de usuarios simultáneos contribuye a garantizar el rendimiento óptimo de las bases de datos. Una latencia elevada en las consultas podría señalar la necesidad de optimizar índices o de ajustar la configuración del servidor.

El monitoreo de métricas permite a los equipos de desarrollo y operaciones responder de manera proactiva a los problemas, optimizar el rendimiento y planificar capacidades futuras basadas en datos reales.

7.2.2 Logs

Los logs representan una pieza fundamental de cara a la gestión y operación de sistemas informáticos, ya que proporcionan un **registro detallado y cronológico de eventos, acciones y errores**. A diferencia de las métricas, que ofrecen una vista cuantitativa y resumida, los logs brindan un contexto **cualitativo** y detallado que resulta esencial para el análisis postincidente, la seguridad y el cumplimiento normativo.

OPERATION NAME	STATUS	TIME	TIME STAMP	SUBSCRIPTION	EVENT INITIATED BY
Create or Update Virtual Machine	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	
Append	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	Microsoft Azure Policy Insights
DeployIfNotExists	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	Microsoft Azure Policy Insights
DeployIfNotExists	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	Microsoft Azure Policy Insights
Create or Update Virtual Machine	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	
Create or Update Virtual Machine	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	
Create or Update Virtual Machine Extension	Succeeded	3 min ago	Sat May 11 ...	Contoso IT - demo	
Create Deployment	Failed	3 min ago	Sat May 11 ...	Contoso IT - demo	
Create Deployment	Failed	3 min ago	Sat May 11 ...	Contoso IT - demo	
DeployIfNotExists	Failed	4 min ago	Sat May 11 ...	Contoso IT - demo	
Returns Storage Account SAS Token	Succeeded	5 min ago	Sat May 11 ...	Contoso IT - demo	Windows Azure Application Insights
Validate Deployment	Succeeded	6 min ago	Sat May 11 ...	Contoso IT - demo	

Figura 59. Registro de logs

(A) Ejemplos de logs

A continuación veremos algunos ejemplos:

- **Logs de aplicaciones:** Registran las acciones realizadas por una aplicación, incluyendo inicio y cierre de sesión, errores internos, excepciones capturadas y transacciones específicas. De este modo, ayudan a entender el comportamiento de la aplicación desde el punto de vista del usuario y del sistema.
- **Logs de servidores web:** Contienen detalles sobre las solicitudes HTTP recibidas, incluyendo la dirección IP del solicitante, la página solicitada, el código de estado HTTP y el agente de usuario. Resultan cruciales para analizar el tráfico web y detectar patrones anómalos o ataques.

- Logs de sistemas operativos: Proporcionan información sobre eventos del sistema, como arranques y suspensiones, actualizaciones de software, fallos de hardware y acciones de usuarios. Adquieren importancia para diagnosticar problemas del sistema y para las auditorías de seguridad.
- Logs de seguridad: Especializados en registrar intentos de acceso, tanto exitosos como fallidos, cambios en las configuraciones de seguridad y otras acciones relevantes de cara a la seguridad de la información. Resultan esenciales para la detección de intrusiones y la gestión de vulnerabilidades.

(B) Casos de uso habitual

- Diagnóstico de errores de aplicación: Cuando una aplicación falla o se comporta de manera inesperada, los desarrolladores revisan sus logs para rastrear las acciones que ocasionaron el error, lo que permite identificar la causa raíz y desarrollar una solución específica.
- Análisis de tráfico web: Los administradores de sistemas y los especialistas en seguridad utilizan los logs de servidores web para monitorear el tráfico, identificar patrones de acceso inusual y detectar posibles ataques, como inyecciones SQL o ataques de fuerza bruta.
- Auditorías de seguridad y cumplimiento: Las organizaciones recurren a los logs de seguridad para verificar el cumplimiento de las políticas internas y las regulaciones externas. Mediante estos registros, se puede constatar que se han adoptado las medidas adecuadas para proteger los datos y la infraestructura.
- Optimización del rendimiento: El análisis de logs permite identificar cuellos de botella en el rendimiento de las aplicaciones y la infraestructura. Así, por ejemplo, los logs de aplicaciones que muestran errores de tiempo de espera reiterados pueden indicar la necesidad de optimizar la base de datos o de aumentar los recursos del servidor.

En definitiva, los logs resultan indispensables para una amplia gama de actividades, desde el **desarrollo de software** hasta la **gestión de la seguridad de la información**. Proporcionan una visión detallada y analítica de lo que sucede dentro de los sistemas y aplicaciones, lo que permite a los equipos técnicos actuar de manera informada y precisa. Al almacenar, gestionar y analizar logs de manera efectiva, las organizaciones pueden mejorar significativamente su capacidad para diagnosticar problemas, asegurar sus sistemas y cumplir con los requisitos regulatorios, contribuyendo así a la creación de entornos de TI más robustos, seguros y eficientes.

Cabe señalar que muchos países disponen de **regulaciones** en función del servicio proporcionado que obligan a mantener durante un tiempo definido el registro de logs. Además, estos logs han de presentar ciertos **protocolos** de copias de seguridad y de protección de datos de los usuarios.

RECUERDA

Los logs representan una pieza fundamental de cara a la gestión y operación de sistemas informáticos, ya que proporcionan un registro detallado y cronológico de eventos, acciones y errores.

7.2.3 Trazas

Para entender las trazas, resulta necesario comprender qué es un **sistema distribuido**, se trata de aplicaciones integradas por componentes ubicados en diferentes redes informáticas que se comunican y coordinan sus acciones mediante la transferencia de mensajes entre sí.

De este modo, proporcionan una elevada disponibilidad, escalabilidad y flexibilidad. Son ejemplos habituales de sistemas distribuidos las arquitecturas de microservicios, las aplicaciones en la nube, así como las plataformas de procesamiento de datos a gran escala o los nodos de máquinas virtuales.

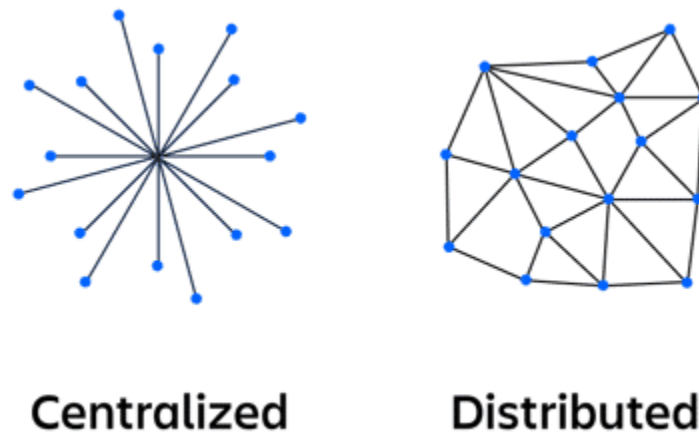


Figura 60. Distribución de las trazas

Un caso simple de aplicación distribuida es una web que consta de un frontend y un backend, donde ambos componentes se encuentran aislados y en máquinas diferentes, aunque se comunican entre sí a través de mensajes.

Las trazas son una herramienta fundamental de cara al **diagnóstico y optimización de aplicaciones distribuidas**.

A diferencia de los logs, que registran eventos específicos, y de las métricas, que proporcionan mediciones cuantitativas, las trazas ofrecen una **narrativa detallada del recorrido de una solicitud a través de los distintos componentes y servicios**. Esta capacidad para rastrear el trayecto completo de una solicitud resulta importante a la hora de analizar la complejidad

inherente a los sistemas distribuidos. Las trazas permiten a los equipos de DevOps comprender cómo fluyen las solicitudes a través de la infraestructura distribuida, facilitando la identificación de cuellos de botella, dependencias ineficientes y fallos en la ejecución de las solicitudes.

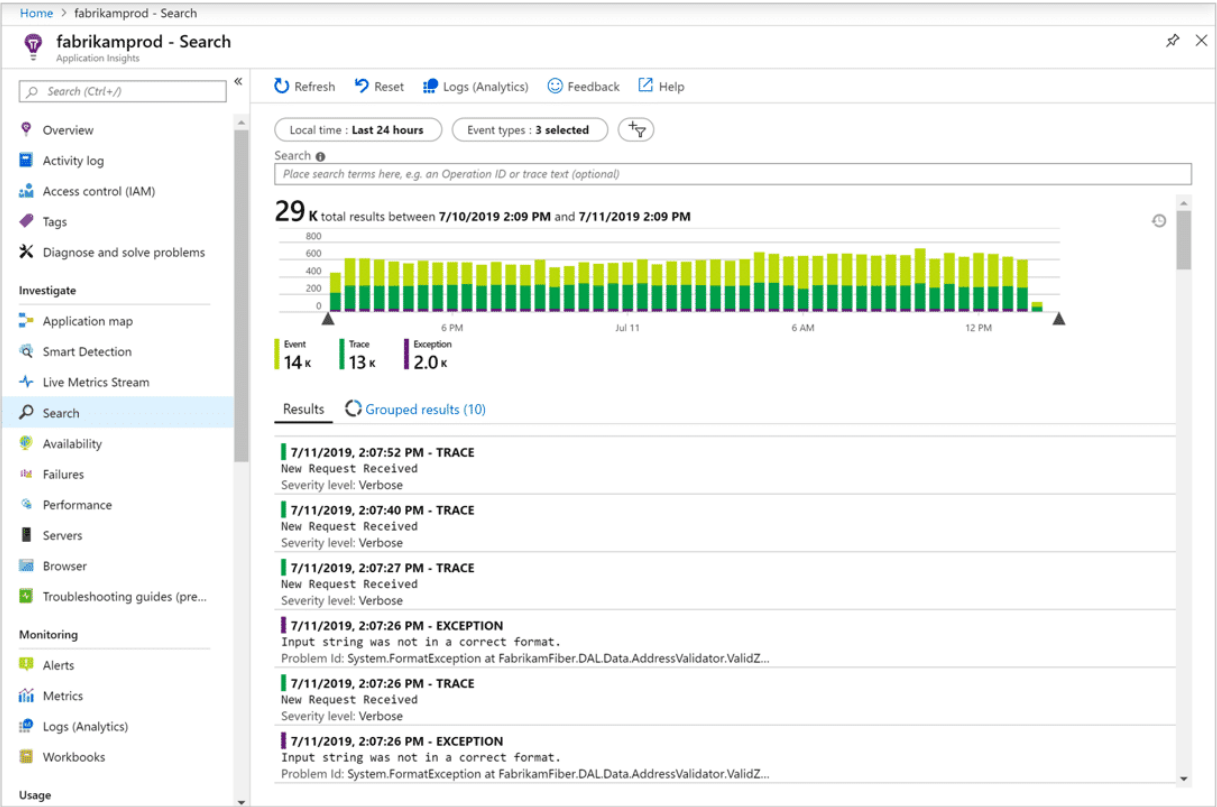


Figura 61. Ejemplo de desarrollo de trazas

(A) Ejemplos de trazas

A continuación veremos una tabla:

Trazas de solicitudes HTTP	Trazas de transacciones de base de datos	Trazas de flujo de aplicaciones
Documentan cada uno de los pasos que efectúa una solicitud HTTP en una arquitectura de microservicios, incluyendo llamadas a bases de datos, servicios externos y procesamiento interno	Registran las operaciones realizadas en una base de datos como parte de una transacción, permitiendo identificar problemas de rendimiento o bloqueos en el acceso a los datos.	Mapean el flujo de una operación a través de las diferentes capas de una aplicación, desde la interfaz de usuario hasta la lógica de negocio y las operaciones de backend.

(B) Casos de uso habitual

- Identificación de cuellos de botella en microservicios: En entornos de microservicios, las trazas permiten a desarrolladores y operadores visualizar el flujo completo de una solicitud a través de múltiples servicios, lo que resulta crucial para detectar cuellos de botella o dependencias ineficientes que podrían incidir en el rendimiento.
- Análisis de fallos en la ejecución de solicitudes: Las trazas proporcionan el contexto necesario para entender por qué ha fallado una solicitud o no se ha comportado como se esperaba. Por ejemplo, cuando falla una solicitud de pago en un sistema de comercio electrónico, las trazas pueden ayudar a identificar si el problema se originó en el servicio de pagos, en la comunicación con un servicio bancario externo o en otro lugar.
- Optimización del rendimiento de aplicaciones: Dado que proporcionan una vista detallada de cómo fluyen las solicitudes a través de una aplicación, las trazas permiten a los ingenieros analizar y optimizar el rendimiento de las interacciones entre servicios, lo que puede conducir a mejoras significativas en la velocidad de respuesta y en la eficiencia general del sistema.
- Depuración de aplicaciones distribuidas: Las trazas resultan fundamentales para depurar aplicaciones distribuidas, donde los problemas pueden surgir de la interacción compleja entre múltiples componentes. En este sentido, proporcionan una visión clara de tales interacciones, facilitando la identificación y resolución de errores.

Por tanto, las trazas no solo facilitan **la comprensión y el diagnóstico de los sistemas distribuidos**, sino que también constituyen una herramienta para la optimización continua y para **mejorar la resistencia a fallos y la eficiencia** de estas complejas arquitecturas.

7.3 Herramientas de monitoreo y logging en Azure

Azure Monitor es una herramienta de Microsoft Azure diseñada para proporcionar una visión completa y detallada del rendimiento, la salud y la disponibilidad de las aplicaciones y la infraestructura en la nube.

Este servicio integral permite a desarrolladores, operadores de TI y profesionales de DevOps recopilar, analizar y actuar sobre los datos recogidos de sus entornos en Azure y *on-premise*, facilitando la detección y resolución de problemas, la optimización del rendimiento y la mejora de la seguridad.



(A) Recuperación de los datos en azure monitor

Azure Monitor reúne datos de múltiples fuentes para proporcionar una supervisión completa y detallada de aplicaciones, infraestructura y servicios en la nube.

La capacidad de integrar y analizar datos desde diversos orígenes cobra importancia para ofrecer una visión completa del rendimiento, la salud y la disponibilidad de los recursos en Azure, así como en entornos on-premise y en otras nubes.

A continuación, nos detendremos en las principales fuentes a las que puede acudir Azure Monitor para reunir datos:

- Aplicaciones o cargas de trabajo: Contempla datos sobre el rendimiento, el estado y la actividad de las aplicaciones, así como información sobre cargas de trabajo específicas que se ejecutan en infraestructuras como servicio (IaaS), por ejemplo, bases de datos y aplicaciones empresariales en máquinas virtuales.
- Infraestructura: Se trata de datos sobre contenedores y sistemas operativos, lo que abarca desde la supervisión de contenedores y orquestadores como Azure Kubernetes Service (AKS) hasta la recopilación de información sobre el rendimiento y la salud de los sistemas operativos donde se ejecutan las aplicaciones.
- Plataforma Azure: Comprende datos generados por los recursos de Azure, la gestión y el estado de las suscripciones de la plataforma y el funcionamiento de sus servicios a nivel de inquilino, como Microsoft Entra ID. Estos datos ofrecen una visión interna del funcionamiento de los recursos y servicios de Azure.
- Orígenes personalizados: Azure Monitor también permite integrar datos personalizados a través de sus API habilitando la supervisión de información específica sobre aplicaciones o infraestructuras que no se recogen de forma automática.
- Integración con SCOM: Para entornos híbridos, Azure Monitor se puede conectar con System Center Operations Manager (SCOM) para reunir datos de infraestructuras on-premise, centrándose en cargas de trabajo de IaaS y en datos de sistemas operativos.

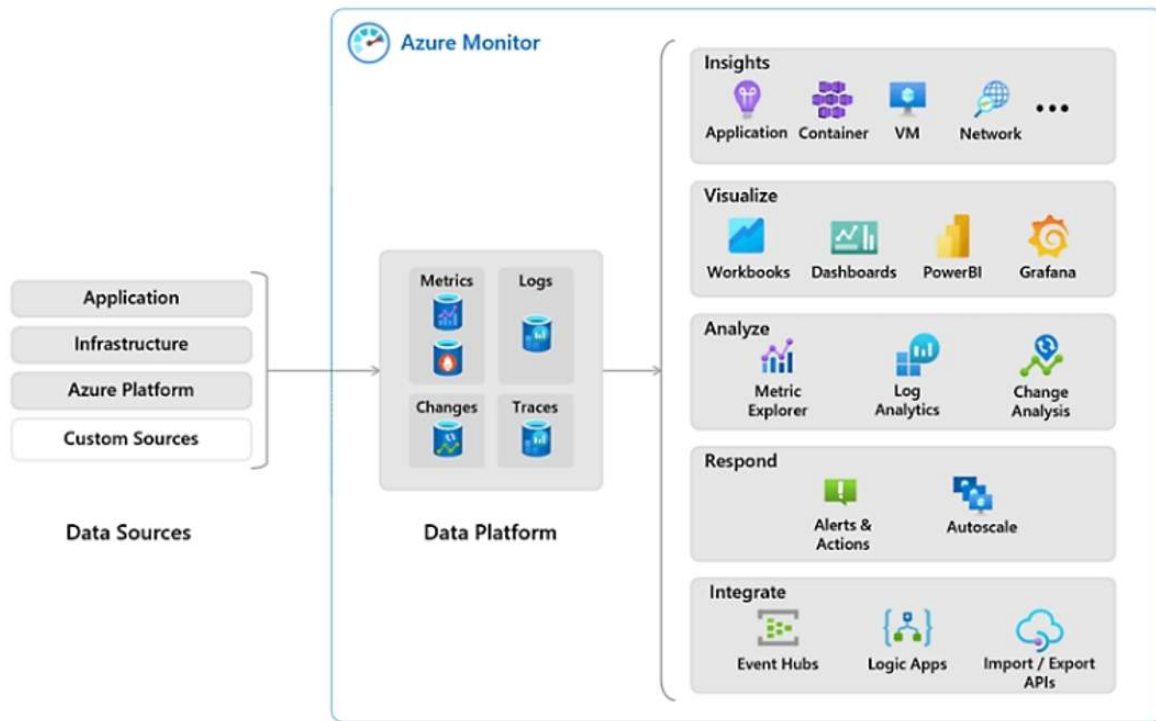


Figura 62. Visualización del monitor de Azure

Azure Monitor emplea diversos métodos para reunir datos de los distintos orígenes mencionados anteriormente, asegurando una cobertura tanto para recursos dentro de Azure como para infraestructuras y aplicaciones externas.

Entre los métodos de recogida, podemos referirnos a los agentes que se instalan en máquinas virtuales o servidores físicos, a integraciones directas con servicios de Azure que reportan automáticamente métricas y logs y a API específicas como la API de REST de Azure Monitor o la API de recopilación para la ingesta de datos personalizados.

Asimismo, para entornos híbridos, Azure Monitor puede integrarse con **System Center Operations Manager (SCOM)** para reunir datos de sistemas locales. Este enfoque multifacético permite que Azure Monitor proporcione una visión completa y detallada del estado y del rendimiento de las aplicaciones, la infraestructura y los servicios.

RECUERDA

Azure Monitor reúne datos de múltiples fuentes para proporcionar una supervisión completa y detallada de aplicaciones, infraestructura y servicios en la nube.

(B) Almacenes y datos en azure monitor

Azure Monitor cuenta con almacenes específicos donde organiza y almacena los datos. Cada uno de estos almacenes se encuentra optimizado para diferentes tipos de datos y escenarios de supervisión, alineándose con los tres pilares de observancia: **métricas, logs y trazas**, junto con un almacén adicional para **cambios**:

- Métricas de Azure Monitor: Se centra en valores numéricos que representan diferentes aspectos de un sistema en un momento específico. Se trata de una base de datos diseñada para el análisis de series temporales. Las métricas se recogen a intervalos regulares y se pueden agregar o comparar para identificar tendencias a lo largo del tiempo. Este almacén admite tanto métricas nativas de Azure Monitor como métricas de Prometheus, una herramienta a la que nos referiremos en futuros módulos.
- Registros de Azure Monitor: Almacena eventos del sistema que pueden diferir en tipo y formato (logs), incluyendo texto estructurado o libre, siempre asociados a una fecha y hora. Azure Monitor reúne y almacena

datos de registro tanto estructurados como no estructurados, posibilitando su análisis y consulta en áreas de trabajo de log analytics, de las que nos ocuparemos a lo largo de este módulo.

- Trazas (seguimientos distribuidos): Este almacén facilita el seguimiento del recorrido de las solicitudes a través de varios servicios y componentes de un sistema distribuido. Los datos de seguimiento se recogen de aplicaciones instrumentadas y se almacenan de manera que permitan identificar problemas de rendimiento y solucionar errores en sistemas complejos.
- Cambios: Registra series de eventos relacionados con aplicaciones y recursos utilizando el servicio de análisis de cambios. Este almacén se basa en Azure Resource Graph para ayudar a comprender los impactos que ejercen en el sistema cambios específicos como las actualizaciones de código.

Asimismo, Azure Monitor ofrece la posibilidad de exportar estos datos para archivarlos de forma económica y a largo plazo en Azure Storage, lo que puede cobrar importancia de cara a la auditoría o al cumplimiento normativo. En el caso de entornos híbridos o de aquellos que integran SCOM MI (una versión de System Center Operations Manager adaptada para Azure), la información se guarda en una instancia de SQL Database.

(C) Visualización de datos de azure monitor

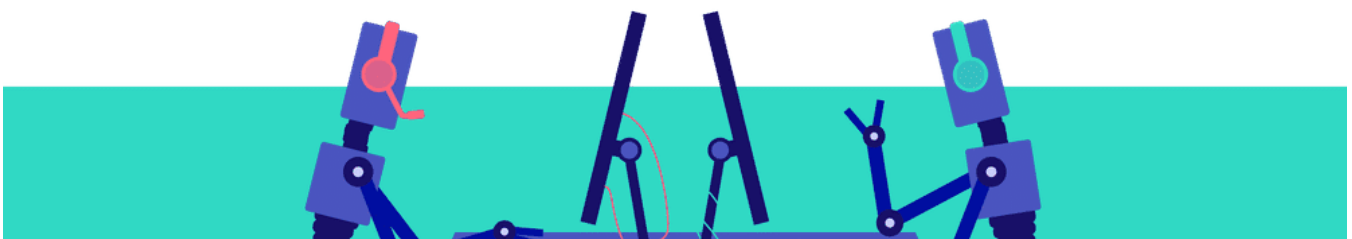
Azure Monitor proporciona una amplia gama de herramientas y opciones para visualizar datos de supervisión, lo que contribuye a resumir y presentar información crítica de manera efectiva a diferentes audiencias. En este sentido, recurre tanto a características propias como a integraciones con otros servicios de Azure y a herramientas externas para ofrecer visualizaciones interactivas y detalladas.

- Paneles de Azure (dashboards): Permiten agrupar diversos tipos de datos en un único panel dentro de Azure Portal, facilitando la visualización conjunta de métricas o registros de actividad, entre otros elementos. Estos paneles son personalizables y compatibles, posibilitando que los

usuarios puedan crear visualizaciones adaptadas a sus necesidades específicas, además de compartirlas con otros usuarios de Azure.

- Libros: Ofrecen un lienzo flexible de cara al análisis de datos y la creación de informes visuales completos dentro de Azure Portal. Los libros pueden consultar datos procedentes de múltiples fuentes para combinarlos en visualizaciones que simplifican la interpretación del estado y el rendimiento del sistema. Son interactivos, permiten la actualización de datos en tiempo real y pueden compartirse entre equipos, favoreciendo la colaboración y el análisis conjunto.
- Power BI: Como servicio de análisis empresarial, Power BI proporciona visualizaciones interactivas junto con la capacidad de integrar datos de múltiples fuentes. Dado que puede configurarse para importar datos de registro desde Azure Monitor, los usuarios cuentan con la posibilidad de aprovechar sus capacidades avanzadas de análisis y visualización para compartir métricas dentro y fuera de la organización.
- Grafana: Esta plataforma de código abierto se especializa en paneles operativos y se integra perfectamente con Azure Monitor de cara a la visualización de métricas y registros. Azure Managed Grafana mejora esta experiencia al optimizar la conexión con almacenes de datos de Azure y facilitar la visualización de datos de supervisión en paneles familiares. Además, Grafana soporta la integración con herramientas de monitorización externas y servicios de nube de terceros, permitiendo la supervisión unificada de múltiples nubes en un solo panel.

¿Sabes que es Azure Application Insights? Avanza para descubrirlo.



7.3.1 Azure Application Insights

Azure Application Insights es un **servicio de monitoreo de rendimiento de aplicaciones** (APM) integrado en la plataforma **Azure Monitor** y diseñado específicamente para desarrolladores de **aplicaciones web y servicios**. Este servicio proporciona una visión detallada del comportamiento de las aplicaciones, contribuyendo a detectar, diagnosticar y solucionar problemas de rendimiento y fallos en tiempo real, además de mejorar la experiencia del usuario al optimizar el rendimiento de la aplicación.

(A) Características clave

- Telemetría completa: Application Insights reúne una amplia gama de datos telemétricos, incluidos detalles de solicitudes, tiempos de respuesta, tasas de fallo, dependencias externas, rendimiento del sistema, excepciones y errores, eventos personalizados y trazas de diagnóstico.
- Análisis de rendimiento: Ofrece herramientas avanzadas para analizar el rendimiento de la aplicación, permitiendo a los desarrolladores identificar rápidamente cuellos de botella, ineficiencias y áreas problemáticas. Los datos recopilados se presentan en dashboards intuitivos y se pueden explorar mediante consultas personalizadas.
- Diagnóstico de fallos: Application Insights proporciona capacidades detalladas de diagnóstico de fallos, incluyendo el análisis de excepciones y errores. De esta forma, contribuye a que los desarrolladores exploren las causas raíz de los fallos mediante la inspección de trazas de pila y la correlación de eventos de fallo con cambios en el código o en la infraestructura.
- Monitoreo en tiempo real: Con soporte para alertas en tiempo real y dashboards en vivo, los equipos pueden monitorizar el estado y el

rendimiento de sus aplicaciones permanentemente, lo que posibilita una respuesta rápida ante cualquier incidente o degradación del rendimiento.

- Integración con el ciclo de desarrollo: Application Insights se integra perfectamente con herramientas de desarrollo y DevOps, incluyendo Visual Studio, Azure DevOps y GitHub. En consecuencia, facilita la incorporación del monitoreo y diagnóstico de aplicaciones al ciclo de vida del desarrollo de software, promoviendo prácticas de DevOps y entrega continua.
- Experiencia del usuario y análisis de uso: Junto con el monitoreo del rendimiento y los fallos, proporciona insights sobre cómo los usuarios interactúan con la aplicación, abarcando análisis de flujos, tasas de conversión y rendimiento de la página, lo que resulta esencial para optimizar la experiencia del usuario y dirigir las decisiones de desarrollo de productos.

(B) Beneficios

- Proactividad de cara a la resolución de problemas: Al proporcionar alertas tempranas sobre problemas de rendimiento y fallos, Application Insights permite a los equipos abordar los problemas antes de que repercutan en los usuarios finales.
- Mejora continua del rendimiento: La detallada telemetría y las herramientas analíticas ayudan a los equipos a entender el comportamiento de sus aplicaciones y a realizar ajustes continuos para optimizar el rendimiento.
- Comprensión de la experiencia del usuario: Al analizar cómo los usuarios interactúan con la aplicación, los equipos pueden adoptar decisiones informadas para mejorar su satisfacción y la usabilidad.

Azure Application Insights es una herramienta indispensable para cualquier equipo de desarrollo que busque asegurar y mejorar la calidad, el rendimiento y la fiabilidad de sus aplicaciones web y servicios. Dado que incorpora el monitoreo profundo y el análisis de rendimiento al proceso de

desarrollo, facilita la creación de aplicaciones robustas, centradas en el usuario y alineadas con las prácticas modernas de desarrollo de software y operaciones.

Dentro del contexto DevOps, Azure Application Insights se posiciona como una herramienta que favorece la colaboración entre desarrollo y operaciones al proporcionar una visión integral y en tiempo real del rendimiento y la salud de las aplicaciones.

Este enfoque permite a los equipos adoptar **prácticas de integración y despliegue continuos (CI/CD)** con una capa adicional de seguridad y eficiencia de cara a la supervisión y el diagnóstico de aplicaciones.

Puesto que automatiza la recolección de telemetría y proporciona análisis en tiempo real, permite a los equipos:

Detectar y solucionar problemas rápidamente	Optimizar el rendimiento de la aplicación	Automatizar alertas
Identificar errores, fallos y cuellos de botella en cuanto se despliegan en producción, reduciendo el tiempo de resolución.	Analizar patrones de uso y rendimiento para implementar mejoras continuas que optimicen la experiencia del usuario.	Configurar alertas basadas en métricas críticas para automatizar el proceso de notificación y respuesta ante incidentes.

Supongamos que un equipo de DevOps trabaja en una aplicación web crítica para el negocio y que utiliza Azure DevOps para automatizar sus pipelines de CI/CD, así como Azure Application Insights para el monitoreo continuo. Cada vez que se realiza un *commit* en el repositorio, se desencadena automáticamente un pipeline que, tras superar las pruebas automatizadas, despliega la aplicación en producción. Simultáneamente, Application Insights comienza a monitorear el rendimiento y la salud de la aplicación:



Detección automática de regresiones: tras un despliegue reciente, Application Insights detecta un incremento en los tiempos de respuesta y

una tasa anormalmente elevada de errores 500 (error interno en el servidor). Las alertas configuradas notifican inmediatamente al equipo esta circunstancia.

- Diagnóstico y rollback rápido: El equipo utiliza los datos de telemetría y las trazas de error proporcionadas por Application Insights para identificar rápidamente la causa: una nueva característica introducida presenta un bug no detectado en las pruebas. Utilizando las capacidades de integración con Azure DevOps, ejecutan un rollback a la versión anterior mientras trabajan en la solución.

Con los datos proporcionados por Application Insights, el equipo puede analizar tendencias a largo plazo, identificar áreas de mejora y optimizar recursos. Así, por ejemplo, si el análisis del uso de la aplicación revela patrones de carga pico inesperados, el equipo decide implementar escalado automático para gestionar mejor dichas cargas, mejorando así la disponibilidad y el rendimiento sin necesidad de intervención manual.

(C) Plataformas y marcos compatibles

Application Insights se integra perfectamente con numerosas plataformas y marcos, garantizando que las aplicaciones puedan monitorearse independientemente de dónde se alojen o cómo se implementen:

- Integración de servicios de Azure: Facilita la habilitación y la configuración directamente desde el portal de Azure y a través de implementaciones de Azure Resource Manager.
- Azure Virtual Machines y Virtual Machine Scale Sets, Azure App Service, funciones de Azure, Azure Spring Apps y Azure Cloud Services: Soporte integral para aplicaciones alojadas en estos servicios de Azure, incluidos los roles web y de trabajo.
- Marcos de registro: Application Insights puede recoger logs de varias fuentes, integrándose con marcos de registro populares como ILogger,

Log4Net, NLog, System.Diagnostics.Trace, Log4J, Logback, java.util.logging y el complemento LogStash.

Los datos reunidos por Application Insights se pueden exportar para análisis avanzados utilizando Azure Monitor para la supervisión en tiempo real y Power BI para visualizaciones y análisis detallados.

7.3.2 Azure Log Analytics

Azure Log Analytics representa un servicio esencial de Azure Monitor, ya que proporciona capacidades avanzadas para la recopilación, búsqueda y análisis de grandes volúmenes de datos de logs generados por aplicaciones, sistemas y servicios alojados en Azure.

De este modo, permite a desarrolladores, administradores de sistemas y profesionales de seguridad obtener *insights* valiosos sobre el funcionamiento y la salud de sus entornos de TI, facilitando la detección temprana de problemas, el análisis de tendencias y la toma de decisiones basadas en datos.

(A) Características claves

- Recopilación de datos: Azure Log Analytics puede reunir logs de una amplia variedad de fuentes, lo que comprende tanto recursos y servicios de Azure como máquinas virtuales, aplicaciones y entornos on-premise. Esta capacidad de recopilación universal proporciona a los usuarios una visión completa del comportamiento de sus sistemas y aplicaciones independientemente de dónde se encuentren alojados.
- Consultas: Mediante Kusto Query Language (KQL), Azure Log Analytics permite a los usuarios diseñar consultas complejas para extraer, filtrar y analizar los datos de logs. KQL es un lenguaje rico y flexible, diseñado específicamente para el análisis de datos de telemetría y logs, que facilita desde consultas simples para la visualización de datos hasta análisis avanzados de cara a la detección de anomalías y patrones.

- Integración: Azure Log Analytics se encuentra totalmente integrado con otros servicios y herramientas de Azure, proporcionando una solución de monitoreo y análisis cohesiva. Así, por ejemplo, se integra con Azure Security Center para mejorar la seguridad y el cumplimiento, con Azure Automation para la orquestación de respuestas a incidentes y con Azure Monitor para una visión centralizada del rendimiento y la salud de los recursos de Azure. Esta integración permite a los usuarios crear flujos de trabajo de diagnóstico y respuesta automatizados, mejorando la eficiencia operativa y la capacidad de reacción ante incidentes.

(B) Casos de uso

- Diagnóstico de problemas de rendimiento: Los administradores pueden recurrir a Log Analytics para investigar problemas de rendimiento en sus aplicaciones y servicios, analizando los logs en busca de patrones o anomalías que puedan indicar la causa raíz de los incidentes.
- Seguridad y cumplimiento: Los profesionales de seguridad pueden aprovechar Log Analytics para recoger y analizar logs de seguridad procedentes de múltiples fuentes, facilitando la detección de amenazas, la investigación de incidentes de seguridad y la auditoría de cara al cumplimiento de normativas.
- Optimización de recursos: Mediante el análisis de las tendencias y los patrones presentes en los datos de uso y rendimiento, los equipos pueden adoptar decisiones informadas sobre la optimización de recursos, como el escalado de servicios o la identificación de recursos infrautilizados.

Resumen

Este módulo sobre DevOps abarca una amplia gama de conceptos y herramientas clave para la implementación exitosa de prácticas DevOps. Comienza con una introducción a la infraestructura como código (IaC), una metodología fundamental que permite a los equipos describir y gestionar su infraestructura de manera programática. Se explica cómo la IaC impacta en los diferentes roles de la organización y se detalla su ciclo de vida, así como las mejores prácticas a seguir.

A continuación, hemos profundizado en Terraform, una herramienta de IaC ampliamente utilizada. Se cubren los fundamentos de Terraform, incluyendo su instalación, los diferentes tipos de bloques, los comandos básicos y avanzados, y la gestión del archivo de estado. También hemos abordado la estructura típica de un proyecto de Terraform, así como prácticas avanzadas como la automatización del flujo de trabajo, la gestión de estado, la modularización y la optimización de costes.

Se ha estudiado la automatización y orquestación de redes en el contexto DevOps. Destacando la importancia de la automatización de redes e introduciendo conceptos básicos de redes y mostrando cómo Terraform puede utilizarse para automatizar la creación de recursos de red, como redes virtuales, tablas de rutas y grupos de seguridad. Se abordan además consideraciones avanzadas, como la gestión segura de datos, el orden de las operaciones y la colaboración entre equipos.

En la segunda mitad del módulo nos hemos centrado en la **plataforma de nube pública Azure**. Proporcionando una visión general de Azure, incluyendo su historia, la estructura de regiones y zonas de disponibilidad, y la gestión de suscripciones y costes. También hemos explorado los principales recursos de Azure, como máquinas virtuales, almacenamiento, redes y bases de datos.

Posteriormente, hemos profundizado en los despliegues en Azure, cubriendo herramientas y estrategias, así como una introducción al Portal de Azure y a la CLI de Azure. Dedicando una sección a las plantillas de Azure Resource Manager (ARM) y al lenguaje Bicep, que permiten la automatización y gestión de infraestructuras en Azure.

Finalmente, hemos abarcado la plataforma **Azure DevOps Services**, que proporciona un conjunto integrado de herramientas para la gestión ágil de proyectos, el control de versiones, la automatización de la compilación y el despliegue, y las pruebas. Explorando también en detalle los diferentes componentes de Azure DevOps, como Azure Boards, Azure Repos, Azure Pipelines, Azure Artifacts y Azure Test Plans.

Como último punto hemos abordado con una sección sobre el monitoreo y el registro en Azure, destacando la importancia de estas prácticas, los tipos de datos recopilados (métricas, registros y trazas) y las herramientas de Azure, como Application Insights y Log Analytics.

Bibliografía

- Humble, J., y Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
- Joy, J. (2020). Azure for Architects: Create secure, scalable, high-availability applications on the cloud. Packt Publishing.
- Kief, J., y Portnoy, A. (2017). Terraform: Up & Running: Writing Infrastructure as Code. O'Reilly Media.
- Kim, G., Humble, J., Debois, P., y Willis, J. (2016). The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations. IT Revolution Press.
- Limoncelli, T. A., Hogan, C. J., y Chalup, S. R. (2016). The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems, Volume 2. Addison-Wesley Professional.
- Minasi, M., Gibson, R., Finn, C., Henry, W., y Hynes, B. (2019). Mastering Windows Server 2019. Sybex.
- Mitra, S. (2021). Mastering Azure: A Comprehensive Guide to Becoming an Azure Expert. Packt Publishing.
- Morris, K. (2016). Infrastructure as Code: Managing Servers in the Cloud. O'Reilly Media.

- Turnbull, J. (2018). The Ansible Book. Leanpub.
- Walls, M., y Smith, W. (2018). Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way. O'Reilly Media.

Referencias

- Microsoft Corporation (2020a). Exam Ref AZ-900 Microsoft Azure Fundamentals. Microsoft Press.
- Microsoft Corporation (2020b). Exam Ref AZ-204 Developing Solutions for Microsoft Azure. Microsoft Press.
- Microsoft Corporation. (2021). Exam Ref AZ-104 Microsoft Azure Administrator. Microsoft Press.

Webgrafia

- Amazon Web Services. (n.d.). AWS CloudFormation Documentation. Recuperado de <https://docs.aws.amazon.com/cloudformation/index.html>
- Amazon Web Services (AWS). (n.d.). Amazon VPC Documentation. Recuperado de <https://docs.aws.amazon.com/vpc/index.html>
- Atlassian. (n.d.). Jira Software Documentation. Recuperado de <https://confluence.atlassian.com/jirasoftware>
- DevOpsDays. (n.d.). DevOpsDays Events. Recuperado de <https://devopsdays.org/>
- HashiCorp. (n.d.). Terraform Documentation. Recuperado de <https://www.terraform.io/docs/index.html>
- Microsoft. (2023a). Azure Artifacts Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/artifacts/>

- Microsoft. (2023b). Azure Boards Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/boards/>
- Microsoft. (2023c). Azure DevOps Services Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/>
- Microsoft. (2023d). Azure Pipelines Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/pipelines/>
- Microsoft. (2023e). Azure Repos Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/repos/>
- Microsoft. (2023f). Azure Test Plans Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/devops/test-plans/>
- Microsoft Azure. (2021a). Azure Active Directory. Recuperado de <https://docs.microsoft.com/en-us/azure/active-directory/>
- Microsoft Azure. (2021b). Azure Application Insights overview. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>
- Microsoft Azure. (2021c). Azure Log Analytics overview. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/logs/log-analytics-overview>
- Microsoft Azure. (2021d). Azure Monitor alerts. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/alerts/alerts-overview>
- Microsoft Azure. (2021e). Azure Monitor data collection. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/essentials/data-sources>
- Microsoft Azure. (2021f). Azure Monitor integration. Recuperado de <https://docs.microsoft.com/en-us/azure/azuremonitor/essentials/integration>

- Microsoft Azure. (2021g). Azure Monitor logs. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/essentials/data-platform-logs>
- Microsoft Azure. (2021h). Azure Monitor logs alerts. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/logs/alerts>
- Microsoft Azure. (2021i). Azure Monitor logs data sources. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/logs/data-sources>
- Microsoft Azure. (2021j). Azure Monitor logs integration. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/logs/integration>
- Microsoft Azure. (2021k). Azure Monitor logs queries. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/logs/log-queries>
- Microsoft Azure. (2021l). Azure Monitor metrics. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/essentials/data-platform-metrics>
- Microsoft Azure. (2021m). Azure Monitor overview. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/overview>
- Microsoft Azure. (2021n). Azure Monitor traces. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-monitor/essentials/data-platform-traces>
- Microsoft Azure. (2021o). What is Azure Monitor?. Recuperado de <https://azure.microsoft.com/en-us/services/monitor/>
- Microsoft Azure. (2024a). Azure CLI. Recuperado de <https://docs.microsoft.com/en-us/cli/azure/>

- Microsoft Azure. (2024b). Azure Container Instances. Recuperado de <https://docs.microsoft.com/en-us/azure/container-instances/>
- Microsoft Azure. (2024c). Azure Portal. Recuperado de <https://portal.azure.com>
- Microsoft Azure. (2024d). Azure Resource Manager Templates. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-resource-manager/templates/>
- Microsoft Azure. (2024e). Bicep Language for ARM Templates. Recuperado de <https://docs.microsoft.com/en-us/azure/azure-resource-manager/bicep/>
- Microsoft Azure. (n.d.). Azure Networking. Recuperado de <https://docs.microsoft.com/en-us/azure/networking/>
- Microsoft Azure. (n.d.). Azure SQL Database. Recuperado de <https://docs.microsoft.com/en-us/azure/sql-database/>
- Microsoft Azure. (n.d.). Azure Storage. Recuperado de <https://docs.microsoft.com/en-us/azure/storage/>
- Microsoft Azure. (n.d.). Azure Virtual Machines. Recuperado de <https://docs.microsoft.com/en-us/azure/virtual-machines/>
- Microsoft Azure. (n.d.). Azure Virtual Network Documentation. Recuperado de <https://docs.microsoft.com/en-us/azure/virtual-network/>
- Microsoft Azure. (n.d.). Regions and Availability Zones in Azure. Recuperado de <https://docs.microsoft.com/en-us/azure/availability-zones/az-overview>
- Microsoft Azure. (n.d.). What is Azure?. Recuperado de <https://azure.microsoft.com/en-us/overview/what-is-azure/>

- Red Hat. (n.d.). Ansible Documentation. Recuperado de <https://docs.ansible.com/>
- The Agile Alliance. (n.d.). The Agile Manifesto. Recuperado de <https://agilemanifesto.org/>